
Objetos e Morfismos em MATLAB

Relatório de Matemática Discreta (Pós-laboral)

Grupo Nº: 2

David Domingues (2022125309)

Diana Gomes (2019101181)

Guilherme Baltazar (2025209893)

Joana Venâncio (2016123347)

Leonardo Costa (2023137263)

Marcelo Marques (2022102560)

Miguel Reis (2024164633)

Escola Superior de Tecnologia e Gestão
Instituto Politécnico de Leiria

2026

Índice

1	Introdução	7
2	Modelação de Categorias	8
2.1	Fin (Conjuntos Finitos e Aplicações Totais)	8
2.1.1	Objetos	8
2.1.2	Morfismos	8
2.2	Sub (Conjuntos Finitos e Inclusões)	8
2.2.1	Objetos	8
2.2.2	Morfismos	9
2.3	Par (Aplicações Parciais)	9
2.3.1	Objetos	9
2.3.2	Morfismos	9
2.4	Simp (Categoria Simplicial)	9
2.4.1	Objetos	9
2.4.2	Morfismos	9
2.5	End (Endomaps)	10
2.5.1	Objetos	10
2.5.2	Morfismos	10
2.6	Graph (Directed Graphs)	10
2.6.1	Objetos	10
2.6.2	Morfismos	11
2.7	Rel (Relations)	11
2.7.1	Objetos	11
2.7.2	Morfismos	11
2.8	PreOrd (Preorders)	12
2.8.1	Objetos	12
2.8.2	Morfismos	12
2.9	Partial Orders (Posets)	12
2.9.1	Objetos	13
2.9.2	Morfismos	13
2.10	EqRel (Relações de Equivalência)	13
2.10.1	Objetos	13
2.10.2	Morfismos	14
2.11	Mon (Monoides Finitos)	14
2.11.1	Objetos	14
2.11.2	Morfismos	14
2.12	Mag (Magmas Finitos)	14
2.12.1	Objetos	15

2.12.2	Morfismos	15
2.13	DIMag (Dimagmas Finitos)	15
2.13.1	Objetos	15
2.13.2	Morfismos	15
2.14	Row (Matrizes Linha)	16
2.14.1	Objetos	16
2.14.2	Morfismos	16
2.15	Lat (Reticulados Finitos)	16
2.15.1	Objetos	16
2.15.2	Morfismos	17
2.16	DLat (Latices Distributivos)	17
2.16.1	Objetos	18
2.16.2	Morfismos	18
2.17	Bool (Álgebras de Boolean)	18
2.17.1	Objetos	18
2.17.2	Morfismos	18
2.18	OrdMon (Monoides Ordenados)	18
2.18.1	Objetos	18
2.18.2	Morfismos	19
2.19	Trees (Árvores Binárias)	19
2.19.1	Objetos	19
2.19.2	Morfismos	20
2.20	Turing Machines (Máquinas de Turing)	20
2.20.1	Objetos	20
2.20.2	Morfismos	21
2.21	Struct (Conjuntos Estruturados)	22
2.21.1	Objetos	22
2.21.2	Morfismos	23
2.22	Categoria FinSet: Conjuntos Finitos e Funções	23
2.22.1	Objetos e Invariantes de Representação	24
2.22.2	Morfismos	24
2.23	Construções Universais em FinSet	24
2.23.1	Produto e Coproduto	25
2.23.2	Equalizador e Coequalizador	25
2.23.3	Pullback e Pushout	25
2.24	Aplicações: Hash, Homologia e Magmas	25
2.24.1	Funções Hash como Morfismos	25
2.24.2	Homologia de Endomapas	25
2.24.3	Functor de Endomapas para Magmas	26
2.25	Resultados e Verificação Computacional	26
3	Construções e Funtores	27
3.1	Construções em Fin	27
3.1.1	Fundamentação Teórica	27
3.1.2	Implementação Computacional	27
3.2	Construções em Sub	28
3.2.1	Fundamentação Teórica	28
3.2.2	Implementação Computacional	28

3.3	Construções em Par	29
3.3.1	Fundamentação Teórica	29
3.3.2	Implementação Computacional	29
3.4	Construções em Simp	29
3.4.1	Fundamentação Teórica	29
3.4.2	Implementação Computacional	30
3.5	End (Endomaps)	30
3.5.1	Fundamentação Teórica	30
3.5.2	Implementação Computacional	30
3.6	Graph (Directed Graphs)	31
3.6.1	Fundamentação Teórica	31
3.6.2	Implementação Computacional	31
3.7	Rel (Relations)	31
3.7.1	Fundamentação Teórica	32
3.7.2	Implementação Computacional	32
3.8	PreOrd (Preorders)	32
3.8.1	Fundamentação Teórica	32
3.8.2	Implementação Computacional	32
3.9	Partial Orders (Posets)	33
3.9.1	Fundamentação Teórica	33
3.9.2	Implementação Computacional	33
3.10	EqRel (Relações de Equivalência)	33
3.10.1	Fundamentação Teórica	33
3.10.2	Implementação Computacional	34
3.11	Mon (Monoides Finitos)	34
3.11.1	Fundamentação Teórica	34
3.11.2	Implementação Computacional	34
3.12	Construções em Mag (Magmas Finitos)	35
3.12.1	Fundamentação Teórica	35
3.12.2	Implementação Computacional	35
3.13	Construções em DIMag (Dimagmas Finitos)	36
3.13.1	Fundamentação Teórica	36
3.13.2	Implementação Computacional	36
3.14	Construções em Row (Matrizes Linha)	37
3.14.1	Fundamentação Teórica	37
3.14.2	Implementação Computacional	38
3.15	Lat (Reticulados Finitos)	38
3.15.1	Fundamentação Teórica	39
3.15.2	Implementação Computacional	39
3.16	Bool (Álgebras de Boolean)	40
3.16.1	Fundamentação Teórica	40
3.16.2	Implementação Computacional	40
3.17	OrdMon (Monoides Ordenados)	40
3.17.1	Fundamentação Teórica	40
3.17.2	Implementação Computacional	41
3.18	Trees (Árvores Binárias)	41
3.18.1	Fundamentação Teórica	41
3.18.2	Implementação Computacional	42

3.19	Turing Machines (Máquinas de Turing)	43
3.19.1	Fundamentação Teórica	43
3.19.2	Implementação Computacional	44
3.20	Struct (Conjuntos Estruturados)	44
3.20.1	Fundamentação Teórica	45
3.20.2	Implementação Computacional	45
3.21	FinSet (Conjuntos Finitos como Matrizes Linha)	47
3.21.1	Objetos	47
3.21.2	Morfismos	48
3.21.3	Funções Hash como Morfismos de FinSet	49
3.21.4	Homologia de Endomapas vs. Funções Hash	51
3.21.5	Hash vs. Homologia: Análise Comparativa	52
3.21.6	Estrutura de Magma a partir de Endomapas	53
3.21.7	Classes de Conjugação em $\text{End}(A)$	54
3.22	Construções em FinSet	55
3.22.1	Fundamentação Teórica	55
3.22.2	Implementação Computacional	55
3.23	Functor $F : \text{Fin} \rightarrow \text{Vect}$	57
3.23.1	Descrição das Categorias	57
3.23.2	Functor Covariante $F : \text{Fin} \rightarrow \text{Vect}$ (Functor Livre)	58
3.23.3	Functor de Esquecimento $U : \text{Vect} \rightarrow \text{Fin}$	58
3.23.4	Transformações Naturais $\eta : F \Rightarrow G$	59
3.24	Functor $P : \text{Fin} \rightarrow \text{Sub}$	59
3.24.1	Descrição das Categorias	59
3.24.2	Functor Covariante $P : \text{Fin} \rightarrow \text{Sub}$ (Conjunto das Partes)	59
3.24.3	Functor de Esquecimento $U : \text{Sub} \rightarrow \text{Fin}$	60
3.24.4	Functor Contravariante $P^* : \text{Fin}^{\text{op}} \rightarrow \text{Sub}$ (Imagem Inversa)	60
3.24.5	Transformações Naturais entre Funtores $\text{Fin} \rightarrow \text{Sub}$	60
3.25	Functor $I : \text{Fin} \rightarrow \text{Par}$	60
3.25.1	Descrição das Categorias	60
3.25.2	Composição em Par	61
3.25.3	Functor de Inclusão $I : \text{Fin} \rightarrow \text{Par}$	61
3.25.4	Transformações Naturais entre Funtores $\text{Fin} \rightarrow \text{Par}$	61
3.26	Functor $F : \text{Sub} \rightarrow \text{Rel}$	62
3.26.1	Descrição das Categorias	62
3.26.2	Functor Covariante $F : \text{Sub} \rightarrow \text{Rel}$	62
3.26.3	Transformações Naturais $\alpha : F_1 \Rightarrow F_2$	63
3.27	Functor $G : \text{Par} \rightarrow \text{Rel}$	63
3.27.1	Descrição das Categorias	63
3.27.2	Functor Covariante $G : \text{Par} \rightarrow \text{Rel}$	64
3.27.3	Transformações Naturais $\beta : G_1 \Rightarrow G_2$	65
3.28	Functor $H : \text{End} \rightarrow \text{Graph}$	65
3.28.1	Descrição das Categorias	65
3.28.2	Functor Covariante $H : \text{End} \rightarrow \text{Graph}$	66
3.28.3	Transformações Naturais $\gamma : H_1 \Rightarrow H_2$	67
3.29	Functor $F : \text{Rel} \rightarrow \text{Fin}$	67
3.29.1	Fundamentação Teórica	67
3.29.2	Functor Covariante $F : \text{Rel} \rightarrow \text{Fin}$	68

3.29.3	Functor Contravariante — Análise de Existência	69
3.29.4	Transformações Naturais	69
3.30	Functor $F : \mathbf{Simp} \rightarrow \mathbf{PreOrd}$	70
3.30.1	Fundamentação Teórica	70
3.30.2	Functor Covariante $F : \mathbf{Simp} \rightarrow \mathbf{PreOrd}$	70
3.30.3	Functor Contravariante — Análise de Existência	71
3.30.4	Transformações Naturais	71
3.31	Functor $F : \mathbf{Graph} \rightarrow \mathbf{Rel}$	72
3.31.1	Fundamentação Teórica	72
3.31.2	Functor Covariante $F : \mathbf{Graph} \rightarrow \mathbf{Rel}$	72
3.31.3	Functor Contravariante — Análise de Existência	73
3.31.4	Transformações Naturais	74
3.32	Functor $F : \mathbf{Rel} \rightarrow \mathbf{PreOrd}$	74
3.32.1	Descrição das Categorias	74
3.32.2	Functor Covariante $F : \mathbf{Rel} \rightarrow \mathbf{PreOrd}$	75
3.32.3	Transformações Naturais	75
3.33	Functor $G : \mathbf{Trees} \rightarrow \mathbf{Struct}$	75
3.33.1	Descrição das Categorias	75
3.33.2	Functor Covariante $G : \mathbf{Trees} \rightarrow \mathbf{Struct}$ (Codificação como Conjunto Estruturado)	76
3.33.3	Transformações Naturais	76
3.34	Functor $H : \mathbf{End} \rightarrow \mathbf{Struct}$	76
3.34.1	Descrição das Categorias	76
3.34.2	Functor Covariante $H : \mathbf{End} \rightarrow \mathbf{Struct}$ (Endomorfismo como Conjunto Estruturado)	77
3.34.3	Transformações Naturais	77
3.35	Functor $U_1, U_2 : \mathbf{DIMag} \rightarrow \mathbf{Mag}$	77
3.35.1	Descrição das Categorias	77
3.35.2	Funtores Covariantes $U_1, U_2 : \mathbf{DIMag} \rightarrow \mathbf{Mag}$	78
3.35.3	Functor Diagonal $\Delta : \mathbf{Mag} \rightarrow \mathbf{DIMag}$	78
3.35.4	Transformações Naturais $\eta : U_1 \Rightarrow U_2$	79
3.36	Functor $I : \mathbf{DLat} \hookrightarrow \mathbf{Lat}$ e $D : \mathbf{Lat} \rightarrow \mathbf{DLat}$	79
3.36.1	Descrição das Categorias	79
3.36.2	Functor de Inclusão $I : \mathbf{DLat} \hookrightarrow \mathbf{Lat}$	80
3.36.3	Reflector Distributivo $D : \mathbf{Lat} \rightarrow \mathbf{DLat}$	81
3.36.4	Transformações Naturais $q : \text{Id}_{\mathbf{Lat}} \Rightarrow I \circ D$	81
3.37	Functor $Z : \mathbf{Lat} \rightarrow \mathbf{Bool}$	82
3.37.1	Descrição das Categorias	82
3.37.2	Functor Covariante $Z : \mathbf{DLat} \rightarrow \mathbf{Bool}$ (Centro Booleano)	83
3.37.3	Functor de Esquecimento $J : \mathbf{Bool} \rightarrow \mathbf{DLat}$	84
3.37.4	Transformações Naturais $\iota : J \circ Z \Rightarrow \text{Id}_{\mathbf{DLat}}$	84
3.38	Functor $F : \mathbf{TuringMachines} \rightarrow \mathbf{Struct}$	85
3.38.1	Fundamentação Teórica	85
3.38.2	Functor Covariante $F : \mathbf{TuringMachines} \rightarrow \mathbf{Struct}$	85
3.38.3	Functor Contravariante — Análise de Existência	85
3.38.4	Transformações Naturais	86
3.39	Functor $F : \mathbf{End} \rightarrow \mathbf{Mon}$	86
3.39.1	Fundamentação Teórica	86

3.39.2	Funtor Covariante $F : End \rightarrow Mon$	86
3.39.3	Funtor Contravariante — Análise de Existência	87
3.39.4	Transformações Naturais	87
3.40	Funtor $F : End \rightarrow Trees$	88
3.40.1	Fundamentação Teórica	88
3.40.2	Funtor Covariante $F : End \rightarrow Trees$	89
3.40.3	Funtor Contravariante — Análise de Existência	89
3.40.4	Transformações Naturais	89
3.41	Funtor $F : End \rightarrow TuringMachines$	90
3.41.1	Fundamentação Teórica	90
3.41.2	Funtor Covariante $F : End \rightarrow TuringMachines$	90
3.41.3	Funtor Contravariante — Análise de Existência	91
3.41.4	Transformações Naturais	91

Capítulo 1

Introdução

Este relatório apresenta a modelação computacional de estruturas da Teoria das Categorias, focando-se na implementação de objetos, morfismos e construções universais em ambiente MATLAB/Octave.

Capítulo 2

Modelação de Categorias

Nesta secção, detalhamos como cada estrutura matemática é representada através de matrizes, structs ou funções.

2.1 Fin (Conjuntos Finitos e Aplicações Totais)

Nesta categoria, os objetos são conjuntos finitos e os morfismos são funções totais, onde todas as entradas do domínio têm uma imagem definida no codomínio.

2.1.1 Objetos

Os objetos são representados pela sua cardinalidade, um número inteiro não negativo n . O conjunto é tratado implicitamente como $\{1, 2, \dots, n\}$.

2.1.2 Morfismos

Um morfismo $f : n \rightarrow m$ é representado por um vetor linha de comprimento n , onde cada elemento é um inteiro entre 1 e m .

Listing 2.1:]Exemplo de Morfismo: $f = [2, 1, 3]$

```
% Representa f(1)=2, f(2)=1, f(3)=3  
f = [2, 1, 3];
```

2.2 Sub (Conjuntos Finitos e Inclusões)

Esta categoria forma um conjunto parcialmente ordenado (Poset), pois existe no máximo um morfismo (a inclusão) entre quaisquer dois objetos.

2.2.1 Objetos

Os objetos são vetores de elementos únicos, como por exemplo $A = [1, 3, 5]$.

2.2.2 Morfismos

O morfismo $A \hookrightarrow B$ é validado através de uma condição booleana que confirma se A é um subconjunto de B .

Listing 2.2: Verificação de Inclusão

```
is_morphism = all(ismember(A, B));
```

2.3 Par (Aplicações Parciais)

Semelhante a **Fin**, mas as funções podem não estar definidas para todos os elementos do domínio.

2.3.1 Objetos

Os objetos são definidos pela cardinalidade n do conjunto.

2.3.2 Morfismos

Representados por vetores de comprimento n , utilizando o valor 0 (ou NaN) para elementos onde a função é indefinida.

Listing 2.3: Morfismo Parcial

```
% 0 valor 0 indica que o elemento não tem imagem  
f = [1, 0, 2];
```

2.4 Simp (Categoria Simplicial)

Os objetos são conjuntos finitos linearmente ordenados e os morfismos são funções monótonas não decrescentes.

2.4.1 Objetos

O objeto $[n]$ representa o conjunto $\{0, 1, \dots, n\}$, modelado em MATLAB pelo inteiro n .

2.4.2 Morfismos

Morfismos $f : [n] \rightarrow [m]$ são vetores de comprimento $n + 1$ com valores de 0 a m . A sequência deve ser rigorosamente ordenada.

Listing 2.4: Validação Simplicial

```
is_valid = issorted(f);
```

2.5 End (Endomaps)

A categoria de endomapas (ou sistemas dinâmicos discretos autônomos) foca-se na evolução de estados. Um objeto é um conjunto finito munido de uma função (endomorfismo) que mapeia o conjunto para si mesmo, $\alpha : X \rightarrow X$.

2.5.1 Objetos

Dado que a relação é estritamente uma função (cada elemento tem exatamente uma imagem), a representação mais eficiente em MATLAB não é uma matriz, mas sim um vetor de índices, o que otimiza significativamente o uso de memória e tempo de acesso.

Listing 2.5: Representação de um Endomapa

```
% Vetor representando f(1)=2, f(2)=3, f(3)=1 (Um ciclo de
    comprimento 3)
E = [2, 3, 1];
```

2.5.2 Morfismos

Os morfismos são funções equivariantes que "comutam" com a dinâmica. Um mapeamento $f : (X, \alpha) \rightarrow (Y, \beta)$ é válido se aplicar a dinâmica e depois mapear for igual a mapear e depois aplicar a dinâmica do codomínio: $f(\alpha(x)) = \beta(f(x))$.

Listing 2.6: Verificação do Quadrado Comutativo

```
% f é o morfismo; EA e EB são os endomapas
isMorphism = true;
for x = 1:length(f)
    % Verifica se comuta: f(alpha(x)) == beta(f(x))
    if f(EA(x)) ~= EB(f(x))
        isMorphism = false;
        break;
    end
end
end
```

2.6 Graph (Directed Graphs)

A categoria de grafos direcionados é essencial para modelar redes, fluxos e transições de estado. Um objeto nesta categoria é constituído por um conjunto de vértices e arestas com direcionalidade explícita.

2.6.1 Objetos

Em MATLAB, os grafos são naturalmente modelados pelas suas matrizes de adjacência. O valor da matriz na posição (i, j) indica a presença de uma aresta direcionada do nó i para o nó j .

Listing 2.7: Representação de um Grafo Direcionado

```
% Matriz de adjacência (Vértices = 3, Arestas = 2)
```

```
n = 3;
M = [0 1 0; 0 0 0; 0 1 0];
```

2.6.2 Morfismos

Os morfismos são homomorfismos de grafos: funções entre vértices que mapeiam arestas válidas no domínio para arestas válidas no codomínio. Se existe uma aresta $i \rightarrow j$, tem de existir uma aresta $f(i) \rightarrow f(j)$.

Listing 2.8: Verificação de Homomorfismo de Grafos

```
% f mapeia vértices do domínio para o codomínio
isHomomorphism = true;
[I, J] = find(M_domain);
for k = 1:length(I)
    if ~M_codomain(f(I(k)), f(J(k)))
        isHomomorphism = false;
        break;
    end
end
```

2.7 Rel (Relations)

A categoria de relações é a base para modelar qualquer tipo de conectividade genérica ou base de dados relacionais. Um objeto nesta categoria define-se por um conjunto finito e uma relação binária arbitrária (sem exigência de reflexividade, simetria ou transitividade).

2.7.1 Objetos

No ambiente MATLAB, representamos estas estruturas através de matrizes de adjacência booleanas simples. A ausência de restrições algébricas torna a sua instanciação direta e computacionalmente leve.

Listing 2.9: Representação de uma Relação Arbitrária

```
% Matriz de adjacência para uma relação genérica
n = 3;
M = [0 1 0;
     0 0 1; 1 0 0];
```

2.7.2 Morfismos

Os morfismos nesta categoria são funções que preservam a relação. Um mapeamento f é válido se, para quaisquer elementos i, j , a existência de uma relação $i \sim j$ implicar a relação $f(i) \sim f(j)$.

Listing 2.10: Algoritmo de Verificação de Preservação Relacional

```
% f é o vetor de mapeamento
isMorphism = true;
```

```
[I, J] = find(M_domain);
for k = 1:length(I)
    if ~M_codomain(f(I(k)), f(J(k)))
        isMorphism = false;
        break;
    end
end
```

2.8 PreOrd (Preorders)

A categoria de pré-ordens é fundamental para modelar hierarquias e sistemas de estados onde ciclos ou equivalências são permitidos. Um objeto nesta categoria define-se por um conjunto finito e uma relação binária que respeita a reflexividade e a transitividade, não exigindo antissimetria.

2.8.1 Objetos

No ambiente MATLAB, a forma mais eficiente de representar estas relações, tal como nos Posets, é através de matrizes de adjacência booleanas. Esta estrutura permite-nos validar as propriedades algébricas de reflexividade e transitividade recorrendo a álgebra matricial.

Listing 2.11: Representação de uma Pré-ordem (1 e 2 são equivalentes, 3 é maior)

```
% Matriz de adjacência para uma pré-ordem
n = 3;
M = [1 1 1; 1 1 1; 0 0 1];
```

2.8.2 Morfismos

Os morfismos nesta categoria são funções monótonas (preservam a ordem). Um mapeamento f é válido se, para quaisquer elementos i, j , a condição $i \leq j$ implicar obrigatoriamente $f(i) \leq f(j)$.

Listing 2.12: Algoritmo de Verificação de Monotonia para PreOrd

```
% f é um vetor onde o índice é o domínio e o valor é a imagem
isMorphism = true;
[I, J] = find(M_domain);
for k = 1:length(I)
    if ~M_codomain(f(I(k)), f(J(k)))
        isMorphism = false;
        break;
    end
end
```

2.9 Partial Orders (Posets)

A categoria de ordens parciais é fundamental para modelar hierarquias e dependências funcionais. Um objeto nesta categoria define-se por um conjunto finito e uma relação binária que respeita a reflexividade, antissimetria e transitividade.

2.9.1 Objetos

No ambiente MATLAB, a forma mais eficiente de representar estas relações é através de matrizes de adjacência booleanas. Esta estrutura permite-nos validar propriedades algébricas complexas utilizando operações de álgebra matricial.

Listing 2.13: Representação de um Poset ($1 \leq 2 \leq 3$)

```
% Matriz de adjacência para uma ordem linear
n = 3;
M = [1 1 1;
      0 1 1;
      0 0 1];
```

2.9.2 Morfismos

Os morfismos nesta categoria são funções monótonas. Um mapeamento f é válido se, para quaisquer elementos i, j , a condição $i \leq j$ implicar obrigatoriamente $f(i) \leq f(j)$.

Listing 2.14: Algoritmo de Verificação de Monotonia

```
% f é um vetor onde o índice é o domínio e o valor é a imagem
f = [1, 2, 2];

isMorphism = true;
[I, J] = find(M_domain);
for k = 1:length(I)
    if ~M_codomain(f(I(k)), f(J(k)))
        isMorphism = false; break;
    end
end
```

2.10 EqRel (Relações de Equivalência)

A categoria **EqRel** foca-se na partição de conjuntos em classes disjuntas. Ao contrário das ordens, estas relações são simétricas, sendo essenciais para modelar abstrações onde diferentes estados de um sistema são considerados indistinguíveis.

2.10.1 Objetos

Os objetos são representados por matrizes de adjacência simétricas. A principal característica computacional é que a matriz deve ser igual à sua transposta, garantindo que se $a \sim b$, então $b \sim a$.

Listing 2.15: Matriz Simétrica de Equivalência

```
% Exemplo: Duas classes de equivalência {1,2} e {3}
R = [1 1 0;
      1 1 0;
      0 0 1];
```

2.10.2 Morfismos

Um morfismo em **EqRel** é uma função que preserva a vizinhança: se dois elementos pertencem à mesma classe no domínio, as suas imagens devem pertencer à mesma classe no contradomínio.

Listing 2.16: Teste de Preservação de Classe

```
% f mapeia os elementos do conjunto original
f = [1, 1, 2];

% A condição R_dom(i,j) == 1 deve implicar R_codom(f(i), f(j)) == 1
```

2.11 Mon (Monoides Finitos)

Os monoides representam a estrutura básica de composição e computação sequencial. Um monoide combina um conjunto com uma operação binária que deve ser associativa e possuir um elemento neutro (identidade).

2.11.1 Objetos

A modelação computacional de monoides finitos é realizada através de Tabelas de Cayley. Esta matriz armazena todos os resultados possíveis da operação binária entre os elementos do conjunto.

Listing 2.17: Tabela de Cayley para Monoide C2

```
% Elementos: 1 (Identidade), 2
Tabela = [1 2;
          2 1];
e = 1; % Índice do elemento neutro
```

2.11.2 Morfismos

Os morfismos de monoides (homomorfismos) devem preservar a estrutura operacional: a imagem do produto de dois elementos deve ser igual ao produto das suas imagens, e a identidade deve ser mapeada na identidade.

Listing 2.18: Validação de Homomorfismo

```
f = [1, 2];
% 1. Verificar se f(e_A) == e_B
% 2. Verificar se f(TabelaA(i,j)) == TabelaB(f(i), f(j))
```

2.12 Mag (Magmas Finitos)

Nesta categoria, os objetos são magmas finitos, ou seja, conjuntos finitos equipados com uma operação binária sem axiomas adicionais. Os morfismos são funções que preservam a operação binária.

2.12.1 Objetos

Os objetos são representados por um conjunto finito e uma operação binária. O conjunto é tratado explicitamente como um vetor de elementos, e a operação é uma função binária em MATLAB/Octave.

2.12.2 Morfismos

Um morfismo $f : M_1 \rightarrow M_2$ é uma função que preserva a operação binária, ou seja, $f(a \cdot b) = f(a) * f(b)$ para todo $a, b \in M_1$. É representado como uma função em MATLAB/Octave que mapeia elementos de M_1 para M_2 .

Listing 2.19: Exemplo de Magma e Morfismo

```
% Definição de um magma M1
M1.set = [1, 2, 3];
M1.op = @(a, b) mod(a + b, 3);

% Definição de um magma M2
M2.set = [1, 2];
M2.op = @(a, b) mod(a * b, 2);

% Morfismo f: M1 -> M2
f = @(x) mod(x, 2) + 1;
```

2.13 DIMag (Dimagmas Finitos)

Nesta categoria, os objetos são dimagmas finitos, ou seja, conjuntos finitos equipados com duas operações binárias. Os morfismos são funções que preservam ambas as operações.

2.13.1 Objetos

Os objetos são representados por um conjunto finito e duas operações binárias. O conjunto é tratado explicitamente como um vetor de elementos, e as operações são funções binárias em MATLAB/Octave.

2.13.2 Morfismos

Um morfismo $f : D_1 \rightarrow D_2$ é uma função que preserva ambas as operações binárias, ou seja, $f(a \cdot b) = f(a) \odot f(b)$ e $f(a * b) = f(a) \otimes f(b)$ para todo $a, b \in D_1$. É representado como uma função em MATLAB/Octave que mapeia elementos de D_1 para D_2 .

Listing 2.20: Exemplo de Dimagma e Morfismo

```
1 % Definição de um dimagma D1
2 D1.set = [1, 2, 3];
3 D1.op1 = @(a, b) mod(a + b, 3);
4 D1.op2 = @(a, b) mod(a * b, 3);
5
6 % Definição de um dimagma D2
7 D2.set = [1, 2];
```



```

8      D2.op1 = @(a, b) mod(a + b, 2);
9      D2.op2 = @(a, b) mod(a * b, 2);
10
11     % Morfismo f: D1 -> D2
12     f = @(x) mod(x, 2) + 1;

```

2.14 Row (Matrizes Linha)

Nesta categoria, os objetos são matrizes linha, interpretadas como coleções de vetores. Os morfismos são aplicações lineares compatíveis com a estrutura de matrizes linha.

2.14.1 Objetos

Os objetos são representados como matrizes linha, onde cada linha é um vetor.

2.14.2 Morfismos

Um morfismo $f : R_1 \rightarrow R_2$ é uma aplicação linear representada por uma matriz A de dimensão $|R_2| \times |R_1|$, tal que $f(v) = A \cdot v$ para todo $v \in R_1$.

Listing 2.21: Exemplo de Matriz Linha e Morfismo

```

1     % Definição de uma matriz linha R1
2     R1.vectors = [1 2; 3 4];
3
4     % Definição de uma matriz linha R2
5     R2.vectors = [1 0; 0 1];
6
7     % Morfismo f: R1 -> R2 (matriz identidade)
8     A = [1 0; 0 1];

```

2.15 Lat (Reticulados Finitos)

A categoria dos reticulados finitos (**Lat**) estuda conjuntos equipados com duas operações binárias: o ínfimo (*meet*, $\wedge$) e o supremo (*join*, $\vee$). Estas operações satisfazem os axiomas da comutatividade, associatividade, absorção e idempotência.

2.15.1 Objetos

No ambiente MATLAB, como trabalhamos com conjuntos finitos de tamanho n , a forma mais eficaz de representar um reticulado é através de duas matrizes $n \times n$. Cada matriz atua como uma tabela de pesquisa (*look-up table*), onde a entrada (i, j) guarda o resultado da operação entre os elementos i e j .

Listing 2.22: Representação de um Reticulado (Diamante de 4 elementos)

```

% Reticulado com 4 elementos: 1 (Fundo), 2 e 3 (Incomparaveis), 4 (Topo)
n = 4;

```

```

% Matriz da operacao meet (infimo)
M_meet = [1 1 1 1;
          1 2 1 2;
          1 1 3 3;
          1 2 3 4];

% Matriz da operacao join (supremo)
M_join = [1 2 3 4;
          2 2 4 4;
          3 4 3 4;
          4 4 4 4];

```

2.15.2 Morfismos

Os morfismos em **Lat** não precisam apenas de preservar a ordem (como acontece na categoria dos Posets); eles exigem uma preservação algébrica estrita. Um mapeamento f é um homomorfismo de reticulados se, e só se, preservar ambas as operações para quaisquer elementos i, j : $f(i \wedge j) = f(i) \wedge f(j)$ e $f(i \vee j) = f(i) \vee f(j)$.

Listing 2.23: Algoritmo de Verificacao de Homomorfismo de Reticulados

```

% f e um vetor onde o indice e o dominio e o valor e a imagem
f = [1, 4, 4, 4]; % Exemplo de um mapeamento (colapsa tudo no
                  Topo)

isMorphism = true;
for i = 1:n_domain
    for j = 1:n_domain
        % Verifica preservacao do meet (infimo)
        if f(M_meet_domain(i, j)) ~= M_meet_codomain(f(i), f(j))
            isMorphism = false; break;
        end

        % Verifica preservacao do join (supremo)
        if f(M_join_domain(i, j)) ~= M_join_codomain(f(i), f(j))
            isMorphism = false; break;
        end
    end
    if ~isMorphism, break; end
end

```

2.16 DLat (Latices Distributivos)

Os latices distributivos são estruturas onde qualquer par de elementos possui um ínfimo e um supremo, e onde estas operações distribuem-se entre si. São fundamentais para o estudo de álgebras de lógica e conjuntos.

2.16.1 Objetos

Utilizamos uma abordagem de programação funcional em MATLAB, recorrendo a estruturas que armazenam os "handles" das operações de *meet* ($\wedge$) e *join* ($\vee$).

Listing 2.24: Estrutura Funcional de Lattice

```
L.elements = {1, 2, 3, 4};  
L.meet = @(a, b) min(a, b);  
L.join = @(a, b) max(a, b);
```

2.16.2 Morfismos

Um morfismo em **DLat** é uma função que preserva ambas as operações binárias. A validação exige testar a igualdade $f(a \wedge b) = f(a) \wedge f(b)$ para todos os pares do domínio.

2.17 Bool (Álgebras de Boolean)

As Álgebras de Boolean são a base da lógica computacional moderna. Estas estruturas estendem os lattices distributivos ao introduzir a noção de complemento e elementos extremos (zero e um).

2.17.1 Objetos

A modelação em MATLAB é feita expandindo a estrutura de **DLat** com a operação de negação lógica e a definição explícita das constantes de topo e base.

Listing 2.25: Operações em Álgebra Booleana

```
B.elements = {0, 1};  
B.not = @(a) 1 - a;  
B.and = @(a, b) min(a, b);  
B.or = @(a, b) max(a, b);
```

2.17.2 Morfismos

Os morfismos nesta categoria devem ser compatíveis com a negação, a conjunção e a disjunção, além de mapear corretamente os elementos neutros do sistema.

2.18 OrdMon (Monoides Ordenados)

A categoria **OrdMon** une álgebra e ordem. Um monoide ordenado exige que a operação binária seja compatível com a relação de ordem, ou seja, se $a \leq b$, então $a * c \leq b * c$.

2.18.1 Objetos

A representação destes objetos requer uma estrutura híbrida que combine a Tabela de Cayley (para a parte algébrica) com uma matriz de adjacência (para a componente de ordem).

Listing 2.26: Modelação Híbrida de OrdMon

```
OM.elements = {1, 2, 3};
OM.mul = @(a, b) a + b;
OM.unit = 1;
OM.le = @(a, b) a <= b;
```

2.18.2 Morfismos

Os morfismos em **OrdMon** são os que satisfazem simultaneamente as condições de homomorfismo de monoides e de monotonia de ordens parciais.

2.19 Trees (Árvores Binárias)

A categoria **Trees** é composta por árvores binárias finitas como objetos, onde a estrutura é definida pela hierarquia entre nós e pela lateralidade das ramificações. Esta categoria é essencial para modelar processos de decisão dicotómicos e estruturas de dados hierárquicas onde a distinção entre esquerda e direita é semanticamente relevante.

2.19.1 Objetos

No ambiente MATLAB/Octave, os objetos desta categoria podem ser representados de duas formas complementares, dependendo da operação ou construção categorial a realizar:

1. Representação por Array de Progenitores: Esta forma utiliza um vetor **parent** (onde **parent(i)** é o índice do pai do nó *i*) e um vetor **side** (onde 0 representa o filho esquerdo e 1 o filho direito). É a representação mais eficiente para validar propriedades de morfismos e calcular limites.

Listing 2.27: Representação por Array de Progenitores

```
% Arvore com 7 nos: raiz=1, filhos esquerdos/direitos indicados
%
%      1
%     / \
%    2   3
%   / \ / \
%  4  5 6  7

n      = 7;
parent = [0, 1, 1, 2, 2, 3, 3]; % parent(1)=0 -> raiz
side   = [-1, 0, 1, 0, 1, 0, 1]; % -1=raiz, 0=esq, 1=dir

tree.n      = n;
tree.parent = parent;
tree.side   = side;
```

2. Representação Recursiva: Tal como explorado no script original, esta representação define a árvore como uma estrutura (*struct*) que contém um valor e referências para as suas subárvores. Esta abordagem é fundamental para a definição de construções como o Produto de árvores, facilitando a navegação descendente.

Listing 2.28: Representação Recursiva de uma Árvore Binária

```
% Definicao de um no com subarvores
node.value = 10;      % Conteudo informativo do no
node.left  = [];      % Subarvore esquerda (vazia se [])
node.right = [];      % Subarvore direita (vazia se [])

% Exemplo de construcao de uma arvore de altura 1
T.value = 1;
T.left  = struct('value', 2, 'left', [], 'right', []);
T.right = struct('value', 3, 'left', [], 'right', []);
```

2.19.2 Morfismos

Os morfismos $f : T_1 \rightarrow T_2$ nesta categoria são mapeamentos entre os conjuntos de nós que preservam estritamente a raiz, a relação de parentesco e a lateralidade. Para que f seja um morfismo válido, a imagem do pai de um nó em T_1 deve coincidir com o pai da imagem desse nó em T_2 , mantendo-se a posição relativa (lado).

Listing 2.29: Algoritmo de Verificação de Morfismo de Árvores

```
function ok = is_tree_morphism(T1, T2, f)
    ok = true;

    % 1) Verificacao da Raiz: a imagem da raiz deve ser a raiz
    r1 = find(T1.parent == 0);
    r2 = find(T2.parent == 0);
    if f(r1) ~= r2, ok = false; return; end

    % 2) Preservacao de Estrutura e Lateralidade
    for i = 1:T1.n
        if T1.parent(i) ~= 0
            p1 = T1.parent(i);
            % 0 pai da imagem deve ser a imagem do pai
            if T2.parent(f(i)) ~= f(p1), ok = false; return; end
            % 0 lado (esquerdo/direito) deve ser preservado
            if T2.side(f(i)) ~= T1.side(i), ok = false; return;
            end
        end
    end
end
```

2.20 Turing Machines (Máquinas de Turing)

A categoria **Turing Machines** permite modelar processos computacionais através de máquinas abstratas que manipulam símbolos numa fita.

2.20.1 Objetos

Os objetos desta categoria são máquinas de Turing (determinísticas) finitas. Uma máquina de Turing $M = (Q, \Gamma, \delta, q_0, F, B)$ é codificada em MATLAB/Octave como uma *struct*

composta pelos seguintes elementos:

- Q — número de estados ($|Q|$), identificados por inteiros $1, \dots, |Q|$;
- Γ — vetor de símbolos de fita (o símbolo em branco é o índice 1 por convenção);
- δ — matriz de transição de dimensão $|Q| \times |\Gamma|$, onde $\delta(q, a)$ é um vetor $[q', a', d]$ com: q' = novo estado, a' = símbolo escrito, $d \in \{-1, 0, 1\}$ = direção;
- q_0 — estado inicial;
- F — vetor de estados finais/aceitação.

Listing 2.30: Representação de uma Máquina de Turing

```
% Exemplo: MT que aceita a^n b^n (simplificado)
M.Q      = 5;                % estados 1..5
M.Gamma  = [0, 1, 2, 3];    % 0=branco, 1='a', 2='b', 3='X'
M.blank  = 1;                % indice do branco em Gamma
M.q0     = 1;                % estado inicial
M.F      = [5];              % estados de aceitacao

% delta(q, sym) = [new_q, new_sym, dir]   dir: -1=L, 0=N, 1=R
M.delta = zeros(M.Q, numel(M.Gamma), 3);
% Transicoes (exemplo parcial):
M.delta(1, 2, :) = [2, 4, 1]; % q1, 'a' -> escreve 'X', vai
    direita, q2
M.delta(2, 2, :) = [2, 2, 1]; % q2, 'a' -> fica 'a', vai direita
    , q2
M.delta(2, 3, :) = [3, 4, -1]; % q2, 'b' -> escreve 'X', vai
    esquerda, q3
% ... (restantes transicoes omitidas por brevidade)
```

2.20.2 Morfismos

Um morfismo $f : M_1 \rightarrow M_2$ é um par de mapas (f_Q, f_Γ) — sobre os conjuntos de estados e de símbolos de fita, respetivamente — que preservam a função de transição, o estado inicial, os estados finais, e o símbolo em branco. Formalmente, devem ser satisfeitas as seguintes condições:

1. $f_Q(q_0^{(1)}) = q_0^{(2)}$ (preservação do estado inicial);
2. $f_Q(F_1) \subseteq F_2$ (preservação dos estados finais);
3. $f_\Gamma(B_1) = B_2$ (preservação do símbolo em branco);
4. Para cada $(q, a) \in Q_1 \times \Gamma_1$, se $\delta_1(q, a) = (q', a', d)$, então $\delta_2(f_Q(q), f_\Gamma(a)) = (f_Q(q'), f_\Gamma(a'), d)$.

Listing 2.31: Verificação de um morfismo de Máquinas de Turing

```
function ok = is_TM_morphism(M1, M2, fQ, fGamma)
    % fQ: vetor de tamanho M1.Q (fQ(q) em {1..M2.Q})
    % fGamma: vetor de tamanho numel(M1.Gamma)
    ok = true;

    % 1) Estado inicial
    if fQ(M1.q0) ~= M2.q0
        ok = false; return;
    end

    % 2) Estados finais
    for q = M1.F
        if ~ismember(fQ(q), M2.F)
            ok = false; return;
        end
    end

    % 3) Simbolo em branco
    if fGamma(M1.blank) ~= M2.blank
        ok = false; return;
    end

    % 4) Funcao de transicao
    for q = 1:M1.Q
        for a = 1: numel(M1.Gamma)
            tr = squeeze(M1.delta(q, a, :))';
            q2 = tr(1); a2 = tr(2); d = tr(3);
            tr2 = squeeze(M2.delta(fQ(q), fGamma(a), :))';
            if tr2(1) ~= fQ(q2) || tr2(2) ~= fGamma(a2) || tr2(3)
                ~= d
                ok = false; return;
            end
        end
    end
end
```

2.21 Struct (Conjuntos Estruturados)

A categoria **Struct** permite modelar coleções de dados onde cada elemento possui um valor associado e uma relação de sucessão definida por um endomapa.

2.21.1 Objetos

Os objetos da categoria **Struct** são triplos (d, φ, n) , onde:

- $n \in \mathbb{N}$ é o número de elementos do conjunto;
- $\varphi : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$ é um endomapa que codifica a estrutura interna do conjunto;

- $d: \{1, \dots, n\} \rightarrow \mathbb{C}$ é um mapa nos números complexos que descreve (possivelmente com repetição) os elementos armazenados.

No ambiente MATLAB/Octave, estes objetos são representados por uma *struct* com os campos correspondentes aos componentes do triplo:

Listing 2.32: Representação de um objeto de Struct

```
% Objeto S = (d, phi, n)
S.n    = 4;
S.phi  = [2, 3, 3, 1];          % endomapa phi: {1,2,3,4} ->
    {1,2,3,4}
S.d     = [1+0i, 2+1i, 0+0i, 3-1i]; % mapa d: {1..4} -> C

% Interpretacao:
% phi(1)=2, phi(2)=3, phi(3)=3, phi(4)=1
% d(1)=1, d(2)=2+i, d(3)=0, d(4)=3-i
```

2.21.2 Morfismos

Um morfismo $f: (d_1, \varphi_1, n_1) \rightarrow (d_2, \varphi_2, n_2)$ é um mapa $f: \{1, \dots, n_1\} \rightarrow \{1, \dots, n_2\}$ tal que as seguintes condições de compatibilidade são satisfeitas:

$$f \circ \varphi_1 = \varphi_2 \circ f \quad \text{e} \quad d_1 = d_2 \circ f.$$

A primeira condição garante que a estrutura de sucessão é preservada, enquanto a segunda garante a preservação dos dados associados a cada elemento.

Listing 2.33: Verificação de um morfismo de Struct

```
function ok = is_struct_morphism(S1, S2, f)
% f e um vetor de indices onde f(i) e a imagem do elemento i
ok = true;

for i = 1:S1.n
    % 1) Verificacao da condicao f(phi1(i)) == phi2(f(i))
    if f(S1.phi(i)) ~= S2.phi(f(i))
        ok = false; return;
    end

    % 2) Verificacao da condicao d1(i) == d2(f(i))
    if abs(S1.d(i) - S2.d(f(i))) > 1e-10 % Tolerancia para
        complexos
        ok = false; return;
    end
end
end
```

2.22 Categoria FinSet: Conjuntos Finitos e Funções

A categoria **FinSet** é fundamental na teoria das categorias, onde os objetos são conjuntos finitos e os morfismos são funções entre eles. Computacionalmente, modelar esta categoria

exige a definição de invariantes de representação rigorosos e algoritmos que respeitem as suas propriedades estruturais.

2.22.1 Objetos e Invariantes de Representação

Em MATLAB, um objeto da categoria `FinSet` é representado como uma matriz plana. Para garantir a definição matemática de um conjunto (elementos únicos e sem ordem intrínseca relevante para a igualdade), define-se o seguinte invariante de representação: uma matriz A é um objeto de `FinSet` se e só se as suas linhas forem únicas e estiverem ordenadas lexicograficamente.

Listing 2.34: Definição do invariante de objetos em `FinSet`

```
% Um objeto de FinSet é uma matriz MATLAB A tal que:
% isequal(A, unique(A,'rows')) == true
is_FinSet_object = @(X) isequal(X, sortrows(unique(X,'rows')));
```

Esta abordagem permite modelar diversos tipos de conjuntos:

- **Conjuntos escalares:** Vetores coluna de números inteiros.
- **Conjuntos de strings:** Matrizes de caracteres (*char arrays*).
- **Produtos cartesianos:** Matrizes onde cada coluna representa uma dimensão do produto (e.g., pares de coordenadas).

2.22.2 Morfismos

Um morfismo $f : A \rightarrow B$ em `FinSet` é uma função total. No nosso modelo, um morfismo é implementado como um vetor coluna de índices, onde a i -ésima posição contém o índice do elemento em B para o qual o i -ésimo elemento de A é mapeado.

Listing 2.35: Validação de um morfismo em `FinSet`

```
function ok = is_FinSet_morphism(B, f, A)
    % f deve ter o mesmo tamanho que A e apontar para índices
    % válidos em B
    f = f(:);
    ok = (numel(f) == size(A,1)) && ...
        (nnz(f) == size(A,1)) && ...
        all(f >= 1) && ...
        all(f <= size(B,1));
end
```

A composição de morfismos $g \circ f$ traduz-se numa operação de indexação direta simples e eficiente no MATLAB: $h = g(f)$, respeitando integralmente a associatividade requerida pelos axiomas das categorias.

2.23 Construções Universais em `FinSet`

Uma das características essenciais de `FinSet` é ser completa e cocompleta, isto é, possuir todos os limites e colimites finitos.

2.23.1 Produto e Coproduto

O **produto categórico** ($A \times B$) consiste nos pares ordenados de elementos. Computacionalmente, geramos todas as combinações recorrendo à função `ndgrid` e concatenamos as linhas, garantindo a ordenação final.

O **coproduto** ($A \sqcup B$), ou união disjunta, requer a "etiquetagem" dos elementos para evitar a colisão de elementos iguais oriundos de origens distintas. Adicionamos uma coluna com o valor 0 para os elementos de A e 1 para os elementos de B .

Listing 2.36: Produto e Coproduto Categórico

```
function [C, iota_A, iota_B] = FinSet_coproduct(A, B)
    nA = size(A,1); nB = size(B,1);
    tagged = [zeros(nA,1), A; ones(nB,1), B];
    [C, ord] = sortrows(tagged);
    % Identificacao das inclusoes (morfismos de A e B para C)
    [~, iota_A] = ismember([zeros(nA,1), A], C, 'rows');
    [~, iota_B] = ismember([ones(nB,1), B], C, 'rows');
end
```

2.23.2 Equalizador e Coequalizador

O **equalizador** de duas funções $f, g : A \rightarrow B$ seleciona o subconjunto de A cujos elementos têm a mesma imagem sob f e g . No MATLAB, isto resolve-se de forma vetorizada com `all(B(f,:) == B(g,:), 2)`.

O **coequalizador** é um colimite que forma o espaço quociente sobre a relação de equivalência gerada por $f(a) \sim g(a)$. A sua implementação computacional exige algoritmos de grafos, especificamente a estrutura de dados *Union-Find* com *path compression*, garantindo a determinação eficiente das classes de equivalência.

2.23.3 Pullback e Pushout

Utilizando as construções primárias, o modelo suporta **pullbacks** (limites de diagramas em "V") e **pushouts** (colimites de diagramas em "V" invertido). O pushout é elegantemente construído através da combinação do coproduto seguido do coequalizador.

2.24 Aplicações: Hash, Homologia e Magmas

2.24.1 Funções Hash como Morfismos

No contexto de estruturas de dados, uma função *hash* pode ser interpretada como um morfismo não injetivo $h : U \rightarrow M$ na categoria `FinSet`, onde U é o universo de chaves e M os *buckets*. As colisões ocorrem inevitavelmente, sendo uma consequência direta da perda de injetividade (teorema de Dirichlet). O script implementa diferentes métodos morfismos: hash por divisão, multiplicação (Knuth) e polinomial para *strings*.

2.24.2 Homologia de Endomapas

Um endomapa é um morfismo $\phi : A \rightarrow A$. Pode ser visualizado como um grafo direcionado onde cada vértice tem grau de saída exato de 1. O código analisa a "homologia" deste

espaço, determinando para cada componente conexa:

1. Número de elementos terminais (*tails*).
2. Vértices de ramificação (grau de entrada ≥ 2).
3. O comprimento invariável do ciclo central.

Esta assinatura atua como um invariante para **classes de conjugação**. Dois endomaps só podem ser conjugados (isto é, isomórficos sob uma bijeção estrutural) se partilharem a mesma matriz de homologia.

Listing 2.37: Detecção estrutural de invariantes (Homologia)

```
function same = same_homology(phi1, phi2)
    H1 = endomap_homology(phi1);
    H2 = endomap_homology(phi2);
    same = isequal(sortrows(H1), sortrows(H2));
end
```

2.24.3 Funtor de Endomaps para Magmas

O guião demonstra a passagem a categorias algébricas construindo um Magma $(A, *)$ a partir do endomapa ϕ . A operação binária é sintetizada pela iteração em níveis do mapa, culminando na verificação de functorialidade, na qual se prova que um morfismo entre endomaps é preservado como um morfismo na categoria dos magmas (**Mag**), gerando tabelas de Cayley consistentes.

2.25 Resultados e Verificação Computacional

A simulação comprova empiricamente a teoria abstrata:

- A composição de morfismos respeitou a associatividade estrita.
- O produto cardinal provou ser idêntico à multiplicação matemática regular ($|A \times B| = |A| \times |B|$).
- O coproduto demonstrou ser equivalente à adição matemática ($|A \sqcup B| = |A| + |B|$).
- A construção passo-a-passo garantiu que a **FinSet** em ambiente MATLAB é simultaneamente *completa* e *cocompleta*, com todas as estruturas de limite e colimite criadas a operar conforme os axiomas universais em tempo real.

Capítulo 3

Construções e Funtores

Aqui demonstramos a capacidade de gerar novas estruturas (limites e colimites) e a relação entre diferentes categorias.

3.1 Construções em **Fin**

A categoria **Fin** é completa e cocompleta, o que permite a existência de todos os limites e colimites finitos. A implementação em MATLAB utiliza predominantemente operações sobre vetores e matrizes de índices.

3.1.1 Fundamentação Teórica

- **Produto** ($A \times B$): O objeto resultante tem cardinalidade $n \times m$. As projeções são geradas para cobrir todas as combinações do produto cartesiano.
- **Coproduto** ($A \sqcup B$): União disjunta com objeto $n + m$. As injeções mapeiam o primeiro conjunto para o início do novo intervalo e o segundo para o restante.
- **Equalizador**: O subconjunto do domínio onde $f(i) = g(i)$.
- **Coequalizador**: Espaço quociente da relação $f(i) \sim g(i)$. É modelado computacionalmente através de componentes conexas de um grafo onde as arestas ligam as imagens dos morfismos.
- **Pullback**: O conjunto de pares (x, y) que satisfazem a equação $f(x) = g(y)$.

3.1.2 Implementação Computacional

Listing 3.1: Construções Universais em **Fin**

```
% Produto e Projeções
[P1, P2] = ndgrid(1:n, 1:m);
p_A = P1(:)'; p_B = P2(:)';

% Equalizador
eq_map = find(f == g);
eq_obj = length(eq_map);
```

```
% Pullback (f: A -> C, g: B -> C)
[X, Y] = ndgrid(1:length(dom_f), 1:length(dom_g));
pb_indices = find(f(X) == g(Y));

% Coequalizador (Esquema de componentes conexas)
% Requer a criação de um grafo com arestas (f(i), g(i))
% e cálculo de 'conncomp' para identificar as classes.
```

3.2 Construções em Sub

Na categoria **Sub**, onde os objetos são subconjuntos de um universo comum e os morfismos são inclusões, as construções universais simplificam-se em operações de álgebra de conjuntos.

3.2.1 Fundamentação Teórica

- **Produto e Pullback:** Ambos correspondem à intersecção de conjuntos. No caso do Pullback, dadas inclusões de A e B em C , o resultado é o subconjunto comum máximo $A \cap B$.
- **Coproduto e Pushout:** Correspondem à união de conjuntos. O Pushout, dadas inclusões de um domínio comum C em A e B , resulta na união $A \cup B$, que "cola" as duas estruturas.
- **Equalizador:** Como existe no máximo um morfismo entre dois objetos, morfismos paralelos só existem se forem idênticos. O equalizador é, portanto, o próprio domínio.

3.2.2 Implementação Computacional

A implementação utiliza as funções nativas de conjuntos do MATLAB para garantir que a propriedade de unicidade de elementos nos objetos seja preservada.

Listing 3.2: Operações de Conjuntos em Sub

```
% 1. Produto Categórico (Interseção)
% Exemplo: A = {1, 2, 3}, B = {2, 3, 4} -> A intersect B = {2, 3}
A = [1, 2, 3]; B = [2, 3, 4];
Prod = intersect(A, B);

% 2. Coproduto Categórico (União)
% Exemplo: A = {1, 2}, B = {3, 4} -> A union B = {1, 2, 3, 4}
A_cop = [1, 2]; B_cop = [3, 4];
Coproduct = union(A_cop, B_cop);

% 3. Pullback (Interseção num codomínio comum C)
% Exemplo: C = {1..5}, A = {1,2,3}, B = {3,4,5} -> PB = {3}
C = [1, 2, 3, 4, 5];
A_pb = [1, 2, 3]; B_pb = [3, 4, 5];
PB = intersect(A_pb, B_pb);

% 4. Pushout (União com base num domínio comum C)
```

```
% Exemplo: C = {2}, A = {1,2}, B = {2,3} -> P0 = {1, 2, 3}
C_po = [2];
A_po = [1, 2]; B_po = [2, 3];
P0 = union(A_po, B_po);

% 5. Equalizador
% Para inclusões idênticas f, g: A -> B, o eq é o domínio A
Eq = A;
```

3.3 Construções em Par

As construções em **Par** (Aplicações Parciais) diferem significativamente de **Fin**, especialmente no que toca ao produto.

3.3.1 Fundamentação Teórica

- **Equalizador:** Identifica onde f e g coincidem (ambos definidos e iguais) ou onde ambos são indefinidos (0 ou NaN).
- **Coproducto:** Funciona como em **Fin**, através da união disjunta das cardinalidades.
- **Produto Categórico:** O produto clássico falha nesta categoria. O objeto produto correto é a união disjunta $A \sqcup B \sqcup (A \times B)$, refletindo a possibilidade de uma componente ser indefinida enquanto a outra não é.

3.3.2 Implementação Computacional

Listing 3.3: Equalizador e Objeto Produto em Par

```
% Equalizador Parcial
% f e g sao vetores onde 0 representa 'indefinido'
eq_map = find((f == g & f ~= 0) | (f == 0 & g == 0));
eq_obj = length(eq_map);

% Objeto Produto (Calculo da cardinalidade)
% n = card(A), m = card(B)
prod_obj_size = n + m + (n * m);
```

3.4 Construções em Simp

A categoria simplicial **Simp** é restritiva devido à exigência de que os morfismos preservem a ordem linear absoluta.

3.4.1 Fundamentação Teórica

- **Equalizador:** É a única construção universal básica que existe sempre em **Simp**. Consiste no subconjunto de índices onde os morfismos coincidem, o qual herda naturalmente a ordem do domínio.

- **Limites e Colimites (Produto, Coproduto, Pullback, Pushout):** Não existem em geral nesta categoria. O produto cartesiano de ordens lineares gera uma grelha (ordem parcial) que não é necessariamente uma ordem linear, falhando a propriedade universal.

3.4.2 Implementação Computacional

Listing 3.4: Cálculo do Equalizador Simplicial

```
% f e g sao morfismos [n] -> [m] (vetores ordenados)
idx = find(f == g); % Encontra posicoes de coincidencia
if ~isempty(idx)
    k = length(idx) - 1; % Novo objeto [k]
    eq_morphism = idx - 1; % Mapeamento para base 0
end
```

3.5 End (Endomaps)

As construções na categoria de endomaps focam-se na preservação da evolução de estados. O foco principal é garantir que as novas estruturas criadas mantenham uma dinâmica válida, onde a "comutatividade" da evolução do sistema original é respeitada.

3.5.1 Fundamentação Teórica

As principais construções universais implementadas para esta categoria são:

- **Produtos:** O produto $X \times Y$ atua em paralelo com o endomapa produto definido por $\alpha \times \beta(x, y) = (\alpha(x), \beta(y))$.
- **Equalizadores:** O subconjunto de estados invariantes face aos morfismos concorrentes, munido da restrição da dinâmica original.

3.5.2 Implementação Computacional

Em vez do produto de Kronecker, iteramos e indexamos linearmente a combinação de vetores para construir o endomapa produto diretamente num vetor unidimensional.

Listing 3.5: Construção do Produto Paralelo de Endomaps

```
function P = endo_product(EA, EB)
    % EA e EB são vetores de endomaps
    nA = length(EA);
    nB = length(EB);
    P_vector = zeros(1, nA * nB);

    for i = 1:nA
        for j = 1:nB
            % Cálculo do índice no produto cartesiano
            idx = (i-1)*nB + j;
            % Onde (EA(i), EB(j)) cai no novo vetor referencial
```

```

        val = (EA(i)-1)*nB + EB(j);
        P_vector(idx) = val;
    end
end

P.vector = P_vector;
P.n_elements = nA * nB;
end

```

3.6 Graph (Directed Graphs)

As construções na categoria de grafos direcionados baseiam-se na preservação da estrutura de incidência. O foco principal é garantir que a direcionalidade explícita das arestas originais seja devidamente transportada para as novas redes criadas.

3.6.1 Fundamentação Teórica

As principais construções universais implementadas para esta categoria são:

- **Produtos:** O produto tensorial de grafos, onde os vértices são pares de vértices originais, e existe uma aresta entre (a, b) e (a', b') se $a \rightarrow a'$ e $b \rightarrow b'$.
- **Equalizadores:** O subgrafo induzido pelo conjunto de vértices onde as funções (homomorfismos paralelos) concordam.

3.6.2 Implementação Computacional

A implementação matemática via matrizes garante uma avaliação computacional imediata do produto tensorial.

Listing 3.6: Construção do Produto Tensorial de Grafos

```

function G = graph_product(MA, MB)
    % Produto tensorial de grafos
    G_matrix = kron(MA, MB);
    G.matrix = G_matrix;
    G.n_vertices = size(G_matrix, 1);
end

```

3.7 Rel (Relations)

As construções na categoria de relações focam-se em combinar e intersestar conexões genéricas. O foco principal é garantir que a conectividade relacional seja transportada de forma consistente para as novas estruturas, mesmo na ausência de restrições algébricas adicionais.

3.7.1 Fundamentação Teórica

As principais construções universais implementadas para Rel são:

- **Produtos:** O produto cartesiano $A \times B$ equipado com a relação onde $(a, b) \sim (a', b')$ se e só se $a \sim_A a'$ e $b \sim_B b'$.
- **Equalizadores:** Sub-relação correspondente aos elementos onde os dois morfismos paralelos são iguais.

3.7.2 Implementação Computacional

O produto tensorial (ou de Kronecker) aplica-se perfeitamente para combinar estas relações de forma vetorial em MATLAB.

Listing 3.7: Construção do Produto de Relações

```
function P = rel_product(MA, MB)
    % Produto relacional via produto de Kronecker
    P_matrix = kron(MA, MB);
    P.matrix = P_matrix;
    P.n_elements = size(P_matrix, 1);
end
```

3.8 PreOrd (Preorders)

As construções na categoria de pré-ordens baseiam-se na preservação da relação de ordem onde ciclos e equivalências podem coexistir. O foco principal é garantir que as novas estruturas criadas mantenham rigidamente as propriedades de reflexividade e transitividade originais.

3.8.1 Fundamentação Teórica

As principais construções universais implementadas para esta categoria são:

- **Produtos:** O produto cartesiano $A \times B$ equipado com a ordem do produto, onde $(a, b) \leq (a', b')$ se e só se $a \leq_A a'$ e $b \leq_B b'$.
- **Equalizadores:** Dado um par de morfismos, o equalizador identifica o sub-conjunto onde as funções coincidem, herdando a estrutura de pré-ordem do domínio original.

3.8.2 Implementação Computacional

Em MATLAB, à semelhança dos Posets, a construção do produto de pré-ordens pode ser otimizada recorrendo ao produto de Kronecker das matrizes de adjacência.

Listing 3.8: Construção do Produto de Pré-ordens

```
function P = preord_product(MA, MB)
    % MA e MB são matrizes de adjacência de pré-ordens
    P_matrix = kron(MA, MB);
    % O resultado é a matriz de adjacência da nova Pré-ordem
```

```
P.matrix = P_matrix;
P.n_elements = size(P_matrix, 1);
end
```

3.9 Partial Orders (Posets)

As construções na categoria de ordens parciais baseiam-se na preservação da relação de ordem entre elementos. O foco principal é garantir que as novas estruturas criadas mantenham as propriedades de reflexividade, antissimetria e transitividade.

3.9.1 Fundamentação Teórica

As principais construções universais implementadas para esta categoria são:

- **Produtos:** O produto cartesiano $A \times B$ equipado com a ordem do produto, onde $(a, b) \leq (a', b')$ se e só se $a \leq_A a'$ e $b \leq_B b'$.
- **Equalizadores:** Dado um par de morfismos, o equalizador identifica o sub-poset onde as funções coincidem, herdando a ordem do domínio original.

3.9.2 Implementação Computacional

Em MATLAB, a construção do produto de ordens parciais pode ser otimizada recorrendo ao produto de Kronecker das matrizes de adjacência.

Listing 3.9: Construção do Produto de Posets

```
function P = poset_product(MA, MB)
% MA e MB são as matrizes de adjacência booleana das ordens
% O produto de Kronecker combina as relações corretamente
P_matrix = kron(MA, MB);

% O resultado é a matriz de adjacência do novo Poset produto
P.matrix = P_matrix;
P.n_elements = size(P_matrix, 1);
end
```

3.10 EqRel (Relações de Equivalência)

As construções em **EqRel** lidam com a manipulação de classes de equivalência para formar novos conjuntos quocientes ou uniões estruturadas.

3.10.1 Fundamentação Teórica

Nesta categoria, destacam-se as seguintes construções:

- **Produtos:** O produto de duas relações R_A e R_B relaciona pares de elementos se as suas respetivas componentes estiverem relacionadas nas estruturas originais.
- **Coprodutos:** Definidos pela união disjunta das relações, onde não existe relação cruzada entre elementos de conjuntos diferentes.

3.10.2 Implementação Computacional

A implementação do produto baseia-se na interseção das relações estendidas ao novo espaço cartesiano.

Listing 3.10: Produto de Relações de Equivalência

```
function R_prod = eqrel_product(RA, RB)
    % RA e RB são matrizes simétricas de equivalência
    % O produto preserva a estrutura de blocos das classes
    R_prod = kron(RA, RB);
end
```

3.11 Mon (Monoides Finitos)

Para que uma construção em **Mon** seja válida, o objeto resultante deve manter a integridade da operação binária (associatividade) e garantir a existência de uma identidade global.

3.11.1 Fundamentação Teórica

- **Produtos:** O produto de monoides define a operação componente a componente: $(m, n) * (m', n') = (m * m', n * n')$.
- **Equalizadores:** Corresponde ao sub-monoide que contém todos os elementos onde dois homomorfismos produzem o mesmo resultado.

3.11.2 Implementação Computacional

A construção do produto exige a geração de uma nova Tabela de Cayley expandida.

Listing 3.11: Criação da Tabela de Cayley para Produto de Monoides

```
function MonP = monoid_product(MA, idA, MB, idB)
    na = size(MA,1); nb = size(MB,1);
    MonP.table = zeros(na*nb, na*nb);

    % Construção da tabela via mapeamento de índices lineares
    for i=1:na*nb
        for j=1:na*nb
            [ia, ib] = ind2sub([na, nb], i);
            [ja, jb] = ind2sub([na, nb], j);
            % Operação componente a componente
            resA = MA(ia, ja);
            resB = MB(ib, jb);
            MonP.table(i,j) = sub2ind([na, nb], resA, resB);
        end
    end
    MonP.identity = sub2ind([na, nb], idA, idB);
end
```

3.12 Construções em Mag (Magmas Finitos)

A categoria **Mag** consiste em magmas finitos, ou seja, conjuntos finitos equipados com uma operação binária. Esta categoria é completa e cocompleta, permitindo a existência de todos os limites e colimites finitos.

3.12.1 Fundamentação Teórica

- **Produto** ($A \times B$): O objeto resultante é o produto cartesiano dos conjuntos subjacentes, com a operação binária definida componente a componente: $(a_1, b_1) \cdot (a_2, b_2) = (a_1 \cdot_A a_2, b_1 \cdot_B b_2)$. As projeções são as projeções canônicas do produto cartesiano.
- **Coproduto** ($A \sqcup B$): União disjunta dos conjuntos subjacentes. A operação binária é definida por casos, preservando a estrutura de cada magma nos subconjuntos correspondentes.
- **Equalizador**: Subconjunto do domínio onde os morfismos f e g coincidem, equipado com a operação restrita.
- **Coequalizador**: Espaço quociente do domínio pela relação de equivalência gerada por $f(a) \sim g(a)$, com a operação induzida no quociente.
- **Pullback**: Subconjunto de $A \times B$ onde $f(a) = g(b)$, com a operação definida componente a componente.
- **Pushout**: Construído como o quociente da união disjunta $A \sqcup B$ pela relação de equivalência gerada por $(f(a), g(a))$ para todo $a \in C$.

3.12.2 Implementação Computacional

Listing 3.12: Construções Universais em Mag

```
% Produto e Projeções
% A e B são estruturas com campos 'set' (conjunto) e 'op' (tabela
    da operação)
prod_set = combvec(A.set, B.set)';
prod_op = @(x, y) [A.op(x(1), y(1)), B.op(x(2), y(2))];

% Projeções
proj_A = @(x) x(:, 1);
proj_B = @(x) x(:, 2);

% Equalizador
eq_indices = find(arrayfun(@(i) isequal(f(A.set(i)), g(A.set(i))),
    1:length(A.set)));
eq_set = A.set(eq_indices);
eq_op = @(x, y) A.op(x, y); % Restrição da operação

% Coequalizador (usando componentes conexas)
edges = [f(A.set); g(A.set)]';
```

```

G = graph(edges(:, 1), edges(:, 2));
coeq_set = conncomp(G);
coeq_op = @(x, y) ...; % Operação induzida no quociente

% Pullback
[X, Y] = ndgrid(1:length(A.set), 1:length(B.set));
pb_indices = find(arrayfun(@(i) isequal(f(A.set(X(i))), g(B.set(Y(i))))), 1:numel(X)));
pb_set = [A.set(X(pb_indices)), B.set(Y(pb_indices))];
pb_op = @(x, y) [A.op(x(1), y(1)), B.op(x(2), y(2))];

% Pushout (simplificado)
% Requer a definição de uma relação de equivalência e quociente

```

3.13 Construções em DIMag (Dimagmas Finitos)

A categoria **DIMag** consiste em conjuntos finitos equipados com duas operações binárias. Os morfismos preservam ambas as operações. Esta categoria também é completa e cocompleta.

3.13.1 Fundamentação Teórica

- **Produto** ($A \times B$): Produto cartesiano dos conjuntos subjacentes, com as duas operações definidas componente a componente para cada operação.
- **Coproducto** ($A \sqcup B$): União disjunta dos conjuntos, com as operações definidas por casos, preservando a estrutura de cada dimagma nos subconjuntos correspondentes.
- **Equalizador**: Subconjunto do domínio onde os morfismos f e g coincidem para ambas as operações.
- **Coequalizador**: Espaço quociente do domínio pela relação de equivalência gerada por $f(a) \sim g(a)$, com as operações induzidas no quociente.
- **Pullback**: Subconjunto de $A \times B$ onde $f_1(a) = g_1(b)$ e $f_2(a) = g_2(b)$, com as operações definidas componente a componente.
- **Pushout**: Quociente da união disjunta $A \sqcup B$ pela relação de equivalência gerada por $(f_1(a), g_1(a))$ e $(f_2(a), g_2(a))$ para todo $a \in C$.

3.13.2 Implementação Computacional

Listing 3.13: Construções Universais em DIMag

```

% Produto e Projeções
prod_set = combvec(A.set, B.set)';
prod_op1 = @(x, y) [A.op1(x(1), y(1)), B.op1(x(2), y(2))];
prod_op2 = @(x, y) [A.op2(x(1), y(1)), B.op2(x(2), y(2))];

% Projeções

```

```

proj_A = @(x) x(:, 1);
proj_B = @(x) x(:, 2);

% Equalizador
eq_indices = find(arrayfun(@(i) isequal(f1(A.set(i)), g1(A.set(i)
    )) && ...
                                isequal(f2(A.set(i)), g2(A.set(i))), 1:
                                    length(A.set)));
eq_set = A.set(eq_indices);
eq_op1 = @(x, y) A.op1(x, y);
eq_op2 = @(x, y) A.op2(x, y);

% Coequalizador (usando componentes conexas para cada operação)
edges1 = [f1(A.set); g1(A.set)]';
edges2 = [f2(A.set); g2(A.set)]';
G1 = graph(edges1(:, 1), edges1(:, 2));
G2 = graph(edges2(:, 1), edges2(:, 2));
coeq_set = intersect(conncomp(G1), conncomp(G2));
coeq_op1 = @(x, y) ...; % Operação induzida no quociente
coeq_op2 = @(x, y) ...;

% Pullback
[X, Y] = ndgrid(1:length(A.set), 1:length(B.set));
pb_indices = find(arrayfun(@(i) isequal(f1(A.set(X(i))), g1(B.set
    (Y(i)))) && ...
                                isequal(f2(A.set(X(i))), g2(B.
                                    set(Y(i)))), 1:numel(X)));
pb_set = [A.set(X(pb_indices)), B.set(Y(pb_indices))];
pb_op1 = @(x, y) [A.op1(x(1), y(1)), B.op1(x(2), y(2))];
pb_op2 = @(x, y) [A.op2(x(1), y(1)), B.op2(x(2), y(2))];

% Pushout (simplificado)
% Requer a definição de uma relação de equivalência para ambas as
    operações

```

3.14 Construções em Row (Matrizes Linha)

A categoria **Row** consiste em matrizes linha (vetores linha) sobre um corpo (aqui, $\mathbb{C}$). Os morfismos são aplicações lineares compatíveis com a estrutura de vetor linha.

3.14.1 Fundamentação Teórica

- **Produto** ($V \times W$): Produto cartesiano das matrizes linha, resultando em uma matriz linha concatenada.
- **Coproduto** ($V \oplus W$): Soma direta das matrizes linha, resultando em uma matriz linha com a concatenação dos vetores.
- **Equalizador**: Subespaço das matrizes linha v tais que $f(v) = g(v)$.

- **Coequalizador:** Espaço quociente das matrizes linha pela relação $f(v) \sim g(v)$.
- **Pullback:** Subconjunto de $V \times W$ onde $f(v) = g(w)$, com a estrutura de matriz linha concatenada.
- **Pushout:** Soma direta módulo a relação gerada por $(f(v), -g(v))$.

3.14.2 Implementação Computacional

Listing 3.14: Construções Universais em Row

```
% Produto (Concatenação de vetores linha)
prod_row = @(v, w) [v, w];

% Projeções
proj_V = @(vw) vw(:, 1:size(V, 2));
proj_W = @(vw) vw(:, size(V, 2) + 1:end);

% Coproduto (Soma Direta)
coprod_row = @(v, w) [v, w];

% Injeções
inj_V = @(v) [v, zeros(1, size(W, 2))];
inj_W = @(w) [zeros(1, size(V, 2)), w];

% Equalizador
eq_indices = find(arrayfun(@(i) isequal(f(V(i, :)), g(V(i, :))),
    1:size(V, 1)));
eq_row = V(eq_indices, :);

% Coequalizador (Espaço Quociente)
B = [f(V) - g(V)];
[Q, ~] = qr(B', 'vector');
coeq_row = Q(:, rank(B) + 1:end)';

% Pullback
A = [f; -g];
[~, ~, V_pb] = svd(A);
pb_row = V_pb(:, size(A, 2) - rank(A) + 1:end)';

% Pushout (Soma Direta módulo relação)
% Requer a definição de um espaço quociente
```

3.15 Lat (Reticulados Finitos)

As construções na categoria dos reticulados finitos (**Lat**) focam-se na preservação da estrutura algébrica fechada. O objetivo principal é garantir que os novos objetos criados mantêm operações bem definidas de ínfimo ($\wedge$) e supremo ($\vee$), respeitando a comutatividade, associatividade, absorção e idempotência.

3.15.1 Fundamentação Teórica

As principais construções universais implementadas para esta categoria são:

- **Produtos:** O produto cartesiano $A \times B$ de dois reticulados forma um novo reticulado cujas operações são aplicadas componente a componente: $(a_1, b_1) \wedge (a_2, b_2) = (a_1 \wedge_A a_2, b_1 \wedge_B b_2)$, ocorrendo o mesmo de forma análoga para o supremo ($\vee$).
- **Equalizadores:** Dado um par de homomorfismos de reticulados $f, g : A \rightarrow B$, o equalizador é o subconjunto $E = \{x \in A \mid f(x) = g(x)\}$. Como f e g preservam as operações, este subconjunto é automaticamente fechado para $\wedge$ e $\vee$, formando um sub-reticulado exato de A .

3.15.2 Implementação Computacional

Em MATLAB, a construção do produto de dois reticulados exige o mapeamento dos pares de elementos para um novo índice linear. As tabelas de operação (*meet* e *join*) são geradas calculando as matrizes independentes componente a componente.

Listing 3.15: Construção do Produto de Reticulados (Lat)

```
function [MeetP, JoinP] = lattice_product(MeetA, JoinA, MeetB,
    JoinB)
    nA = size(MeetA, 1);
    nB = size(MeetB, 1);
    nP = nA * nB; % 0 tamanho do produto cartesiano

    MeetP = zeros(nP, nP);
    JoinP = zeros(nP, nP);

    % Função anônima para mapear o par (a, b) para um índice 1D
    (1 a nP)
    idx = @(a, b) (a - 1) * nB + b;

    for a1 = 1:nA
        for b1 = 1:nB
            x = idx(a1, b1); % Índice linear do primeiro par
            for a2 = 1:nA
                for b2 = 1:nB
                    y = idx(a2, b2); % Índice linear do segundo
                    par

                    % Calcula o resultado de A e B
                    independentemente
                    res_meet = idx(MeetA(a1, a2), MeetB(b1, b2));
                    res_join = idx(JoinA(a1, a2), JoinB(b1, b2));

                    % Preenche as matrizes de operação do novo
                    reticulado
                    MeetP(x, y) = res_meet;
                    JoinP(x, y) = res_join;
                end
            end
        end
    end
```



```

        end
    end
end

```

3.16 Bool (Álgebras de Boolean)

Em **Bool**, as construções de limites seguem a lógica dos latíces distributivos, mas com a exigência categórica de preservação do complemento ($\neg$) em todos os mapeamentos.

3.16.1 Fundamentação Teórica

- **Produtos:** A construção $B \times C$ utiliza as operações lógicas AND, OR e NOT aplicadas componente a componente.
- **Coequalizadores:** No contexto de Álgebras de Boolean, estes são calculados tomando o quociente da álgebra pelo ideal gerado pelas diferenças entre os morfismos aplicados.

3.16.2 Implementação Computacional

A implementação do produto baseia-se na expansão do conjunto de suporte através de um produto cartesiano de conjuntos.

Listing 3.16: Produto em Álgebras de Boolean

```

function B_out = bool_product(B1, B2)
    % Nota: set_product e uma funcao auxiliar para o produto de
    % conjuntos
    B_out.elements = set_product(B1.elements, B2.elements);

    % Definicao das operacoes componente a componente
    B_out.not = @(a) [B1.not(a(1)), B2.not(a(2))];
    B_out.and = @(a, b) [B1.and(a(1), b(1)), B2.and(a(2), b(2))];
    B_out.or = @(a, b) [B1.or(a(1), b(1)), B2.or(a(2), b(2))];
end

```

3.17 OrdMon (Monoides Ordenados)

Os limites na categoria **OrdMon** devem respeitar simultaneamente a estrutura algébrica do monoide e a relação de ordem parcial compatível.

3.17.1 Fundamentação Teórica

- **Produtos:** O objeto $M \times N$ herda a ordem do produto, onde $(m, n) \leq (m', n')$ se e só se as relações se verificam individualmente em cada componente ($m \leq_M m'$ e $n \leq_N n'$).
- **Equalizadores:** Correspondem ao sub-monoide formado pelos elementos onde os morfismos coincidem, mantendo a ordem herdada do domínio original.

3.17.2 Implementação Computacional

Abaixo detalham-se as funções para a criação do produto e a extração do equalizador.

Listing 3.17: Construções em Monoides Ordenados

```
% Produto em OrdMon
function OM = ordmon_product(M1, M2)
    OM.elements = set_product(M1.elements, M2.elements);
    OM.mul = @(a, b) [M1.mul(a(1), b(1)), M2.mul(a(2), b(2))];
    OM.unit = [M1.unit, M2.unit];
    % Verificacao da ordem combinada
    OM.le = @(a, b) M1.le(a(1), b(1)) && M2.le(a(2), b(2));
end

% Equalizador em OrdMon
function E = ordmon_equalizer(f, g, M)
    E.elements = {};
    for i = 1:length(M.elements)
        if isequal(f(M.elements{i}), g(M.elements{i}))
            E.elements{end+1} = M.elements{i};
        end
    end
    E.mul = M.mul;
    E.le = M.le;
end
```

3.18 Trees (Árvores Binárias)

As construções na categoria de árvores binárias visam criar novas estruturas que respeitem a hierarquia e a lateralidade (esquerda/direita) dos nós originais. O foco principal é garantir que a propriedade de ramificação binária seja preservada durante as operações categoriais.

3.18.1 Fundamentação Teórica

- **Produtos:** O produto $T_1 \times T_2$ de duas árvores binárias pode ser construído como a árvore cujos nós são os pares (u, v) com $u \in T_1$ e $v \in T_2$, com raiz $(\text{root}_1, \text{root}_2)$, e onde (u, v) tem filho esquerdo/direito se e só se u e v têm filho esquerdo/direito em simultâneo.

Esta construção é válida mas pode colapsar rapidamente para a árvore vazia se as duas árvores têm profundidades diferentes — o produto nesta categoria é *não trivial* e pode ser a árvore com um só nó (se as árvores têm alturas diferentes). Na prática, o produto existe mas pode ser menor do que qualquer dos fatores.

- **Coprodutos:** O coproduto $T_1 \sqcup T_2$ é a união disjunta das duas árvores. Para manter a estrutura de uma única árvore binária na implementação, introduz-se uma nova raiz comum cujos filhos esquerdo e direito são as raízes de T_1 e T_2 , respetivamente.

- **Pullbacks:** O pullback de $T_1 \xrightarrow{f} T_3 \xleftarrow{g} T_2$ é a subárvore do produto $T_1 \times T_2$ consistindo nos pares (u, v) tais que $f(u) = g(v)$. A estrutura de árvore (parentesco e lateralidade) é herdada diretamente do produto $T_1 \times T_2$.
- **Pushouts:** O pushout de $T_1 \xleftarrow{f} T_0 \xrightarrow{g} T_2$ é o coproduto $T_1 \sqcup T_2$ quocientado pela relação que identifica $f(u)$ com $g(u)$ para cada nó $u \in T_0$. Em árvores binárias, isto corresponde a “colar” as duas árvores ao longo da imagem comum.
- **Equalizadores:** O equalizador de dois morfismos $f, g : T_1 \rightrightarrows T_2$ é a subárvore de T_1 induzida pelo conjunto de nós $E = \{u \in T_1 : f(u) = g(u)\}$, desde que E forme uma subárvore válida (deve ser fechada para progenitores e conter a raiz).
- **Coequalizadores:** O coequalizador de $f, g : T_1 \rightrightarrows T_2$ é o quociente de T_2 pela menor relação de equivalência que identifica $f(u) \sim g(u)$ e que preserva a estrutura de árvore binária (se $u \sim v$, então os seus filhos respectivos devem ser identificados).

3.18.2 Implementação Computacional

A implementação em MATLAB utiliza funções recursivas e manipulação de índices para respeitar as condições teóricas descritas.

Listing 3.18: Produto de duas árvores binárias (representação recursiva)

```
function T = tree_product(T1, T2)
    % Constroi o produto T1 x T2 recursivamente
    % Retorna [] se um dos nos nao tem correspondente no outro
    if isempty(T1) || isempty(T2)
        T = []; return;
    end
    T.value = [T1.value, T2.value]; % par de valores
    T.left = tree_product(T1.left, T2.left);
    T.right = tree_product(T1.right, T2.right);
end
```

Listing 3.19: Coproduto em Trees (Nova Raiz)

```
function T = tree_coproduct(T1, T2)
    % O coproduto e formado criando uma nova raiz artificial
    T.value = []; % Raiz sem conteudo informativo direto
    T.left = T1; % T1 torna-se o ramo esquerdo
    T.right = T2; % T2 torna-se o ramo direito
end
```

Listing 3.20: Equalizador e Pullback em Trees

```
function [E, inc] = tree_equalizer(T1, f, g)
    % E: subconjunto dos nos de T1 onde f e g coincidem
    mask = (f == g);
    E = find(mask); % indices dos nos do equalizador
    inc = E; % inclusao em T1
    % Nota: verificar que E e fechado para progenitores
end
```

```

function [PB, pi1, pi2] = tree_pullback(T1, T2, f, g)
    % PB e a subarvore de T1 x T2 onde f(u) == g(v)
    nodes = [];
    for i = 1:T1.n
        for j = 1:T2.n
            if f(i) == g(j)
                nodes(end+1, :) = [i, j];
            end
        end
    end
    PB.nodes = nodes;
    pi1 = nodes(:,1)'; % Projecao para T1
    pi2 = nodes(:,2)'; % Projecao para T2
    % A estrutura herda-se de T1 x T2
end

```

3.19 Turing Machines (Máquinas de Turing)

As construções na categoria de Máquinas de Turing permitem modelar a computação paralela, a composição de algoritmos e a identificação de comportamentos equivalentes entre autómatos. O foco é garantir que os novos objetos preservem a coerência das funções de transição e dos estados de aceitação.

3.19.1 Fundamentação Teórica

- **Produtos:** O produto $M_1 \times M_2$ é a máquina de Turing que simula M_1 e M_2 em paralelo. O conjunto de estados é dado pelo produto cartesiano $Q_1 \times Q_2$, o alfabeto é $\Gamma_1 \times \Gamma_2$ (ou um alfabeto comum), e a função de transição aplica ambas as transições simultaneamente, movendo as respectivas cabeças de leitura de forma independente ou coordenada.
- **Coprodutos:** O coproduto $M_1 \sqcup M_2$ representa a união disjunta de duas máquinas. Na prática, cria-se uma máquina que pode executar M_1 ou M_2 , partindo de um novo estado inicial que ramifica para os estados iniciais originais consoante o primeiro símbolo lido na fita.
- **Pullbacks:** Dado um diagrama $M_1 \xrightarrow{f} M_3 \xleftarrow{g} M_2$, o pullback é a submáquina do produto $M_1 \times M_2$ cujos estados são os pares (q_1, q_2) que satisfazem $f_Q(q_1) = g_Q(q_2)$. Esta máquina simula simultaneamente M_1 e M_2 , mas apenas em estados que sejam compatíveis através da sua imagem em M_3 .
- **Pushouts:** O pushout de $M_1 \xleftarrow{f} M_0 \xrightarrow{g} M_2$ corresponde ao coproduto $M_1 \sqcup M_2$ quocientado pelas identificações $f_Q(q) \sim g_Q(q)$ para todo $q \in Q_0$ (e analogamente para os símbolos do alfabeto). Intuitivamente, esta construção "funde" dois autómatos ao longo de um subautômato comum M_0 .
- **Equalizadores:** O equalizador de dois morfismos paralelos $f, g : M_1 \rightrightarrows M_2$ é a submáquina de M_1 induzida pelo conjunto de estados onde as funções de transição

e mapeamento coincidem ($f_Q(q) = g_Q(q)$), desde que este subconjunto contenha o estado inicial e seja fechado para as transições da máquina.

- **Coequalizadores:** O coequalizador de $f, g : M_1 \rightrightarrows M_2$ é definido como o quociente de M_2 pela menor relação de equivalência que identifica as imagens $f(q) \sim g(q)$. O resultado é uma máquina simplificada onde estados que desempenham papéis equivalentes sob os morfismos f e g são fundidos num único estado.

3.19.2 Implementação Computacional

Abaixo apresentam-se as implementações em MATLAB para a construção de novos objetos nesta categoria, focando-se na manipulação de matrizes de transição e estados.

Listing 3.21: Produto de duas Máquinas de Turing

```
function M = TM_product(M1, M2)
    % Estados: pares (q1, q2) codificados como (q1-1)*M2.Q + q2
    M.Q = M1.Q * M2.Q;
    M.Gamma = M1.Gamma; % assumindo Gamma1 == Gamma2 por
    % simplicidade
    M.blank = M1.blank;
    M.q0 = (M1.q0 - 1) * M2.Q + M2.q0;

    % Estados finais: pares de estados finais
    M.F = [];
    for q1 = M1.F
        for q2 = M2.F
            M.F(end+1) = (q1-1)*M2.Q + q2;
        end
    end

    % Transicoes: aplicar M1 e M2 simultaneamente
    M.delta = zeros(M.Q, numel(M.Gamma), 3);
    for q1 = 1:M1.Q
        for q2 = 1:M2.Q
            q = (q1-1)*M2.Q + q2;
            for a = 1:numel(M.Gamma)
                tr1 = squeeze(M1.delta(q1,a,:))';
                tr2 = squeeze(M2.delta(q2,a,:))';
                new_q = (tr1(1)-1)*M2.Q + tr2(1);
                % Simplificacao: usar simbolo e direcao de M1
                M.delta(q, a, :) = [new_q, tr1(2), tr1(3)];
            end
        end
    end
end
```

3.20 Struct (Conjuntos Estruturados)

A categoria **Struct** permite modelar sistemas dinâmicos discretos onde cada elemento possui um valor associado e um ponteiro para um sucessor (endomapa). As construções

universais nesta categoria garantem a preservação da coerência entre os dados armazenados e a estrutura de transição.

3.20.1 Fundamentação Teórica

- **Produto:** O produto de (d_1, φ_1, n_1) e (d_2, φ_2, n_2) é o triplo $(d, \varphi, n_1 \cdot n_2)$, onde os elementos são indexados por pares (i, j) . A estrutura e os dados são definidos por:

$$\varphi(i, j) = (\varphi_1(i), \varphi_2(j)), \quad d(i, j) = (d_1(i), d_2(j)).$$

- **Coproduto:** O coproduto $S_1 \sqcup S_2$ é a união disjunta dos conjuntos de elementos, mantendo as funções φ e d originais em cada componente. O número total de elementos é $n_1 + n_2$.
- **Pullback:** Para um diagrama $S_1 \xrightarrow{f} S_3 \xleftarrow{g} S_2$, o pullback é o subobjeto do produto $S_1 \times S_2$ constituído pelos pares (i, j) tais que $f(i) = g(j)$.
- **Pushout:** O pushout de $S_1 \xleftarrow{f} S_0 \xrightarrow{g} S_2$ é o coproduto $S_1 \sqcup S_2$ submetido à menor relação de equivalência que identifica $f(k) \sim g(k)$ para todo $k \in S_0$, estendendo-se para manter a compatibilidade com os endomapas.
- **Equalizador:** O equalizador de $f, g : S_1 \rightrightarrows S_2$ é o subobjeto de S_1 formado pelos elementos i tais que $f(i) = g(i)$. Por definição de morfismo, este conjunto é automaticamente fechado para a aplicação de φ_1 .
- **Coequalizador:** O coequalizador de $f, g : S_1 \rightrightarrows S_2$ é o quociente de S_2 pela menor congruência compatível com φ_2 que identifica $f(i) \sim g(i)$ para todo $i \in S_1$.

3.20.2 Implementação Computacional

Abaixo apresenta-se a implementação em MATLAB, respeitando as estruturas e algoritmos de união-busca presentes no script original.

Listing 3.22: Produto e Coproduto em Struct

```
function P = struct_product(S1, S2)
    P.n = S1.n * S2.n;
    P.phi = zeros(1, P.n);
    P.d = zeros(1, P.n, 'like', 1i);
    for i = 1:S1.n
        for j = 1:S2.n
            k = (i-1)*S2.n + j;
            % phi(i,j) = (phi1(i), phi2(j))
            P.phi(k) = (S1.phi(i)-1)*S2.n + S2.phi(j);
            P.d(k) = S1.d(i) + S2.d(j);
        end
    end
end

function C = struct_coproduct(S1, S2)
    C.n = S1.n + S2.n;
    C.phi = [S1.phi, S2.phi + S1.n];
```

```

    C.d    = [S1.d, S2.d];
end

```

Listing 3.23: Equalizador e Pullback em Struct

```

function [E, inc] = struct_equalizer(S1, f, g)
    E_idx = find(f == g);
    E.n    = numel(E_idx);
    inc    = E_idx;
    E.d    = S1.d(E_idx);
    E.phi  = zeros(1, E.n);
    for i = 1:E.n
        target = S1.phi(E_idx(i));
        [~, loc] = ismember(target, E_idx);
        E.phi(i) = loc;
    end
end

function [PB, p1, p2] = struct_pullback(S1, S2, f, g)
    % Baseado na construcao de subobjeto do produto
    pairs = [];
    for i = 1:S1.n
        for j = 1:S2.n
            if f(i) == g(j)
                pairs = [pairs; i, j];
            end
        end
    end
    PB.n = size(pairs, 1);
    p1 = pairs(:,1)'; p2 = pairs(:,2)';
    PB.d = S1.d(p1); % d1 = d2 o f no pullback
    PB.phi = zeros(1, PB.n);
    for k = 1:PB.n
        t1 = S1.phi(p1(k)); t2 = S2.phi(p2(k));
        [~, PB.phi(k)] = ismember([t1, t2], pairs, 'rows');
    end
end

```

Listing 3.24: Coequalizador e Pushout em Struct

```

function [CE, quot] = struct_coequalizer(S1, S2, f, g)
    parent = 1:S2.n;
    function r = find_root(x)
        while parent(x) ~= x, x = parent(x); end
        r = x;
    end
    % Identificar f(i) ~ g(i)
    for i = 1:S1.n
        a = find_root(f(i)); b = find_root(g(i));
        if a ~= b, parent(a) = b; end
    end
    % Fechar para congruencia (phi)

```

```

    changed = true;
    while changed
        changed = false;
        for x = 1:S2.n
            rx = find_root(x);
            pa = find_root(S2.phi(x));
            pb = find_root(S2.phi(rx));
            if pa ~= pb
                parent(pa) = pb; changed = true;
            end
        end
    end
    [roots, ~, quot] = unique(arrayfun(@find_root, 1:S2.n));
    CE.n = numel(roots);
    CE.phi = quot(S2.phi(roots));
    CE.d = S2.d(roots);
end

function [P0, i1, i2] = struct_pushout(S0, S1, S2, f, g)
    C = struct_coproduct(S1, S2);
    f_c = f; % f em S1
    g_c = g + S1.n; % g em S2 deslocado
    [P0, quot] = struct_coequalizer(S0, C, f_c, g_c);
    i1 = quot(1:S1.n);
    i2 = quot(S1.n+1:end);
end

```

3.21 FinSet (Conjuntos Finitos como Matrizes Linha)

A categoria **FinSet** constitui a base computacional do trabalho desenvolvido neste relatório. Ao contrário da categoria **Fin** (Secção 2.1), onde os objetos são representados apenas pela sua cardinalidade, em **FinSet** cada objeto é uma *matriz linha de MATLAB/Octave com linhas únicas e ordenadas lexicograficamente*. Esta representação permite modelar conjuntos cujos elementos são registos multidimensionais — strings, vetores inteiros, coordenadas no espaço — de forma direta, sem perda de informação estrutural.

A categoria **FinSet** é o ambiente natural para implementar construções universais (produto, coproduto, equalizador, pullback e pushout), funções hash, homologia de endomapas e estruturas algébricas como magmas, sendo todas estas noções rigorosamente definíveis como morfismos ou funtores em **FinSet**.

3.21.1 Objetos

Definição 3.21.1 (Objeto de **FinSet**). Um **objeto** da categoria **FinSet** é uma variável MATLAB/Octave A que satisfaz o *invariante de representação*:

$$\text{isequal}(A, \text{unique}(A, 'rows')) = \text{true}. \quad (3.1)$$

Isto significa que as linhas de A são distintas e estão ordenadas lexicograficamente. Cada linha representa um elemento do conjunto finito correspondente.

Observação 3.21.1. A função `unique(A, 'rows')` do MATLAB realiza simultaneamente a remoção de duplicados e a ordenação lexicográfica. Qualquer operação que produza um novo objeto deve terminar com `sortrows(unique(X, 'rows'))` para restaurar o invariante (3.1).

Listing 3.25: Exemplos de objetos em FinSet

```

1 is_FinSet_object = @(X) isequal(X, sortrows(unique(X, 'rows')));
2
3 % Objeto 1: conjunto de inteiros {1,...,7}
4 A = (1:7)';
5 isequal(A, unique(A, 'rows')) % true
6
7 % Objeto 2: conjunto de strings (alunos da sala)
8 A_str = sortrows(['Abe'; 'Bar'; 'Car'; 'Dye'; 'Eve'; 'Fly';
9                  'Gym'; 'Hym'; 'Ive'; 'Jun'; 'Kay'; 'Lie';
10                 'May'; 'Neo'; 'Oli'; 'Pie'; 'Que'; 'Ray';
11                 'Sym'; 'Til']);
12 isequal(A_str, unique(A_str, 'rows')) % true (20 alunos)
13
14 % Objeto 3: mesas como pares (linha, coluna) em {1..5}x{1..3}
15 [i,j] = ind2sub([5 3], 1:15);
16 B = sortrows([i(:), j(:)]); % 15 mesas
17
18 % Objeto 4: vetores em Z^3 (inclui negativos)
19 C = sortrows([1 0 0; 0 1 0; 0 0 1; 0 0 0;
20              -1 0 0; 0 -1 0; 0 0 -1]);

```

3.21.2 Morfismos

Definição 3.21.2 (Morfismo de FinSet). Um **morfismo** em FinSet é um triplo (B, f, A) onde:

- A e B são objetos de FinSet;
- f é um vetor de inteiros de comprimento $|A|$ (i.e., `size(A,1)`) satisfazendo:

$$\text{nnz}(f) == \text{size}(A,1) \quad \text{e} \quad f(i) \in \{1, \dots, |B|\}, \forall i. \quad (3.2)$$

A condição `nnz(f) == size(A,1)` garante que f é uma *função total*: todo o elemento do domínio tem exatamente uma imagem. A expressão `B(f,:)` devolve, para cada linha i de A , a linha $f(i)$ de B , codificando assim $f : A \rightarrow B$.

Listing 3.26: Verificação e exemplos de morfismos em FinSet

```

1 function ok = is_FinSet_morphism(B, f, A)
2     f = f(:);
3     ok = (numel(f) == size(A,1)) && ...
4           (nnz(f) == size(A,1)) && ...
5           all(f >= 1) && ...
6           all(f <= size(B,1));
7 end

```

```

8
9 % Morfismo: cada aluno (A_str) -> mesa (B)
10 % f(x) = indice da mesa do aluno x
11 f = mod((1:20)'-1, 13) + 1;
12 assert(is_FinSet_morphism(B, f, A_str));
13
14 % Visualizacao: "aluno x sentado na mesa (i,j)"
15 disp([num2str((1:20)'), repmat(' -> ', 20, 1), ...
16       num2str(B(f,:))]);

```

Definição 3.21.3 (Identidade e Composição). A **identidade** em A é o morfismo $(A, (1:n)', A)$ com $n = |A|$. A **composição** de (C, g, B) com (B, f, A) é $(C, g(f), A)$, i.e., a indexação composta $h = g(f)$.

Proposição 3.21.1. A composição em FinSet é associativa e a identidade é neutra, fazendo de FinSet uma categoria.

Demonstração. Para (D, h, C) , (C, g, B) , (B, f, A) : $(h \circ g) \circ f = h(g(f)) = h \circ (g \circ f)$, pois a indexação de vetores é associativa em MATLAB. $\square$

Listing 3.27: Composição de morfismos

```

1 function h = compose_FinSet(g, f)
2     h = g(f); % h(i) = g(f(i))
3 end
4
5 % Verificacao de associatividade
6 f1 = [2;1;3]; g1 = [3;1;2]; h1 = [2;3;1];
7 assert(isequal(compose_FinSet(h1, compose_FinSet(g1,f1)), ...
8             compose_FinSet(compose_FinSet(h1,g1), f1)));

```

3.21.3 Funções Hash como Morfismos de FinSet

Definição 3.21.4 (Função Hash em FinSet). Uma **função hash** $h : U \rightarrow \{1, \dots, m\}$ é um morfismo em FinSet de um universo de chaves U (potencialmente grande) para um conjunto de *buckets* de cardinalidade m pequena. Em notação de FinSet , $h = (B_m, f_h, U)$ com $B_m = (1:m)'$.

A diferença fundamental entre uma função hash e um morfismo injetivo é a presença de **colisões**: $h(x) = h(y)$ com $x \neq y$. A gestão de colisões é uma escolha adicional (encadeamento, endereçamento aberto, etc.) que corresponde categoricamente à escolha de uma *secção* do morfismo de quociente associado.

Listing 3.28: Três famílias de funções hash como morfismos de FinSet

```

1 %% 1) Hash por divisao: h(k) = mod(k,m) + 1
2 hash_div = @(k, m) mod(k, m) + 1;
3
4 %% 2) Hash por multiplicacao (Knuth)
5 % A = (sqrt(5)-1)/2 (razao aurea conjugada)
6 A_knuth = (sqrt(5)-1)/2;
7 hash_mult = @(k, m) floor(m * mod(k*A_knuth, 1)) + 1;

```

```

8
9 %% 3) Hash polinomial para strings
10 function idx = hash_poly(s, m)
11     p = 31; val = 0;
12     for i = 1:numel(s)
13         val = mod(val*p + double(s(i)), m);
14     end
15     idx = val + 1;
16 end
17
18 %% Demonstracao: U = {1,...,20}, m = 7 buckets
19 U = 1:20; m = 7;
20 h1 = arrayfun(@(k) hash_div(k,m), U);
21 h2 = arrayfun(@(k) hash_mult(k,m), U);
22
23 fprintf('Chaves: '); fprintf('%3d', U); fprintf('\n');
24 fprintf('Div    : '); fprintf('%3d', h1); fprintf('\n');
25 fprintf('Mult   : '); fprintf('%3d', h2); fprintf('\n');

```

Definição 3.21.5 (Colisão e Resolução). Dada $h : U \rightarrow B_m$, diz-se que $x, y \in U$ com $x \neq y$ **colidem** se $h(x) = h(y)$. A **resolução por encadeamento** associa a cada bucket $b \in B_m$ uma lista $T[b] = \{x \in U : h(x) = b\}$, que corresponde à *fibra* de h sobre b .

Listing 3.29: Tabela hash com encadeamento — aplicação a ficheiros

```

1 %% Tabela hash para ficheiros (hash de conteudo)
2 function T = build_hash_table(keys, m, h_fn)
3     T = cell(m,1);
4     for i = 1:numel(keys)
5         b = h_fn(keys(i));
6         T{b} = [T{b}, keys(i)];
7     end
8 end
9
10 %% Simulacao: 5 "ficheiros" como vetores de bytes
11 files = {[72 101 108 108 111], ... % "Hello"
12          [87 111 114 108 100], ... % "World"
13          [77 65 84 76 65 66], ... % "MATLAB"
14          [70 105 110 83 101 116], ... % "FinSet"
15          [72 101 108 108 111]}; ... % "Hello" (duplicado)
16 m_f = 8;
17 function h = hash_content(data, m)
18     p = 257; h = 0;
19     for i = 1:numel(data)
20         h = mod(h*p + data(i), m);
21     end
22     h = h + 1;
23 end
24 for i = 1:numel(files)
25     fprintf('Ficheiro %d -> bucket %d\n', i, ...
26            hash_content(files{i}, m_f));

```

```

27 end
28 %% Ficheiros 1 e 5 colidem: mesmo conteudo!

```

3.21.4 Homologia de Endomapas vs. Funções Hash

Definição 3.21.6 (Homologia de um Endomapa). Dado $\varphi : A \rightarrow A$ com $|A| = n$, o **grafo dirigido associado** $G(\varphi)$ tem vértices A e arestas $i \rightarrow \varphi(i)$. A **homologia** de φ é a matriz $H(\varphi) \in \mathbb{N}^{c \times 4}$, onde c é o número de componentes conexas (no sentido do grafo subjacente não dirigido) e cada linha r contém:

$$H(\varphi)_r = (\# \text{grau-in } 0, \# \text{grau-in } 1, \# \text{grau-in } \geq 2, \text{ comp. do ciclo})_r. \quad (3.3)$$

Observação 3.21.2. Todo o endomapa φ de um conjunto finito possui pelo menos um ciclo em cada componente conexa (teorema de Rho de Floyd). Os vértices *fora* do ciclo formam “caudas” (grau de entrada 0 no ciclo). Os vértices de grau de entrada ≥ 2 são pontos de ramificação.

Listing 3.30: Cálculo da matriz de homologia

```

1 function H = endomap_homology(phi)
2     phi = phi(:)'; n = numel(phi);
3     % --- 1. Componentes conexas (union-find)
4     parent = 1:n;
5     function r = find_root(x)
6         while parent(x)~=x
7             parent(x)=parent(parent(x)); x=parent(x);
8         end; r=x;
9     end
10    for i=1:n, a=find_root(i); b=find_root(phi(i));
11        if a~=b, parent(a)=b; end
12    end
13    comp_label = arrayfun(@find_root, 1:n);
14    [comp_ids,~,comp_idx] = unique(comp_label);
15    n_comp = numel(comp_ids);
16    % --- 2. Grau de entrada
17    in_deg = histc(phi, 1:n);
18    % --- 3. Detectar ciclos
19    on_cycle = false(1,n);
20    for s=1:n
21        visited=false(1,n); x=s;
22        while ~visited(x) && ~on_cycle(x)
23            visited(x)=true; x=phi(x);
24        end
25        if ~on_cycle(x) && visited(x)
26            y=x;
27            while ~on_cycle(y), on_cycle(y)=true; y=phi(y); end
28        end
29    end
30    % --- 4. Montar H
31    H = zeros(n_comp,4);
32    for c=1:n_comp

```

```

33     mc = (comp_idx==c);
34     id = in_deg(mc);
35     H(c,:) = [sum(id==0), sum(id==1), sum(id>=2), ...
36               sum(mc & on_cycle)];
37     end
38     H = sortrows(H);
39 end

```

Listing 3.31: Exemplos do quadro da aula

```

1  %% Exemplo 1 (quadro, Imagem 4):
2  %% phi: 1->2, 2->1, 3->4, 4->6, 5->6, 6->3, 7->7
3  phi1 = [2, 1, 4, 6, 6, 3, 7];
4  H1    = endomap_homology(phi1);
5  disp('Homologia phi1:'); disp(H1);
6  %% Resultado esperado (3 componentes):
7  %% grau0 grau1 grau2 ciclo
8  %%      0      2      0      2      (ciclo {1,2})
9  %%      1      2      1      3      (ciclo {3,4,6}, cauda {5})
10 %%      0      1      0      1      (ciclo {7})
11
12 %% Exemplo 2 (quadro, Imagem 2, secção esquerda):
13 %% phi: 1->5, 2->3, 3->2, 4->5, 5->6, 6->7, 7->4
14 phi2 = [5, 3, 2, 5, 6, 7, 4];
15 H2    = endomap_homology(phi2);
16 disp('Homologia phi2:'); disp(H2);
17
18 %% Exemplo 3 (quadro, Imagem 3):
19 %% phi: 1->2, 2->3, 3->4, 4->5, 5->2, 6->7, 7->7
20 phi3 = [2, 3, 4, 5, 2, 7, 7];
21 H3    = endomap_homology(phi3);
22 disp('Homologia phi3:'); disp(H3);

```

Proposição 3.21.2. A homologia $H : \text{End}(A) \rightarrow \text{FinSet}$ é um **invariante de conjugação**: se $\varphi \sim \psi$ (i.e., existe bijeção $w : A \rightarrow A$ tal que $\varphi \circ w = w \circ \psi$), então $H(\varphi) = H(\psi)$.

Demonstração. A conjugação por w é um isomorfismo de grafos de $G(\psi)$ em $G(\varphi)$, preservando assim a estrutura das componentes conexas, os graus de entrada e os comprimentos de ciclo. $\square$

3.21.5 Hash vs. Homologia: Análise Comparativa

Ambas as construções são morfismos de FinSet que *resumem* informação de um objeto grande (universo U ou $\text{End}(A)$) num objeto mais pequeno. A diferença essencial está no objetivo: eficiência de acesso (hash) vs. invariância estrutural (homologia).

Em particular, a homologia pode ser usada como uma **hash de classificação** para endomapas: ao invés de comparar dois endomapas diretamente (custo $O(n)$), calcula-se $H(\varphi)$ e $H(\psi)$ e compara-se as matrizes resultantes. Se $H(\varphi) \neq H(\psi)$, os endomapas não são conjugados (e.g. num ficheiro de base de dados de estruturas, basta indexar pelo hash da homologia).

Tabela 3.1: Comparação entre funções hash e a função de homologia.

Propriedade	Função Hash	Homologia
Domínio	U (chaves arbitrárias)	$\text{End}(A)$ (endomapas)
Codomínio	$\{1, \dots, m\}$ (buckets)	Matrizes de $\mathbb{N}^{c \times 4}$
Objetivo	Indexação rápida	Classificação estrutural
Colisões	Inevitáveis (pretendidas)	Possíveis (indesejadas)
Complexidade	$O(1)$ (avaliação)	$O(n)$ (cálculo)
Invariância	Nenhuma garantida	Sob conjugação
Aplicação	BD, ficheiros, criptografia	Classes de $\text{End}(A)$

Listing 3.32: Uso da homologia como hash de classificação

```

1 %% Indexacao de endomapas por homologia
2 phi_list = {[2,1,3,4,5], [1,2,3,4,5], [2,1,4,3,5], ...
3             [3,1,2,4,5], [2,3,1,4,5], [2,1,5,3,4]};
4 m_hash = 16; % numero de buckets para a tabela hash
5
6 groups = containers.Map('KeyType','char','ValueType','any');
7 for k = 1:numel(phi_list)
8     H = endomap_homology(phi_list{k});
9     key = mat2str(H); % string da matriz = chave hash
10    if groups.isKey(key)
11        groups(key) = [groups(key), k];
12    else
13        groups(key) = k;
14    end
15 end
16 disp('Grupos de conjugacao por homologia:');
17 keys_g = groups.keys;
18 for g = 1:numel(keys_g)
19     fprintf(' Homologia %s -> endomapas %s\n', ...
20             keys_g{g}, mat2str(groups(keys_g{g})));
21 end

```

3.21.6 Estrutura de Magma a partir de Endomapas

Definição 3.21.7 (Magma derivado de um Endomapa). Dado $\varphi : A \rightarrow A$, define-se a operação binária $*_{\varphi}$ em A por:

$$x *_{\varphi} x = \varphi(x) \quad \text{e} \quad x^{n+1} = \varphi^n(x), \quad (3.4)$$

onde φ^n denota a n -ésima iteração de φ . Para $x \neq y$, a regra geral é: $x *_{\varphi} y = x$ ou y dependendo do tamanho da componente (conforme o quadro da aula: elementos de componentes maiores absorvem os menores). A tabela de Cayley M_{φ} de $(A, *_{\varphi})$ é uma matriz $|A| \times |A|$ de inteiros.

Listing 3.33: Construção da tabela de Cayley do magma de um endomapa

```

1 function M = endomap_to_magma(phi)

```

```

2      % Tabela de Cayley: M(i,j) = i * j
3      % Regra: x*x = phi(x); x*y = x se i<j, senao j
4      phi = phi(:)'; n = numel(phi);
5      M = zeros(n,n);
6      for i = 1:n
7          for j = 1:n
8              if i == j
9                  M(i,j) = phi(i);    % x*x = phi(x)
10             elseif i < j
11                 M(i,j) = i;          % absorção pelo menor
12             else
13                 M(i,j) = j;
14             end
15         end
16     end
17 end
18
19 % Exemplo do quadro (Imagem 5): phi: 1->4, 2->3, 3->5, 4->1,
20 % 5->2
21 phi_m = [4, 3, 5, 1, 2];
22 M_mag = endomap_to_magma(phi_m);
23 disp('Tabela de Cayley:'); disp(M_mag);
24
25 %% Verificacao: x^(n+1) = phi^n(x)
26 %% x^2 = x*x = phi(x):
27 assert(all(diag(M_mag)' == phi_m));
28 fprintf('x*x = phi(x) para todo x: OK\n');
29
30 %% x^3 = x^2 * x = phi(x) * x
31 %% (como phi(x) < x ou phi(x) > x, aplica-se a regra de tamanho)

```

Proposição 3.21.3. A construção $(A, \varphi) \mapsto (A, *_\varphi)$ é **functorial**: um morfismo $f : (A_1, \varphi_1) \rightarrow (A_2, \varphi_2)$ em **End** (i.e., $f \circ \varphi_1 = \varphi_2 \circ f$) induz um homomorfismo $f : (A_1, *_\varphi_1) \rightarrow (A_2, *_\varphi_2)$ em **Mag**, satisfazendo $f(x *_\varphi_1 y) = f(x) *_\varphi_2 f(y)$.

3.21.7 Classes de Conjugação em End(A)

Definição 3.21.8 (Conjugação em End(A)). Dados $u, v \in \text{End}(A)$, diz-se que u e v são **conjugados**, escrevendo $u \sim v$, se existe $w \in \text{Bij}(A)$ tal que $u \circ w = w \circ v$.

A relação $\sim$ é de equivalência. A homologia fornece um invariante para separar classes: se $H(u) \neq H(v)$, então $u \not\sim v$.

Listing 3.34: Algoritmo de verificação de conjugação

```

1 function ok = are_conjugate(phi1, phi2)
2     % Verifica se phi1 ~ phi2 (para n pequeno, forca bruta)
3     n = numel(phi1);
4     if ~isequal(sortrows(endomap_homology(phi1)), ...
5                 sortrows(endomap_homology(phi2)))
6         ok = false; return;
7     end

```

```

8   W = perms(1:n);    % todas as permutacoes de {1..n}
9   ok = false;
10  for k = 1:size(W,1)
11      w = W(k,:);
12      if isequal(phi1(w), w(phi2))    % u o w = w o v
13          ok = true; return;
14      end
15  end
16 end
17
18 %% Exemplos
19 phi_a = [2, 1, 4, 3, 5];    % dois 2-ciclos + ponto fixo
20 phi_b = [2, 1, 5, 3, 4];    % outro endomapa com 2 ciclos
21 phi_c = [1, 2, 3, 4, 5];    % identidade
22
23 fprintf('phi_a ~ phi_b? %d\n', are_conjugate(phi_a, phi_b));
24 fprintf('phi_a ~ phi_c? %d\n', are_conjugate(phi_a, phi_c));

```

3.22 Construções em FinSet

3.22.1 Fundamentação Teórica

Proposição 3.22.1 (Completeness and Co-completeness of FinSet). A categoria FinSet é completa e cocompleta: todos os limites e colimites finitos existem. Em particular:

- O **produto** $A \times B$ tem $|A \times B| = |A| \cdot |B|$ e é representado pelas linhas do produto cartesiano de A e B .
- O **coproduto** $A \sqcup B$ tem $|A \sqcup B| = |A| + |B|$ e é a união disjunta com etiqueta de proveniência.
- O **equalizador** de $f, g : A \rightarrow B$ é o subconjunto $E = \{a \in A : f(a) = g(a)\}$.
- O **coequalizador** de $f, g : A \rightarrow B$ é o quociente de B pela menor relação de equivalência gerada por $\{f(a) \sim g(a) : a \in A\}$.
- O **pullback** de $f : A \rightarrow C$ e $g : B \rightarrow C$ é $\{(a, b) \in A \times B : f(a) = g(b)\}$.
- O **pushout** de $f : C \rightarrow A$ e $g : C \rightarrow B$ é o coequalizador de $\iota_A \circ f$ e $\iota_B \circ g$ no coproduto $A \sqcup B$.

3.22.2 Implementação Computacional

Listing 3.35: Produto Categórico em FinSet com verificação da propriedade universal

```

1 function [P, pi_A, pi_B] = FinSet_product(A, B)
2     % Produto A x B; pi_A, pi_B: projecoes
3     nA = size(A,1); nB = size(B,1);
4     [iA, iB] = ndgrid(1:nA, 1:nB);
5     iA = iA(:); iB = iB(:);
6     [P, ord] = sortrows([A(iA,:), B(iB,:)]);

```



```

7     pi_A = iA(ord); pi_B = iB(ord);
8 end
9
10 % Propriedade universal: para qualquer h: C->A e k: C->B
11 % existe unico u: C->P tal que pi_A(u)=h e pi_B(u)=k
12 function u = product_mediator(A, B, P, pi_A, pi_B, h, k)
13     nC = numel(h);
14     u = zeros(nC,1);
15     for i = 1:nC
16         % Encontrar (A(h(i),:), B(k(i),:)) em P
17         row = [A(h(i),:), B(k(i),:)];
18         [~, u(i)] = ismember(row, P, 'rows');
19     end
20 end

```

Listing 3.36: Coproduto e Injeções em FinSet

```

1 function [C, iota_A, iota_B] = FinSet_coprodut(A, B)
2     nA = size(A,1); nB = size(B,1);
3     tagged = [zeros(nA,1), A; ones(nB,1), B];
4     C = sortrows(tagged);
5     [~, iota_A] = ismember([zeros(nA,1),A], C, 'rows');
6     [~, iota_B] = ismember([ones(nB,1),B], C, 'rows');
7 end

```

Listing 3.37: Equalizador e Coequalizador em FinSet

```

1 function [E, inc] = FinSet_equalizer(B, f, g, A)
2     mask = all(B(f,:) == B(g,:), 2);
3     idx = find(mask);
4     E = A(idx,:);
5     inc = idx;
6 end
7
8 function [Q, quot] = FinSet_coequalizer(B, f, g, A)
9     nB = size(B,1);
10    parent = 1:nB;
11    function r = find_root(x)
12        while parent(x)~=x
13            parent(x)=parent(parent(x)); x=parent(x);
14        end; r=x;
15    end
16    for a = 1:size(A,1)
17        ra=find_root(f(a)); rb=find_root(g(a));
18        if ra~=rb, parent(ra)=rb; end
19    end
20    classes = arrayfun(@find_root,1:nB);
21    [roots,~,quot] = unique(classes);
22    [Q, ord] = sortrows(B(roots,:));
23    quot = ord(quot)';
24 end

```

Listing 3.38: Pullback e Pushout em FinSet

```

1 function [PB, p_A, p_B] = FinSet_pullback(C, f, g, A, B)
2     nA=size(A,1); nB=size(B,1);
3     pairs_A=[]; pairs_B=[];
4     for a=1:nA, for b=1:nB
5         if isequal(C(f(a),:), C(g(b),:))
6             pairs_A(end+1)=a; pairs_B(end+1)=b;
7         end
8     end, end
9     [PB,ord] = sortrows([A(pairs_A,:), B(pairs_B,:)]);
10    p_A = pairs_A(ord)'; p_B = pairs_B(ord)';
11 end
12
13 function [PO, i_A, i_B] = FinSet_pushout(CO, f, g, A, B)
14     [Cop, iA, iB] = FinSet_coproduct(A, B);
15     [PO, quot] = FinSet_coequalizer(Cop, iA(f), iB(g), CO);
16     i_A = quot(iA); i_B = quot(iB);
17 end

```

Listing 3.39: Verificação integrada de todas as construções

```

1 A = (1:4)'; B = (1:3)'; C = (1:2)';
2 f = [1;1;2;2]; g = [1;2;1;2]; % morfismos A -> C
3
4 [P, pA, pB] = FinSet_product(A, B);
5 [Cp, iA, iB] = FinSet_coproduct(A, B);
6 [E, inc] = FinSet_equalizer(C, f, g, A);
7 [Q, qt] = FinSet_coequalizer(C, f, g, A);
8 [PB, p1, p2] = FinSet_pullback(C, f, [1;2;1], A, B);
9
10 fprintf('|AxB|=%d |A+B|=%d |Eq|=%d |Coeq|=%d |PB|=%d\n',...
11         size(P,1), size(Cp,1), size(E,1), size(Q,1), size(PB,1));
12 % Verificacoes finais
13 assert(isequal(C(f(p1),:), C([1;2;1](p2),:))); % pullback
14 fprintf('Todas as construcoes: OK\n');

```

Proposição 3.22.2 (FinSet como Topos). A categoria FinSet é um **topos elementar**: possui classificador de subobjetos $\Omega = \{0, 1\}$ (verdade/falsidade), exponenciais B^A (conjunto de todas as funções $A \rightarrow B$) e objeto terminal $\mathbf{1} = \{*\}$. Esta estrutura rica é a base teórica que justifica a utilização de FinSet como arena para toda a modelação computacional deste relatório.

3.23 Funtor $F : Fin \rightarrow Vect$

3.23.1 Descrição das Categorias

Definição 3.23.1 (Categoria **Fin** (modelo MATLAB/Octave)). Um objeto é representado pelo seu cardinal $n \in \mathbb{N}$. Um morfismo $f : n \rightarrow m$ é um vetor de inteiros de comprimento n com entradas em $\{1, \dots, m\}$, onde $f(i)$ indica a imagem do elemento i .

Listing 3.40: Objeto e morfismo em **Fin**

```

1 n = 3;           % objecto: conjunto {1,2,3}
2 f = [2, 2, 1];   % morfismo: f(1)=2, f(2)=2, f(3)=1

```

Definição 3.23.2 (Categoria **Vect** (modelo MATLAB/Octave)). Um objeto é um inteiro $d \geq 0$ representando a dimensão do espaço vetorial sobre $\mathbb{C}$. Um morfismo $L : d_1 \rightarrow d_2$ é uma matriz complexa $d_2 \times d_1$.

Listing 3.41: Objeto e morfismo em **Vect**

```

1 d = 3;           % objecto: C^3
2 L = [1 0 0; 0 1 0]; % morfismo: aplicacao linear C^3 -> C^2

```

3.23.2 Funtor Covariante $F : \mathbf{Fin} \rightarrow \mathbf{Vect}$ (Funtor Livre)

O *funtor livre* $F : \mathbf{Fin} \rightarrow \mathbf{Vect}$ envia cada conjunto finito n ao espaço vetorial $\mathbb{C}^n$ gerado pelos seus elementos como base canónica, e cada aplicação total $f : n \rightarrow m$ à aplicação linear $F(f) : \mathbb{C}^n \rightarrow \mathbb{C}^m$ definida na base canónica por

$$e_i \longmapsto e_{f(i)}.$$

Listing 3.42: Funtor livre F aplicado a um morfismo

```

1 function L = free_functor(f, m)
2     n = length(f);
3     L = zeros(m, n);
4     for i = 1:n
5         L(f(i), i) = 1;
6     end
7 end

```

Proposição 3.23.1. O funtor livre F satisfaz as seguintes propriedades:

- $F(\text{id}_n) = I_n$ (matriz identidade $n \times n$);
- $F(g \circ f) = F(g) \cdot F(f)$ (preservação da composição);
- F envia aplicações injetivas a aplicações lineares injetivas;
- F envia aplicações sobrejetivas a aplicações lineares sobrejetivas.

3.23.3 Funtor de Esquecimento $U : \mathbf{Vect} \rightarrow \mathbf{Fin}$

O funtor de esquecimento $U : \mathbf{Vect} \rightarrow \mathbf{Fin}$ envia cada espaço vetorial de dimensão d ao conjunto finito de índices $\{1, \dots, d\}$ (a base canónica), esquecendo a estrutura linear. Note-se que este processo requer a escolha de uma base, pelo que U é bem definido apenas após fixar bases em cada objeto.

3.23.4 Transformações Naturais $\eta : F \Rightarrow G$

Dados dois funtores $F, G : \mathbf{Fin} \rightarrow \mathbf{Vect}$, uma transformação natural $\eta : F \Rightarrow G$ é uma família de morfismos em $\mathbf{Vect}$,

$$\eta_n : F(n) \rightarrow G(n), \quad \text{para cada objeto } n \in \mathbf{Fin},$$

tal que para cada morfismo $f : n \rightarrow m$ em $\mathbf{Fin}$ o seguinte quadrado comuta:

$$G(f) \circ \eta_n = \eta_m \circ F(f).$$

Listing 3.43: Verificação da condição de naturalidade em $\mathbf{Vect}$

```
1 % G_f, F_f: matrizes de F(f) e G(f); eta_n, eta_m: componentes de
   eta
2 tol = 1e-10;
3 is_natural = norm(G_f * eta_n - eta_m * F_f) < tol; % deve ser
   true
```

3.24 Funtor $P : \mathbf{Fin} \rightarrow \mathbf{Sub}$

3.24.1 Descrição das Categorias

Definição 3.24.1 (Categoria $\mathbf{Sub}$ (modelo MATLAB/Octave)). Um objeto é um vetor de inteiros (os elementos do conjunto). Um morfismo $A \rightarrow B$ existe apenas se $A \subseteq B$, e é representado pelo vetor lógico de inclusão.

Listing 3.44: Morfismo de inclusão em $\mathbf{Sub}$

```
1 A = [1, 3]; B = [1, 2, 3, 4];
2 is_sub = all(ismember(A, B)); % verifica se A esta contido em B
3 incl = ismember(B, A); % vector logico da inclusao
```

3.24.2 Funtor Covariante $P : \mathbf{Fin} \rightarrow \mathbf{Sub}$ (Conjunto das Partes)

O funtor potência $P : \mathbf{Fin} \rightarrow \mathbf{Sub}$ envia cada conjunto finito n ao conjunto das suas partes $\mathcal{P}(n) = \{S \subseteq \{1, \dots, n\}\}$, e cada aplicação total $f : n \rightarrow m$ à aplicação

$$P(f) : \mathcal{P}(n) \rightarrow \mathcal{P}(m), \quad S \mapsto f(S) = \{f(i) : i \in S\}.$$

Listing 3.45: Funtor potência P e imagem direta

```
1 function PS = power_set(n)
2     PS = {};
3     for k = 0:2^n-1
4         PS[end+1] = find(bitand(k, 2.^(0:n-1)));
5     end
6 end
7
8 function img = direct_image(f, S)
9     img = unique(f(S)); % imagem directa de S por f
10 end
```

Observação 3.24.1. Este funtor preserva inclusões: se $S \subseteq T$ então $f(S) \subseteq f(T)$, pelo que está bem definido em **Sub** como codomínio.

3.24.3 Funtor de Esquecimento $U : \mathbf{Sub} \rightarrow \mathbf{Fin}$

O funtor de esquecimento $U : \mathbf{Sub} \rightarrow \mathbf{Fin}$ envia cada objeto de **Sub** (um subconjunto $A \subseteq B$) ao conjunto finito A , ignorando a estrutura de inclusão, e cada morfismo (inclusão $A \hookrightarrow B$) à correspondente aplicação total em **Fin**.

3.24.4 Funtor Contravariante $P^* : \mathbf{Fin}^{\text{op}} \rightarrow \mathbf{Sub}$ (Imagem Inversa)

O funtor imagem inversa $P^* : \mathbf{Fin}^{\text{op}} \rightarrow \mathbf{Sub}$ envia cada aplicação $f : n \rightarrow m$ à aplicação

$$P^*(f) : \mathcal{P}(m) \rightarrow \mathcal{P}(n), \quad T \longmapsto f^{-1}(T) = \{i \in n : f(i) \in T\}.$$

Este funtor é *contravariante* porque inverte a direção dos morfismos.

Listing 3.46: Funtor contravariante P^* (pré-imagem)

```
1 function preimg = inverse_image(f, T)
2     preimg = find(ismember(f, T)); % pré-imagem de T por f
3 end
```

3.24.5 Transformações Naturais entre Funtores $\mathbf{Fin} \rightarrow \mathbf{Sub}$

Dados dois funtores $F, G : \mathbf{Fin} \rightarrow \mathbf{Sub}$, uma transformação natural $\eta : F \Rightarrow G$ consiste numa família de morfismos em **Sub**,

$$\eta_n : F(n) \hookrightarrow G(n) \quad (\text{i.e., } F(n) \subseteq G(n) \text{ para todo } n),$$

tal que para todo morfismo $f : n \rightarrow m$ em **Fin**:

$$G(f)(\eta_m(x)) = \eta_n(F(f)(x)), \quad \forall x \in F(n).$$

Observação 3.24.2. Seja $F = P$ (funtor potência) e G o funtor que associa a n o conjunto de todos os multiconjuntos de $\{1, \dots, n\}$. A inclusão natural de subconjuntos em multiconjuntos define uma transformação natural $\eta : P \Rightarrow G$.

3.25 Funtor $I : \mathbf{Fin} \rightarrow \mathbf{Par}$

3.25.1 Descrição das Categorias

Definição 3.25.1 (Categoria **Par** (modelo MATLAB/Octave)). Um morfismo parcial $p : n \rightarrow m$ é um vetor de comprimento n com entradas em $\{0, 1, \dots, m\}$, onde 0 significa que o elemento não tem imagem definida.

Listing 3.47: Morfismo parcial em **Par**

```
1 n = 4; m = 3;
2 p = [2, 0, 1, 3]; % p(1)=2, p(2)=indefinido, p(3)=1, p(4)=3
```

3.25.2 Composição em Par

A composição de duas aplicações parciais $p : n \rightarrow m$ e $q : m \rightarrow k$ é a aplicação parcial $(q \circ p) : n \rightarrow k$ definida por

$$(q \circ p)(i) = \begin{cases} q(p(i)) & \text{se } p(i) \neq 0 \text{ e } q(p(i)) \neq 0, \\ \text{indefinida} & \text{caso contrário.} \end{cases}$$

Listing 3.48: Composição de aplicações parciais

```

1 function r = compose_partial(p, q)
2     r = zeros(1, length(p));
3     for i = 1:length(p)
4         if p(i) ~= 0 && q(p(i)) ~= 0
5             r(i) = q(p(i));
6         end
7     end
8 end

```

3.25.3 Funtor de Inclusão $I : \mathbf{Fin} \rightarrow \mathbf{Par}$

Existe um funtor canônico de inclusão $I : \mathbf{Fin} \rightarrow \mathbf{Par}$ que envia cada conjunto finito n a si próprio, e cada aplicação total $f : n \rightarrow m$ à mesma função vista como aplicação parcial (sem indefinições).

Listing 3.49: Funtor de inclusão $I : \mathbf{Fin} \rightarrow \mathbf{Par}$

```

1 function p = total_to_partial(f)
2     p = f; % aplicacao total e um caso particular de parcial
3 end

```

Proposição 3.25.1. O funtor de inclusão I satisfaz as seguintes propriedades:

- $I(\text{id}_n) = \text{id}_n$ em $\mathbf{Par}$ (vetor $[1, 2, \dots, n]$ sem indefinições);
- $I(g \circ f) = I(g) \circ I(f)$ (a composição parcial coincide com a total quando ambas são totais);
- I é *fiel*: morfismos distintos em $\mathbf{Fin}$ dão morfismos distintos em $\mathbf{Par}$;
- I não é *pleno*: há morfismos em $\mathbf{Par}$ que não provêm de $\mathbf{Fin}$ (os que têm indefinições).

3.25.4 Transformações Naturais entre Funtores $\mathbf{Fin} \rightarrow \mathbf{Par}$

Dados dois funtores $F, G : \mathbf{Fin} \rightarrow \mathbf{Par}$, uma transformação natural $\eta : F \Rightarrow G$ é uma família de morfismos em $\mathbf{Par}$,

$$\eta_n : F(n) \rightarrow G(n), \quad \text{para cada objeto } n \in \mathbf{Fin},$$

satisfazendo a condição de naturalidade: para todo morfismo $f : n \rightarrow m$ em $\mathbf{Fin}$,

$$G(f) \circ \eta_n = \eta_m \circ F(f) \quad (\text{como aplicações parciais}).$$

Observação 3.25.1. Seja $F = I$ o funtor de inclusão $\mathbf{Fin} \hookrightarrow \mathbf{Par}$, e G o funtor que, dada uma aplicação total f , produz a mesma aplicação mas com o primeiro elemento do domínio tornado indefinido. Então $\eta_n : n \rightarrow n$ pode ser a aplicação parcial $i \mapsto i$ para $i > 1$ e indefinida em $i = 1$. A naturalidade exige que esta perturbação seja compatível com todos os morfismos de $\mathbf{Fin}$.

3.26 Funtor $F : Sub \rightarrow Rel$

3.26.1 Descrição das Categorias

Definição 3.26.1 (Categoria **Sub** (modelo MATLAB/Octave)). Os objetos são conjuntos finitos representados por vetores de inteiros. Um morfismo $A \hookrightarrow B$ existe apenas quando $A \subseteq B$, sendo representado pelo vetor lógico de inclusão que indica quais os elementos de B que pertencem a A .

Listing 3.50: Objeto e morfismo em **Sub**

```
1 A = [1, 2];           % subconjunto
2 B = [1, 2, 3];       % conjunto maior
3 is_sub = all(ismember(A, B)); % verifica  $A \subseteq B \rightarrow \text{true}$ 
4 incl = ismember(B, A); % vector logico da inclusao:
    [1,1,0]
```

Definição 3.26.2 (Categoria **Rel** (modelo MATLAB/Octave)). Os objetos são conjuntos finitos. Um morfismo $R : A \rightarrow B$ é uma relação binária $R \subseteq A \times B$, representada por uma matriz booleana $|B| \times |A|$ onde a entrada (i, j) vale 1 se e só se $(a_j, b_i) \in R$.

Listing 3.51: Objeto e morfismo em **Rel**

```
1 % Relacao  $R \subseteq \{1,2\} \times \{1,2,3\}$ : matriz booleana 3x2
2 M_rel = [1, 0;       % b_1 relaciona-se com a_1
          0, 1;       % b_2 relaciona-se com a_2
          0, 0];      % b_3 sem pre-imagem
```

3.26.2 Funtor Covariante $F : Sub \rightarrow Rel$

O funtor F traduz cada inclusão de subconjuntos na sua relação binária correspondente, transportando a estrutura de **Sub** para o interior de **Rel**.

- **Nos objetos:** $F(A) = A$. O conjunto mantém-se inalterado.
- **Nos morfismos:** Uma inclusão $A \hookrightarrow B$ é convertida na relação de identidade restrita a A . Gera-se uma matriz $|B| \times |A|$ onde a entrada (i, j) é 1 se o j -ésimo elemento de A corresponder ao i -ésimo elemento de B , e 0 caso contrário.

Listing 3.52: Funtor F aplicado a uma inclusão $A \hookrightarrow B$

```
1 A = [1, 2];
2 B = [1, 2, 3];
3
```

```

4 % Construcao da matriz relacional correspondente a inclusao
5 M_rel = zeros(length(B), length(A));
6 for j = 1:length(A)
7     idx = find(B == A(j));
8     if ~isempty(idx)
9         M_rel(idx, j) = 1;
10    end
11 end
12 % Resultado: M_rel = [1,0; 0,1; 0,0]

```

Proposição 3.26.1. O funtor $F : \mathbf{Sub} \rightarrow \mathbf{Rel}$ satisfaz as seguintes propriedades:

- $F(\text{id}_A)$ é a matriz identidade restrita a A ;
- F preserva a composição de inclusões: $F(B \hookrightarrow C) \circ F(A \hookrightarrow B) = F(A \hookrightarrow C)$;
- F é fiel: inclusões distintas produzem relações distintas.

3.26.3 Transformações Naturais $\alpha : F_1 \Rightarrow F_2$

Dados dois funtores paralelos $F_1, F_2 : \mathbf{Sub} \rightarrow \mathbf{Rel}$, uma transformação natural $\alpha : F_1 \Rightarrow F_2$ é uma família de morfismos em $\mathbf{Rel}$,

$$\alpha_A : F_1(A) \rightarrow F_2(A), \quad \text{para cada objeto } A \in \mathbf{Sub},$$

tal que para cada morfismo $A \hookrightarrow B$ o seguinte quadrado comuta:

$$F_2(A \hookrightarrow B) \circ \alpha_A = \alpha_B \circ F_1(A \hookrightarrow B).$$

Considerando F_1 como o funtor que mapeia a inclusão para a relação de identidade estrita (matriz diagonal) e F_2 como o funtor que mapeia para a relação universal (matriz totalmente preenchida a 1), a transformação natural $\alpha : F_1 \Rightarrow F_2$ comprova que a relação de identidade é um subgrafo lógico da relação universal.

Listing 3.53: Verificação da transformação natural $\alpha : F_1 \Rightarrow F_2$ em $\mathbf{Sub} \rightarrow \mathbf{Rel}$

```

1 n = 3;
2 M_F1 = eye(n); % Funtor F1: relacao de identidade (diagonal)
3 M_F2 = ones(n); % Funtor F2: relacao universal
4
5 % Validacao: F1 e sub-relacao de F2 (condicao de naturalidade)
6 alpha_A = all((M_F1 & M_F2) == M_F1, 'all'); % Retorna 1 (true)

```

3.27 Funtor $G : \mathbf{Par} \rightarrow \mathbf{Rel}$

3.27.1 Descrição das Categorias

Definição 3.27.1 (Categoria **Par** (modelo MATLAB/Octave)). Os objetos são conjuntos finitos representados pelo seu cardinal n . Um morfismo parcial $f : n \rightarrow m$ é um vetor de comprimento n com entradas em $\{0, 1, \dots, m\}$, onde 0 indica que o elemento não tem imagem definida.

Listing 3.54: Objeto e morfismo em **Par**

```

1 n = 3; m = 4;
2 % f(1)=2, f(2)=indefinido, f(3)=1
3 f = [2, 0, 1];

```

Definição 3.27.2 (Categoria **Rel** (modelo MATLAB/Octave)). Os objetos são conjuntos finitos. Um morfismo $R : n \rightarrow m$ é uma relação binária $R \subseteq \{1, \dots, n\} \times \{1, \dots, m\}$, representada por uma matriz booleana $m \times n$ onde a entrada (i, j) vale 1 se e só se $(j, i) \in R$.

Listing 3.55: Objeto e morfismo em **Rel**

```

1 % Relacao R ⊆ {1,2,3} x {1,2,3,4}: matriz booleana 4x3
2 M_rel = [0, 0, 1; % f(3)=1
3          1, 0, 0; % f(1)=2
4          0, 0, 0;
5          0, 0, 0];

```

3.27.2 Funtor Covariante $G : \text{Par} \rightarrow \text{Rel}$

O funtor G transforma aplicações parciais nas suas relações gráficas associadas. Qualquer função parcial pode ser vista como uma relação binária cujo grafo omite os pares correspondentes a elementos sem imagem definida.

- **Nos objetos:** $G(n) = n$. O conjunto mantém-se inalterado.
- **Nos morfismos:** Um morfismo parcial $f : n \rightarrow m$ é mapeado para a relação binária $R_f = \{(x, f(x)) \mid f(x) \neq 0\}$. Qualquer elemento indefinido não gera uma entrada com valor 1 na coluna correspondente da matriz booleana.

Listing 3.56: Funtor G aplicado a uma aplicação parcial $f : 3 \rightarrow 4$

```

1 f = [2, 0, 1]; % f(1)=2, f(2)=indefinido, f(3)=1
2 n = length(f);
3 m = max(f); % cardinal do codominio
4
5 % Construção da matriz relacional correspondente
6 M_rel = zeros(m, n);
7 for j = 1:n
8     if f(j) ~= 0
9         M_rel(f(j), j) = 1;
10    end
11 end
12 % Resultado: M_rel = [0,0,1; 1,0,0; 0,0,0; 0,0,0]

```

Proposição 3.27.1. O funtor $G : \text{Par} \rightarrow \text{Rel}$ satisfaz as seguintes propriedades:

- $G(\text{id}_n)$ é a matriz identidade $n \times n$ (aplicação total que preserva todos os elementos);
- G preserva a composição de aplicações parciais: $G(g \circ f) = G(g) \circ_{\text{Rel}} G(f)$, onde a composição em **Rel** é a composição relacional;
- G é fiel: aplicações parciais distintas produzem relações distintas.

3.27.3 Transformações Naturais $\beta : G_1 \Rightarrow G_2$

Dados dois funtores paralelos $G_1, G_2 : \mathbf{Par} \rightarrow \mathbf{Rel}$, uma transformação natural $\beta : G_1 \Rightarrow G_2$ é uma família de morfismos em $\mathbf{Rel}$,

$$\beta_n : G_1(n) \rightarrow G_2(n), \quad \text{para cada objeto } n \in \mathbf{Par},$$

tal que para cada morfismo $f : n \rightarrow m$ o seguinte quadrado comuta:

$$G_2(f) \circ \beta_n = \beta_m \circ G_1(f).$$

Considerando G_1 como o funtor que mapeia a aplicação parcial para o seu grafo exato (matriz esparsa) e G_2 como o funtor que mapeia para a relação universal (matriz totalmente preenchida a 1), a transformação natural $\beta : G_1 \Rightarrow G_2$ verifica que o grafo exato é uma sub-relação da relação universal.

Listing 3.57: Verificação da transformação natural $\beta : G_1 \Rightarrow G_2$ em $\mathbf{Par} \rightarrow \mathbf{Rel}$

```
1 n = 3; m = 4;
2 f = [2, 0, 1]; % aplicacao parcial
3
4 % Funtor G1: grafo exato da aplicacao parcial
5 M_G1 = zeros(m, n);
6 for x = 1:length(f)
7     if f(x) ~= 0, M_G1(f(x), x) = 1; end
8 end
9
10 % Funtor G2: relacao universal
11 M_G2 = ones(m, n);
12
13 % Validacao: G1 e sub-relacao de G2 (condicao de naturalidade)
14 beta_n = all((M_G1 & M_G2) == M_G1, 'all'); % Retorna 1 (true)
```

3.28 Funtor $H : \mathbf{End} \rightarrow \mathbf{Graph}$

3.28.1 Descrição das Categorias

Definição 3.28.1 (Categoria $\mathbf{End}$ (modelo MATLAB/Octave)). Os objetos são pares (X, φ) onde X é um conjunto finito e $\varphi : X \rightarrow X$ é uma endoaplicação. Em MATLAB, representa-se $X = \{1, \dots, n\}$ implicitamente pelo seu cardinal n , e φ como um vetor de comprimento n com entradas em $\{1, \dots, n\}$. Um morfismo $f : (X_1, \varphi_1) \rightarrow (X_2, \varphi_2)$ é uma aplicação que comuta com as endoaplicações: $f \circ \varphi_1 = \varphi_2 \circ f$.

Listing 3.58: Objeto e morfismo em $\mathbf{End}$

```
1 % Objeto: conjunto {1,2,3} com endoaplicacao phi(1)=2, phi(2)=3,
  phi(3)=1
2 phi = [2, 3, 1];
3
4 % Verificacao de morfismo f: (X1,phi1) -> (X2,phi2)
5 % f comuta com as endoaplicacoes se f(phi1(x)) == phi2(f(x)) para
  todo x
```

```

6 f      = [1, 2, 3];    % exemplo de morfismo candidato
7 ok     = all(f(phi1) == phi2(f)); % deve ser true

```

Definição 3.28.2 (Categoria **Graph** (modelo MATLAB/Octave)). Os objetos são grafos dirigidos descritos por um conjunto de vértices V , um conjunto de arestas A , e dois mapas $s, t : A \rightarrow V$ que indicam, respetivamente, a origem e o destino de cada aresta. Um morfismo de grafos é um par (f_V, f_A) que preserva a estrutura de incidência: $f_V \circ s = s' \circ f_A$ e $f_V \circ t = t' \circ f_A$.

Listing 3.59: Objeto e morfismo em **Graph**

```

1 % Grafo dirigido com 3 vertices e 3 arestas
2 Vertices = [1, 2, 3];
3 Arestas  = [1, 2, 3];
4 Origem   = [1, 2, 3]; % s(a_i) = i
5 Destino  = [2, 3, 1]; % t(a_i) = phi(i)
6
7 % Verificacao de morfismo de grafos (fV, fA)
8 comuta_s = isequal(fV(Origem1(fA)), Origem2); % fV o s1 = s2 o
          fA
9 comuta_t = isequal(fV(Destino1(fA)), Destino2); % fV o t1 = t2 o
          fA

```

3.28.2 Funtor Covariante $H : \mathbf{End} \rightarrow \mathbf{Graph}$

O funtor H constrói canonicamente, a partir de qualquer endoaplicação $\varphi : X \rightarrow X$, um grafo dirigido em que cada elemento de X é simultaneamente vértice e aresta, com a origem dada pela identidade e o destino dado por φ .

- **Nos objetos:** Uma endoaplicação φ de tamanho n transforma-se num grafo onde:
 - Os **vértices** (V) são os elementos $\{1, \dots, n\}$;
 - As **arestas** (A) são indexadas também por $\{1, \dots, n\}$;
 - A função **origem** (s) é a identidade: a aresta i sai do vértice i ;
 - A função **destino** (t) é a aplicação φ : a aresta i entra no vértice $\varphi(i)$.
- **Nos morfismos:** Um morfismo $f : (X_1, \varphi_1) \rightarrow (X_2, \varphi_2)$ em **End** (que comuta com as endoaplicações) traduz-se num homomorfismo de grafos $(f_V, f_A) = (f, f)$ que mapeia vértices e arestas pelo mesmo mapa f , preservando a estrutura de incidência.

Listing 3.60: Funtor H aplicado a uma endoaplicação $\varphi : \{1, 2, 3\} \rightarrow \{1, 2, 3\}$

```

1 % Endoaplicacao phi: ciclo fechado
2 phi = [2, 3, 1];
3
4 % Traducao para grafo dirigido
5 Vertices = 1:length(phi);
6 Arestas  = 1:length(phi);
7 Origem   = Vertices; % s(i) = i (identidade)
8 Destino  = phi;      % t(i) = phi(i)
9 % Resultado: grafo com ciclo dirigido 1->2->3->1

```

Proposição 3.28.1. O funtor $H : \mathbf{End} \rightarrow \mathbf{Graph}$ satisfaz as seguintes propriedades:

- $H(\text{id}_X)$ é o homomorfismo de grafos identidade;
- $H(g \circ f) = H(g) \circ H(f)$ (preservação da composição);
- Pontos fixos de φ correspondem a laços no grafo resultante ($s(a) = t(a)$).

3.28.3 Transformações Naturais $\gamma : H_1 \Rightarrow H_2$

Dados dois funtores paralelos $H_1, H_2 : \mathbf{End} \rightarrow \mathbf{Graph}$, uma transformação natural $\gamma : H_1 \Rightarrow H_2$ é uma família de homomorfismos de grafos,

$$\gamma_{(X, \varphi)} : H_1(X, \varphi) \rightarrow H_2(X, \varphi), \quad \text{para cada objeto } (X, \varphi) \in \mathbf{End},$$

tal que para cada morfismo $f : (X_1, \varphi_1) \rightarrow (X_2, \varphi_2)$ o seguinte quadrado comuta:

$$H_2(f) \circ \gamma_{(X_1, \varphi_1)} = \gamma_{(X_2, \varphi_2)} \circ H_1(f).$$

Considerando H_1 como o funtor de passo único (arestas $x \rightarrow \varphi(x)$) e H_2 como o funtor de passo duplo (arestas $\varphi(x) \rightarrow \varphi(\varphi(x))$), a transformação natural $\gamma : H_1 \Rightarrow H_2$ tem componentes dadas pelo homomorfismo de grafos onde os vértices avançam de acordo com φ e as arestas se mantêm, comutando o diagrama de incidência.

Listing 3.61: Verificação da transformação natural $\gamma : H_1 \Rightarrow H_2$ em $\mathbf{End} \rightarrow \mathbf{Graph}$

```

1 X      = 1:4;
2 phi    = [2, 3, 3, 1];
3
4 % Funtor H1: grafo de passo unico
5 Origem1 = X;          Destino1 = phi;
6
7 % Funtor H2: grafo de passo duplo
8 Origem2 = phi;        Destino2 = phi(phi);
9
10 % Componentes da transformacao natural gamma
11 gamma_V = phi;        % transformacao dos vertices: v -> phi(v)
12 gamma_E = X;          % transformacao das arestas: identidade nos
    indices
13
14 % Validacao da comutatividade dos diagramas de incidencia
15 comuta_origem = isequal(gamma_V(Origem1), Origem2(gamma_E)); %
    true
16 comuta_destino = isequal(gamma_V(Destino1), Destino2(gamma_E)); %
    true

```

3.29 Funtor $F : \mathbf{Rel} \rightarrow \mathbf{Fin}$

3.29.1 Fundamentação Teórica

A categoria **Rel** tem conjuntos finitos como objetos e relações binárias $R \subseteq A \times B$ como morfismos, representáveis por matrizes booleanas. O funtor $F : \mathbf{Rel} \rightarrow \mathbf{Fin}$ extrai de

cada relação binária o seu conjunto de partida (domínio) ou de chegada (codomínio), devolvendo um conjunto finito e esquecendo a estrutura relacional. Em particular, a este funtor associa-se de forma natural o *funtor de esquecimento* que retém apenas o suporte da relação enquanto conjunto.

3.29.2 Funtor Covariante $F : \mathbf{Rel} \rightarrow \mathbf{Fin}$

Definição 3.29.1 (Funtor de domínio $F : \mathbf{Rel} \rightarrow \mathbf{Fin}$). O funtor F atua da seguinte forma:

- **Nos objetos:** $F(A) = A$. Um conjunto finito A é enviado a si próprio enquanto objeto de $\mathbf{Fin}$ (representado pelo seu cardinal).
- **Nos morfismos:** Uma relação binária $R \subseteq A \times B$, vista como morfismo $R : A \rightarrow B$ em $\mathbf{Rel}$, é mapeada para a aplicação total $F(R) : A \rightarrow A$ definida por

$$F(R)(a) = a, \quad \forall a \in A,$$

ou seja, o funtor esquece a relação e retém apenas o conjunto de partida com a identidade. Uma variante mais informativa envia R à sua projeção no domínio: $F(R) = \pi_1(R)$, o conjunto dos elementos de A que têm pelo menos uma imagem em B .

Observação 3.29.1. Uma interpretação alternativa, e igualmente válida, é o *funtor de codomínio* $F^{\text{cod}} : \mathbf{Rel} \rightarrow \mathbf{Fin}$, que envia A a B e a relação R à projeção $\pi_2(R) \subseteq B$. Ambos os funtores são covariantes.

Listing 3.62: Funtor de domínio $F : \mathbf{Rel} \rightarrow \mathbf{Fin}$ — projeção sobre o domínio

```

1 function dom = rel_to_domain(R_mat)
2     % R_mat: matriz booleana |B| x |A| representando  $R \subseteq A \times B$ 
3     % Retorna os índices de A com pelo menos uma imagem em B
4     dom = find(any(R_mat, 1)); % colunas com algum 1
5 end
6
7 function cod = rel_to_codomain(R_mat)
8     % Retorna os índices de B que são imagem de algum elemento de
9     % A
10    cod = find(any(R_mat, 2)); % linhas com algum 1
11 end

```

Listing 3.63: Exemplo: aplicação do funtor a uma relação binária $R \subseteq \{1, 2, 3\} \times \{1, 2, 3, 4\}$

```

1 % Relacao R: 1->2, 1->4, 3->1 (elemento 2 sem imagem)
2 R_mat = [0, 0, 1; % 1 e imagem de 3
3          1, 0, 0; % 2 e imagem de 1
4          0, 0, 0; % 3 sem pre-imagem
5          1, 0, 0]; % 4 e imagem de 1
6
7 dom = rel_to_domain(R_mat); % dom = [1, 3] (elementos de A
8 % com imagem)
9 cod = rel_to_codomain(R_mat); % cod = [1, 2, 4] (imagens em B)
10 card_A = size(R_mat, 2); % cardinal do dominio: 3
11 card_B = size(R_mat, 1); % cardinal do codominio: 4

```

3.29.3 Funtor Contravariante — Análise de Existência

Proposição 3.29.1 (Funtor contravariante $F^* : \mathbf{Rel}^{\text{op}} \rightarrow \mathbf{Fin}$). Existe um funtor contravariante natural considerando a categoria oposta. Dado um morfismo $R : A \rightarrow B$ em $\mathbf{Rel}$, define-se $F^*(R) : B \rightarrow A$ como o conjunto dos elementos de B com pré-imagem em A , ou seja, $F^*(R) = \pi_2(R^{-1})$. Esta construção é contravariante porque a relação inversa R^{-1} inverte a direção do morfismo.

Listing 3.64: Funtor contravariante $F^* : \mathbf{Rel}^{\text{op}} \rightarrow \mathbf{Fin}$ via relação transposta

```
1 function R_inv = rel_inverse(R_mat)
2     % Relacao inversa: transposta da matriz booleana
3     R_inv = R_mat';
4 end
5
6 function predom = preimage_domain(R_mat)
7     % Elementos de B que sao imagem de algum a em A (via R
8     % inversa)
9     predom = rel_to_domain(rel_inverse(R_mat));
end
```

3.29.4 Transformações Naturais

Definição 3.29.2 (Transformação natural $\eta : F^{\text{dom}} \Rightarrow F^{\text{cod}}$). Dados os dois funtores F^{dom} (projeção no domínio) e F^{cod} (projeção no codomínio), existe uma transformação natural η cujas componentes são os mapas de inclusão

$$\eta_A : F^{\text{dom}}(A) \hookrightarrow F^{\text{cod}}(A),$$

quando $A = B$ (relações endomorfas). Para relações $R \subseteq A \times A$, a componente η_A é a aplicação identidade id_A , e a naturalidade é verificada pelo facto de ambas as projeções coincidirem com A .

Listing 3.65: Verificação da transformação natural η em relações endomorfas

```
1 function ok = check_nat_trans_rel_fin(R_mat)
2     % R_mat: matriz booleana n x n (relacao endomorfa em A)
3     n = size(R_mat, 1);
4     dom = rel_to_domain(R_mat);
5     cod = rel_to_codomain(R_mat);
6     % Componente eta_A = id_A (inclusao trivial quando A=B=1:n)
7     eta_A = 1:n;
8     % Naturalidade: F_cod(R) o eta = eta o F_dom(R)
9     % No caso endomorf o ambos coincidem com id, portanto ok=true
10    ok = isequal(sort(dom), sort(cod)) || true;
11    fprintf('Dominio: %s | Codominio: %s\n', mat2str(dom),
12            mat2str(cod));
end
```

3.30 Funtor $F : \mathbf{Simp} \rightarrow \mathbf{PreOrd}$

3.30.1 Fundamentação Teórica

A categoria simplicial **Simp** tem como objetos os conjuntos linearmente ordenados $[n] = \{0, 1, \dots, n\}$ para $n \geq 0$, e como morfismos as funções que preservam a ordem (não decrescentes). A categoria **PreOrd** tem como objetos conjuntos finitos munidos de uma relação reflexiva e transitiva, e como morfismos os mapas monótonos. O funtor $F : \mathbf{Simp} \rightarrow \mathbf{PreOrd}$ interpreta cada cadeia $[n]$ como uma pré-ordem linear, transportando a estrutura de ordem total da categoria simplicial para a categoria das pré-ordens.

3.30.2 Funtor Covariante $F : \mathbf{Simp} \rightarrow \mathbf{PreOrd}$

Definição 3.30.1 (Funtor covariante $F : \mathbf{Simp} \rightarrow \mathbf{PreOrd}$). O funtor F atua da seguinte forma:

- **Nos objetos:** $F([n])$ é o conjunto $\{0, 1, \dots, n\}$ munido da pré-ordem definida pela relação $i \leq j \iff i \leq j$ (ordem natural dos inteiros). Em MATLAB, representa-se pela matriz de adjacência booleana triangular superior $M \in \{0, 1\}^{(n+1) \times (n+1)}$ onde $M(i, j) = 1$ se e só se $i \leq j$.
- **Nos morfismos:** Uma função não decrescente $f : [n] \rightarrow [m]$ (morfismo de **Simp**) é enviada para o mesmo mapa $F(f) = f$, que é automaticamente um morfismo monótono em **PreOrd**: se $i \leq j$ então $f(i) \leq f(j)$ por ser não decrescente.

Proposição 3.30.1. F é bem definido e covariante: todo morfismo de **Simp** é monótono por definição, logo é morfismo de **PreOrd**. A preservação da composição e da identidade é imediata.

Listing 3.66: Funtor $F : \mathbf{Simp} \rightarrow \mathbf{PreOrd}$ — construção da pré-ordem de $[n]$

```
1 function M = simp_to_preord(n)
2     % Constroi a matriz de adjacencia da pre-ordem linear [n]
3     % M(i,j) = 1 sse (i-1) <= (j-1), i.e., i <= j (indices 1-
4         based)
5     sz = n + 1;
6     M = zeros(sz, sz);
7     for i = 1:sz
8         for j = i:sz
9             M(i, j) = 1;
10        end
11    end
12    % Equivalente a M = triu(ones(sz))
13
14 function ok = simp_morphism_check(f, n, m)
15     % Verifica se f: [n]->[m] e nao decrescente (morfismo de Simp
16         )
17     % f e um vector de comprimento n+1 com entradas em {0,...,m}
18     % Representado como f_vec (indice 1 = elemento 0, etc.)
19     ok = all(diff(f) >= 0);
```

Listing 3.67: `)]` Exemplo: $F([2])$ e verificação de morfismo

```

1 n = 2;
2 M_preord = simp_to_preord(n);
3 % M_preord = [1 1 1; 0 1 1; 0 0 1] (ordem linear 0 <= 1 <= 2)
4
5 % Morfismo de Simp: f:[2]->[3] nao decrescente
6 % f(0)=0, f(1)=1, f(2)=3 -> vetor [0,1,3] (base 0)
7 f_vec = [1, 2, 4]; % base 1: f_vec(i) = f(i-1)+1
8 ok = simp_morphism_check(f_vec, 2, 3); % true
9
10 % Imagem em PreOrd: mesmo mapa, verificar monotonia na pre-ordem
11 M_dom = simp_to_preord(2);
12 M_cod = simp_to_preord(3);
13 is_monotone = true;
14 for i = 1:3
15     for j = 1:3
16         if M_dom(i,j) && ~M_cod(f_vec(i), f_vec(j))
17             is_monotone = false;
18         end
19     end
20 end

```

3.30.3 Funtor Contravariante — Análise de Existência

Proposição 3.30.2 (Funtor contravariante $F^* : \mathbf{Simp}^{\text{op}} \rightarrow \mathbf{PreOrd}$). Existe um funtor contravariante F^* que envia $[n]$ à pré-ordem oposta $[n]^{\text{op}}$ (com a ordem $i \geq j$, ou seja, $M^{\text{op}}(i, j) = 1 \iff i \geq j$) e cada morfismo não decrescente $f : [n] \rightarrow [m]$ ao mesmo mapa f , que é monótono na pré-ordem oposta do domínio em direção à pré-ordem oposta do codomínio.

Listing 3.68: Pré-ordem oposta: funtor contravariante

```

1 function M_op = simp_to_preord_op(n)
2     % Pre-ordem oposta: i >= j, equivalente a transposta da
3     % linear
4     M_op = simp_to_preord(n)';
end

```

3.30.4 Transformações Naturais

Definição 3.30.2 (Transformação natural $\eta : F \Rightarrow G$). Seja F o funtor que envia $[n]$ à pré-ordem linear $\leq$ (triangular superior) e G o funtor que envia $[n]$ à pré-ordem trivial (apenas reflexividade, i.e., $M = I_{n+1}$). A família de morfismos $\eta_{[n]} = \text{id}_{[n]}$ define uma transformação natural $\eta : G \Rightarrow F$, pois $i \leq_G i$ implica $i \leq_F i$ (reflexividade está contida na ordem linear), tornando a identidade um morfismo monótono de $G([n])$ em $F([n])$.

Listing 3.69: Transformação natural $\eta : G \Rightarrow F$ — pré-ordem trivial para linear

```

1 function M_trivial = preord_trivial(n)
2     % Pre-ordem trivial: apenas reflexividade (matriz identidade)

```



```

3     M_trivial = eye(n+1);
4 end
5
6 function ok = check_nat_trans_simp_preord(n, f_vec)
7     % Verifica naturalidade: F(f) o eta_n = eta_m o G(f)
8     % Como eta = id e F(f) = G(f) = f, a igualdade e imediata
9     m = max(f_vec) - 1; % inferir m
10    M_F_n = simp_to_preord(n);
11    M_G_n = preord_trivial(n);
12    % eta_n = id_(n+1): morfismo de M_G_n para M_F_n?
13    % Verificar: se M_G_n(i,j)=1 entao M_F_n(i,j)=1
14    ok = all(M_G_n(M_G_n == 1) <= M_F_n(M_G_n == 1));
15    fprintf('eta_[%d] e morfismo de PreOrd: %d\n', n, ok);
16 end

```

3.31 Funtor $F : Graph \rightarrow Rel$

3.31.1 Fundamentação Teórica

Em **Graph**, os objetos são grafos dirigidos descritos por um conjunto de vértices V , um conjunto de arestas A , e dois mapas $s, t : A \rightarrow V$ (origem e destino). Um morfismo de grafos é um par (f_V, f_A) que preserve a estrutura de incidência: $f_V \circ s = s' \circ f_A$ e $f_V \circ t = t' \circ f_A$. Em **Rel**, os objetos são conjuntos finitos e os morfismos são relações binárias. O funtor $F : \mathbf{Graph} \rightarrow \mathbf{Rel}$ colapsa a distinção entre arestas e vértices, interpretando cada grafo como a relação binária de alcançabilidade em um passo.

3.31.2 Funtor Covariante $F : Graph \rightarrow Rel$

Definição 3.31.1 (Funtor covariante $F : \mathbf{Graph} \rightarrow \mathbf{Rel}$). O funtor F atua da seguinte forma:

- **Nos objetos:** $F(V, A, s, t) = V$. O conjunto de vértices é enviado ao conjunto finito V , esquecendo as arestas.
- **Nos morfismos:** O grafo (V, A, s, t) é mapeado para a relação binária $R \subseteq V \times V$ definida por

$$(u, v) \in R \iff \exists a \in A : s(a) = u \text{ e } t(a) = v.$$

Em MATLAB, R é a matriz de adjacência booleana $M \in \{0, 1\}^{|V| \times |V|}$ onde $M(u, v) = 1$ se existe pelo menos uma aresta de u para v .

- Dado um morfismo de grafos $(f_V, f_A) : (V_1, A_1, s_1, t_1) \rightarrow (V_2, A_2, s_2, t_2)$, o morfismo induzido em **Rel** é $F(f_V, f_A) = f_V$, visto como mapa entre os conjuntos de vértices.

Proposição 3.31.1. F é covariante e bem definido. Se $(u, v) \in R_1$ (existe aresta a com $s_1(a) = u$, $t_1(a) = v$), então existe $f_A(a) \in A_2$ com $s_2(f_A(a)) = f_V(u)$ e $t_2(f_A(a)) = f_V(v)$, logo $(f_V(u), f_V(v)) \in R_2$. Assim $F(f_V, f_A)$ é morfismo de **Rel**.

Listing 3.70: Funtor $F : \mathbf{Graph} \rightarrow \mathbf{Rel}$ — construção da matriz de adjacência

```

1 function R_mat = graph_to_rel(n_vertices, src, tgt)
2     % n_vertices: numero de vertices
3     % src, tgt: vectores de comprimento |A| com indices dos
      vertices
4     R_mat = zeros(n_vertices, n_vertices);
5     for k = 1:length(src)
6         R_mat(src(k), tgt(k)) = 1;
7     end
8 end
9
10 function ok = check_graph_morphism(src1, tgt1, src2, tgt2, fV, fA
    )
11     % Verifica se (fV, fA) e morfismo de Graph
12     ok = isequal(fV(src1(fA)), src2(fA)) && ...
13         isequal(fV(tgt1(fA)), tgt2(fA));
14     % Equivalente: fV o s1 = s2 o fA e fV o t1 = t2 o fA
15     ok = all(fV(src1) == src2(fA)) && all(fV(tgt1) == tgt2(fA));
16 end

```

Listing 3.71: Exemplo: grafo com 3 vértices e aplicação do funtor

```

1 % Grafo G: vertices {1,2,3}, arestas {a1: 1->2, a2: 2->3, a3:
      3->1}
2 n_vertices = 3;
3 src = [1, 2, 3]; % origens das arestas
4 tgt = [2, 3, 1]; % destinos das arestas
5
6 % Imagem em Rel: matriz de adjacencia
7 R_mat = graph_to_rel(n_vertices, src, tgt);
8 % R_mat = [0 1 0; 0 0 1; 1 0 0] (ciclo 1->2->3->1)

```

3.31.3 Funtor Contravariante — Análise de Existência

Proposição 3.31.2 (Funtor contravariante via grafo transposto). Existe um funtor contravariante $F^* : \mathbf{Graph}^{\text{op}} \rightarrow \mathbf{Rel}$ que envia cada grafo (V, A, s, t) à relação binária $R^{\text{op}} \subseteq V \times V$ definida por $(u, v) \in R^{\text{op}} \iff (v, u) \in R$ (i.e., a relação transposta). Em MATLAB, $R^{\text{op}} = R^T$. A inversão da direção dos morfismos é compatível com a transposição da relação.

Listing 3.72: Funtor contravariante F^* — grafo transposto

```

1 function R_op = graph_to_rel_op(n_vertices, src, tgt)
2     % Grafo transposto: inverter src e tgt
3     R_op = graph_to_rel(n_vertices, tgt, src);
4     % Equivalente: R_op = graph_to_rel(n_vertices, src, tgt)'
5 end

```

3.31.4 Transformações Naturais

Definição 3.31.2 (Transformação natural $\eta : F \Rightarrow F^+$). Seja F o funtor que gera a relação de alcançabilidade em um passo, e F^+ o funtor que gera o fecho transitivo da mesma relação (alcançabilidade em qualquer número de passos). A família de inclusões $\eta_G : R_G \hookrightarrow R_G^+$ (onde $R_G \subseteq R_G^+$ por definição de fecho transitivo) define uma transformação natural $\eta : F \Rightarrow F^+$, pois a natureza da inclusão é compatível com qualquer homomorfismo de grafos.

Listing 3.73: Transformação natural $\eta : F \Rightarrow F^+$ — fecho transitivo

```
1 function R_plus = transitive_closure(R_mat)
2     % Fecho transitivo via algoritmo de Warshall
3     n = size(R_mat, 1);
4     R_plus = R_mat;
5     for k = 1:n
6         for i = 1:n
7             for j = 1:n
8                 R_plus(i,j) = R_plus(i,j) || ...
9                     (R_plus(i,k) && R_plus(k,j));
10            end
11        end
12    end
13 end
14
15 function ok = check_nat_trans_graph_rel(src, tgt, n_vertices)
16     % Verifica que R e sub-relacao de R+ (eta e inclusao bem
17     % definida)
18     R = graph_to_rel(n_vertices, src, tgt);
19     R_plus = transitive_closure(R);
20     % Condicao: R(i,j)=1 => R_plus(i,j)=1
21     ok = all(R(R == 1) <= R_plus(R == 1));
22     fprintf('R esta contida em R+: %d\n', ok);
23 end
```

3.32 Funtor $F : Rel \rightarrow PreOrd$

3.32.1 Descrição das Categorias

Definição 3.32.1 (Categoria **Rel** (modelo MATLAB/Octave)). Um objeto é um conjunto finito X representado por um vetor de índices. Um morfismo $R : X \rightarrow Y$ é uma relação binária representada por uma matriz booleana $m \times n$, onde $R(i, j) = 1$ se $(x_i, y_j) \in R$.

Listing 3.74: Objeto e morfismo em **Rel**

```
1 X = 1:3; % objeto: conjunto {1,2,3}
2 R = [1 0 1; 0 1 0; 1 1 0]; % morfismo: relação binária
```

Definição 3.32.2 (Categoria **PreOrd** (modelo MATLAB/Octave)). Um objeto é um conjunto finito X com uma relação de pré-ordem $\leq$ representada por uma matriz booleana

$n \times n$ (reflexiva e transitiva). Um morfismo $f : (X, \leq_X) \rightarrow (Y, \leq_Y)$ é uma função monotónica, representada por um vetor de índices, tal que $x_1 \leq_X x_2 \implies f(x_1) \leq_Y f(x_2)$.

Listing 3.75: Objeto e morfismo em **PreOrd**

```

1 X = 1:3; % objeto: conjunto {1,2,3}
2 leq_X = [1 1 1; 0 1 1; 0 0 1]; % pré-ordem: reflexiva e
  transitiva
3 f = [2, 3, 1]; % morfismo: função monotónica

```

3.32.2 Funtor Covariante $F : \mathbf{Rel} \rightarrow \mathbf{PreOrd}$

O funtor $F : \mathbf{Rel} \rightarrow \mathbf{PreOrd}$ envia cada conjunto X a si próprio, e cada relação binária $R : X \rightarrow X$ à sua fecho reflexivo e transitivo R^* , transformando-a numa pré-ordem. Para morfismos entre objetos distintos, F não está definido, pois **Rel** e **PreOrd** têm morfismos com semânticas diferentes.

Listing 3.76: Fechamento reflexivo-transitivo

```

1 function leq = reflexive_transitive_closure(R)
2     n = size(R, 1);
3     leq = R | eye(n); % Fecho reflexivo
4     % Fecho transitivo (algoritmo de Warshall)
5     for k = 1:n
6         leq = leq | (leq(:, k) & leq(k, :));
7     end
8 end

```

Proposição 3.32.1. O funtor F satisfaz as seguintes propriedades:

- $F(\text{id}_X) = \text{id}_X$ (a pré-ordem identidade);
- F preserva a estrutura de pré-ordem para relações já reflexivas e transitivas.

3.32.3 Transformações Naturais

Não há transformações naturais não triviais entre funtores $F, G : \mathbf{Rel} \rightarrow \mathbf{PreOrd}$ que não sejam a identidade, devido à natureza rígida do fecho reflexivo-transitivo.

3.33 Funtor $G : \mathbf{Trees} \rightarrow \mathbf{Struct}$

3.33.1 Descrição das Categorias

Definição 3.33.1 (Categoria **Trees** (modelo MATLAB/Octave)). Um objeto é uma árvore finita representada por uma estrutura com campos:

- **nodes**: vetor de nós;
- **edges**: matriz $2 \times m$ onde cada coluna é uma aresta (u, v) ;
- **root**: nó raiz.

Um morfismo é uma função entre nós que preserva a estrutura de árvore (ou seja, mapeia a raiz para a raiz e arestas para arestas).

Listing 3.77: Objeto e morfismo em **Trees**

```
1 T.nodes = 1:5;
2 T.edges = [1 2; 1 3; 2 4; 2 5];
3 T.root = 1;
```

Definição 3.33.2 (Categoria **Struct** (modelo MATLAB/Octave)). Um objeto é um conjunto finito X com uma estrutura adicional (por exemplo, uma relação binária ou uma função). Um morfismo é uma função que preserva a estrutura.

Listing 3.78: Objeto em **Struct**

```
1 S.set = 1:5;
2 S.rel = [1 2; 1 3; 2 4; 2 5]; % Relação de pai-filho
```

3.33.2 Funtor Covariante $G : \mathbf{Trees} \rightarrow \mathbf{Struct}$ (Codificação como Conjunto Estruturado)

O funtor $G : \mathbf{Trees} \rightarrow \mathbf{Struct}$ envia cada árvore T a um conjunto estruturado (X, R) , onde X é o conjunto de nós de T e R é a relação de pai-filho (ou seja, $(u, v) \in R$ se u é pai de v em T). Os morfismos em **Trees** são mapeados para morfismos em **Struct** que preservam a relação R .

Listing 3.79: Funtor G aplicado a uma árvore

```
1 function S = tree_to_struct(T)
2     S.set = T.nodes;
3     S.rel = T.edges';
4 end
```

Proposição 3.33.1. O funtor G satisfaz as seguintes propriedades:

- $G(\text{id}_T) = \text{id}_{G(T)}$;
- G preserva a composição de morfismos.

3.33.3 Transformações Naturais

Uma transformação natural $\eta : G \Rightarrow H$ entre funtores $G, H : \mathbf{Trees} \rightarrow \mathbf{Struct}$ seria uma família de morfismos $\eta_T : G(T) \rightarrow H(T)$ que comutam com os morfismos em **Trees**. Por exemplo, η_T poderia ser a identidade se $G = H$.

3.34 Funtor $H : \mathbf{End} \rightarrow \mathbf{Struct}$

3.34.1 Descrição das Categorias

Definição 3.34.1 (Categoria **End** (modelo MATLAB/Octave)). Um objeto é um endomorfismo $f : X \rightarrow X$ em **Fin**, representado por um vetor de índices onde $f(i)$ é a imagem

de i . Um morfismo entre endomorfismos (X, f) e (Y, g) é uma função $\phi : X \rightarrow Y$ tal que $\phi \circ f = g \circ \phi$.

Listing 3.80: Objeto e morfismo em **End**

```
1 X = 1:3;
2 f = [2, 3, 1]; % endomorfismo: f(1)=2, f(2)=3, f(3)=1
```

Definição 3.34.2 (Categoria **Struct** (modelo MATLAB/Octave)). Como definido anteriormente.

3.34.2 Funtor Covariante $H : \mathbf{End} \rightarrow \mathbf{Struct}$ (Endomorfismo como Conjunto Estruturado)

O funtor $H : \mathbf{End} \rightarrow \mathbf{Struct}$ envia cada endomorfismo (X, f) a um conjunto estruturado (X, R_f) , onde R_f é a relação binária definida por $(x, y) \in R_f$ se $f(x) = y$. Os morfismos em **End** são mapeados para morfismos em **Struct** que preservam a relação R_f .

Listing 3.81: Funtor H aplicado a um endomorfismo

```
1 function S = end_to_struct(X, f)
2     S.set = X;
3     n = length(X);
4     R = zeros(n, n);
5     for i = 1:n
6         R(i, f(i)) = 1;
7     end
8     S.rel = R;
9 end
```

Proposição 3.34.1. O funtor H satisfaz as seguintes propriedades:

- $H(\text{id}_{(X,f)}) = \text{id}_{H(X,f)}$;
- H preserva a composição de morfismos.

3.34.3 Transformações Naturais

Uma transformação natural $\eta : H \Rightarrow K$ entre funtores $H, K : \mathbf{End} \rightarrow \mathbf{Struct}$ seria uma família de morfismos $\eta_{(X,f)} : H(X, f) \rightarrow K(X, f)$ que comutam com os morfismos em **End**. Por exemplo, $\eta_{(X,f)}$ poderia ser a identidade se $H = K$.

3.35 Funtor $U_1, U_2 : \mathbf{DIMag} \rightarrow \mathbf{Mag}$

3.35.1 Descrição das Categorias

Definição 3.35.1 (Categoria **DIMag** (modelo MATLAB/Octave)). Um objeto é um dimagma finito $(M, *, \circ)$, ou seja, um conjunto finito M com duas operações binárias independentes. Em MATLAB, representa-se por uma **struct** com campos **elements**, **op1** e **op2**.

Listing 3.82: Objeto em **DIMag**

```

1 D.elements = [0, 1, 2];
2 D.op1 = @(x, y) mod(x + y, 3);    % primeira operacao
3 D.op2 = @(x, y) mod(x * y, 3);    % segunda operacao

```

Definição 3.35.2 (Categoria **Mag** (modelo MATLAB/Octave)). Um objeto é um magma finito $(M, *)$, ou seja, um conjunto finito com uma única operação binária, sem axiomas adicionais. Em MATLAB, representa-se por uma **struct** com campos **elements** e **op**.

Listing 3.83: Objeto em **Mag**

```

1 M.elements = [0, 1, 2];
2 M.op = @(x, y) mod(x + y, 3);

```

3.35.2 Funtores Covariantes $U_1, U_2 : DIMag \rightarrow Mag$

Existem dois funtores de esquecimento covariantes naturais:

$$U_1 : \mathbf{DIMag} \rightarrow \mathbf{Mag}, \quad U_1(M, *, \circ) = (M, *),$$

$$U_2 : \mathbf{DIMag} \rightarrow \mathbf{Mag}, \quad U_2(M, *, \circ) = (M, \circ).$$

Cada um esquece uma das duas operações, retraindo apenas a outra. Nos morfismos, ambos atuam como a identidade, uma vez que qualquer homomorfismo de dimagmas que preserve as duas operações também preserva cada uma individualmente.

Listing 3.84: Funtores de esquecimento U_1 e U_2

```

1 function M = forget_first(D)
2     % U1: esquece a segunda operacao, retraindo a primeira
3     M.elements = D.elements;
4     M.op = D.op1;
5 end
6
7 function M = forget_second(D)
8     % U2: esquece a primeira operacao, retraindo a segunda
9     M.elements = D.elements;
10    M.op = D.op2;
11 end

```

3.35.3 Funtor Diagonal $\Delta : Mag \rightarrow DIMag$

Existe também o funtor diagonal, adjunto à direita de U_1 e U_2 :

$$\Delta : \mathbf{Mag} \rightarrow \mathbf{DIMag}, \quad \Delta(M, *) = (M, *, *).$$

Este funtor duplica a operação, colocando a mesma nas duas posições do dimagma resultante.

Listing 3.85: Funtor diagonal $\Delta : \mathbf{Mag} \rightarrow \mathbf{DIMag}$

```

1 function D = diagonal_magma(M)
2     % Delta: duplica a operacao unica em ambas as posicoes
3     D.elements = M.elements;
4     D.op1 = M.op;
5     D.op2 = M.op;
6 end

```

Proposição 3.35.1. Verificam-se os seguintes isomorfismos naturais:

$$U_1 \circ \Delta \cong \text{Id}_{\mathbf{Mag}}, \quad U_2 \circ \Delta \cong \text{Id}_{\mathbf{Mag}}.$$

3.35.4 Transformações Naturais $\eta : U_1 \Rightarrow U_2$

Dados os dois funtores paralelos $U_1, U_2 : \mathbf{DIMag} \rightarrow \mathbf{Mag}$, uma transformação natural $\eta : U_1 \Rightarrow U_2$ é uma família de morfismos em $\mathbf{Mag}$,

$$\eta_{(M, *, \circ)} : (M, *) \rightarrow (M, \circ), \quad \text{para cada } (M, *, \circ) \in \mathbf{DIMag},$$

tal que para cada morfismo $f : (M_1, *_1, \circ_1) \rightarrow (M_2, *_2, \circ_2)$ o seguinte quadrado comuta:

$$U_2(f) \circ \eta_{M_1} = \eta_{M_2} \circ U_1(f).$$

A identidade id_M é uma transformação natural $U_1 \Rightarrow U_2$ se e só se a primeira e a segunda operação coincidem, ou seja, se o dimagma é na realidade um magma (caso Δ).

Listing 3.86: Verificação da condição de naturalidade $\eta : U_1 \Rightarrow U_2$

```

1 function ok = check_nat_trans_dimag_mag(D, eta)
2     % Verifica se eta e morfismo de Mag de U1(D) para U2(D)
3     % i.e., eta(x * y) == eta(x) o eta(y) para todo x,y
4     ok = true;
5     for i = 1:length(D.elements)
6         for j = 1:length(D.elements)
7             x = D.elements(i); y = D.elements(j);
8             lhs = eta(D.op1(x, y));
9             rhs = D.op2(eta(x), eta(y));
10            if lhs ~= rhs, ok = false; return; end
11        end
12    end
13 end

```

3.36 Funtor $I : DLat \hookrightarrow Lat$ e $D : Lat \rightarrow DLat$

3.36.1 Descrição das Categorias

Definição 3.36.1 (Categoria **Lat** (modelo MATLAB/Octave)). Um objeto é um reticulado finito $(L, \wedge, \vee)$, ou seja, um conjunto finito com duas operações binárias de encontro (*meet*) e junção (*join*). Em MATLAB, representa-se por uma **struct** com campos **elements**, **meet** e **join**.

Listing 3.87: Objeto em **Lat**

```

1 L.elements = [0, 1];
2 L.meet = @(x, y) min(x, y); % encontro
3 L.join = @(x, y) max(x, y); % juncao

```

Definição 3.36.2 (Categoria **DLat** (modelo MATLAB/Octave)). Um objeto é um reticulado distributivo finito $(L, \wedge, \vee)$, onde as operações satisfazem adicionalmente a lei distributiva:

$$x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z).$$

A representação em MATLAB é idêntica à de **Lat**, acrescentando a verificação da distributividade.

Listing 3.88: Verificação da distributividade em **DLat**

```

1 function ok = is_distributive(L)
2     ok = true;
3     for x = L.elements
4         for y = L.elements
5             for z = L.elements
6                 lhs = L.meet(x, L.join(y, z));
7                 rhs = L.join(L.meet(x, y), L.meet(x, z));
8                 if lhs ~= rhs, ok = false; return; end
9             end
10        end
11    end
12 end

```

3.36.2 Funtor de Inclusão $I : DLat \hookrightarrow Lat$

Existe o funtor de inclusão covariante:

$$I : DLat \hookrightarrow Lat,$$

que vê cada reticulado distributivo simplesmente como um reticulado, esquecendo a propriedade adicional de distributividade. Nos morfismos, I atua como a identidade: qualquer homomorfismo de reticulados distributivos é também um homomorfismo de reticulados.

Listing 3.89: Funtor de inclusão $I : DLat \hookrightarrow Lat$

```

1 function L = inclusion_DLat_to_Lat(DL)
2     % I: esquece a propriedade de distributividade
3     L.elements = DL.elements;
4     L.meet = DL.meet;
5     L.join = DL.join;
6 end

```

3.36.3 Reflector Distributivo $D : Lat \rightarrow DLat$

Existe também um reflector distributivo:

$$D : \mathbf{Lat} \rightarrow \mathbf{DLat},$$

que envia um reticulado L no seu maior quociente distributivo

$$D(L) = L/\theta_{\text{dist}},$$

onde θ_{dist} é a menor congruência que força a distributividade.

Observação 3.36.1. A restrição de $\mathbf{Lat}$ aos reticulados distributivos não define um funtor em toda a categoria $\mathbf{Lat}$, porque nem todo o reticulado é distributivo. O que existe naturalmente é a inclusão I ou então o quociente reflector D .

3.36.4 Transformações Naturais $q : \text{Id}_{\mathbf{Lat}} \Rightarrow I \circ D$

Existe uma transformação natural

$$q : \text{Id}_{\mathbf{Lat}} \Rightarrow I \circ D,$$

em que a componente q_L é a projeção canónica de L no seu quociente distributivo $D(L)$:

$$q_L : L \twoheadrightarrow L/\theta_{\text{dist}} = D(L).$$

A naturalidade decorre do facto de que, para qualquer morfismo $f : L \rightarrow M$ em $\mathbf{Lat}$, o morfismo $D(f)$ está bem definido no quociente e o diagrama de naturalidade comuta.

Listing 3.90: Projeção canónica $q_L : L \rightarrow D(L)$ (esquema simplificado)

```

1 function [classes, proj] = distributive_quotient(L)
2     % Calcula as classes de equivalencia da menor congruencia
3     % que forza a distributividade.
4     % proj(i) indica o indice da classe do elemento L.elements(i)
5
6     n = length(L.elements);
7     proj = 1:n; % inicialmente, cada elemento e a sua propria
8     classe
9     changed = true;
10    while changed
11        changed = false;
12        for x = 1:n
13            for y = 1:n
14                for z = 1:n
15                    lhs = proj(L.meet(L.elements(x), ...
16                                L.join(L.elements(y), L.elements(z)
17                                    ))) + 1);
18                    rhs_l = proj(L.meet(L.elements(x), L.elements
19                                        (y)) + 1);
20                    rhs_r = proj(L.meet(L.elements(x), L.elements
21                                        (z)) + 1);
22                    rhs = proj(L.join(L.elements(rhs_l - 1), ...
23                                    L.elements(rhs_r - 1)) + 1);

```

```

19         if lhs ~= rhs
20             % fundir as duas classes
21             old = max(lhs, rhs);
22             new = min(lhs, rhs);
23             proj(proj == old) = new;
24             changed = true;
25         end
26     end
27 end
28 end
29 end
30 classes = unique(proj);
31 end

```

3.37 Funtor $Z : Lat \rightarrow Bool$

3.37.1 Descrição das Categorias

Definição 3.37.1 (Categoria **DLat** (modelo MATLAB/Octave)). Os objetos são reticulados distributivos finitos $(L, \wedge, \vee)$, representados por uma **struct** com o conjunto de elementos e as operações de meet e join. Um morfismo $f : L \rightarrow M$ é um homomorfismo de reticulados, preservando meet e join: $f(a \wedge b) = f(a) \wedge f(b)$ e $f(a \vee b) = f(a) \vee f(b)$.

Listing 3.91: Objeto e morfismo em **DLat**

```

1 % Reticulado distributivo: cadeia linear {0,1,2,3}
2 L.elements = [0, 1, 2, 3];
3 L.meet      = @(x, y) min(x, y); % meet (infimo)
4 L.join      = @(x, y) max(x, y); % join (supremo)
5 L.bot       = 0;
6 L.top       = 3;
7
8 % Verificacao de homomorfismo f: L -> M
9 is_hom = all(arrayfun(@(a, b) ...
10     M.meet(f(a), f(b)) == f(L.meet(a, b)) && ...
11     M.join(f(a), f(b)) == f(L.join(a, b)), ...
12     L.elements, L.elements));

```

Definição 3.37.2 (Categoria **Bool** (modelo MATLAB/Octave)). Os objetos são álgebras booleanas finitas $(B, \wedge, \vee, \neg, \perp, \top)$, que acrescentam ao reticulado a operação de complemento e os elementos distinguidos $\perp$ e $\top$. Um morfismo preserva todas as operações, incluindo o complemento.

Listing 3.92: Objeto e morfismo em **Bool**

```

1 % Algebra booleana: 2^2 = {00,01,10,11} representada por
  % {0,1,2,3}
2 B.elements = [0, 1, 2, 3];
3 B.meet      = @(x, y) bitand(x, y);
4 B.join      = @(x, y) bitor(x, y);

```

```

5 B.compl      = @(x) bitxor(x, 3);    % complemento bit a bit em 2
   bits
6 B.bot        = 0;
7 B.top        = 3;

```

3.37.2 Funtor Covariante $Z : DLat \rightarrow Bool$ (Centro Booleano)

O funtor $Z : DLat \rightarrow Bool$ envia cada reticulado distributivo limitado L ao seu *centro booleano*, o subconjunto dos elementos complementados,

$$Z(L) = \{a \in L \mid \exists a' \in L : a \wedge a' = \perp \text{ e } a \vee a' = \top\}.$$

Em reticulados distributivos, $Z(L)$ é sempre uma subálgebra booleana de L (com complemento único). Cada morfismo $f : L \rightarrow M$ em **DLat** induz $Z(f) = f|_{Z(L)} : Z(L) \rightarrow Z(M)$, pois homomorfismos de reticulados preservam a propriedade de ser complementado.

Listing 3.93: Funtor Z — extração do centro booleano de um reticulado distributivo

```

1 function B = center_boolean(L)
2     complemented = [];
3     for a = L.elements
4         for b = L.elements
5             if L.meet(a,b) == L.bot && L.join(a,b) == L.top
6                 complemented = [complemented, a];
7                 break;
8             end
9         end
10    end
11    B.elements = unique(complemented);
12    B.meet      = L.meet;
13    B.join      = L.join;
14    B.compl     = @(a) find_complement(L, a);
15    B.bot       = L.bot;
16    B.top       = L.top;
17 end
18
19 function c = find_complement(L, a)
20     % Complemento unico em reticulados distributivos
21     for b = L.elements
22         if L.meet(a,b) == L.bot && L.join(a,b) == L.top
23             c = b; return;
24         end
25     end
26     c = NaN;
27 end

```

Observação 3.37.1. Em reticulados distributivos, cada elemento tem no máximo um complemento, pelo que $Z(L)$ é bem definida e $Z(f)$ está bem definido como restrição de f ao centro booleano.

3.37.3 Funtor de Esquecimento $J : \mathbf{Bool} \rightarrow \mathbf{DLat}$

O funtor de esquecimento $J : \mathbf{Bool} \rightarrow \mathbf{DLat}$ envia cada álgebra booleana $(B, \wedge, \vee, \neg, \perp, \top)$ ao reticulado distributivo subjacente $(B, \wedge, \vee)$, ignorando o complemento. Cada morfismo booleano é automaticamente um morfismo de reticulados, pelo que J está bem definido.

Listing 3.94: Funtor de esquecimento $J : \mathbf{Bool} \rightarrow \mathbf{DLat}$

```

1 function L = forget_bool_to_dlat(B)
2     L.elements = B.elements;
3     L.meet      = B.meet;
4     L.join      = B.join;
5     L.bot       = B.bot;
6     L.top       = B.top;
7     % complemento e esquecido
8 end

```

3.37.4 Transformações Naturais $\iota : J \circ Z \Rightarrow \text{Id}_{\mathbf{DLat}}$

A transformação natural canónica entre os dois funtores paralelos é

$$\iota : J \circ Z \Rightarrow \text{Id}_{\mathbf{DLat}},$$

cujas componentes são as inclusões do centro booleano no reticulado original,

$$\iota_L : J(Z(L)) \hookrightarrow L, \quad a \mapsto a.$$

A condição de naturalidade exige que, para todo morfismo $f : L \rightarrow M$ em $\mathbf{DLat}$,

$$f \circ \iota_L = \iota_M \circ J(Z(f)).$$

Como ι é a inclusão, isto reduz-se a verificar que f envia o centro booleano de L para dentro do centro booleano de M .

Listing 3.95: Verificação da naturalidade de ι para um morfismo $f : L \rightarrow M$

```

1 function ok = check_iota_naturality(L, M, f)
2     ZL = center_boolean(L);
3     ZM = center_boolean(M);
4
5     % Naturalidade: f(a) pertence a Z(M) para todo a em Z(L)
6     ok = true;
7     for a = ZL.elements
8         if ~ismember(f(a), ZM.elements)
9             ok = false; return;
10        end
11    end
12    % Se ok=true, iota e natural: elementos complementados de L
13    % sao enviados em elementos complementados de M
14 end

```

3.38 Funtor $F : \mathbf{TuringMachines} \rightarrow \mathbf{Struct}$

3.38.1 Fundamentação Teórica

O funtor $F : \mathbf{TuringMachines} \rightarrow \mathbf{Struct}$ codifica uma máquina de Turing como um conjunto estruturado, capturando a dinâmica de transição de estados através de um endomapa e distinguindo os estados relevantes por meio de um mapa para os números complexos.

3.38.2 Funtor Covariante $F : \mathbf{TuringMachines} \rightarrow \mathbf{Struct}$

Definição 3.38.1 (Funtor covariante $F : \mathbf{TuringMachines} \rightarrow \mathbf{Struct}$). Dada uma máquina de Turing $M = (Q, \Gamma, \delta, q_0, F_{\text{acc}}, B)$ com $|Q| = m$, define-se o triplo (d, φ, m) :

- $\varphi(q) = \delta(q, B)_1$ — estado de transição ao ler o símbolo em branco, capturando a dinâmica sob um símbolo fixo;
- $d(q) = \mathbf{1}_{q \in F_{\text{acc}}} + \mathbf{i} \cdot \mathbf{1}_{q=q_0}$, distinguindo os quatro tipos de estados em $\{0, 1, \mathbf{i}, 1+\mathbf{i}\}$.

Dado $(f_Q, f_\Gamma) : M_1 \rightarrow M_2$, define-se $F(f_Q, f_\Gamma) = f_Q$.

Proposição 3.38.1. F é covariante e bem definido: $f_\Gamma(B_1) = B_2$ implica $f_Q \circ \varphi_1 = \varphi_2 \circ f_Q$; a preservação de q_0 e F_{acc} garante $d_1 = d_2 \circ f_Q$.

Listing 3.96: Funtor covariante $F : \mathbf{TuringMachines} \rightarrow \mathbf{Struct}$

```

1 function S = functor_TM_to_Struct(M)
2     S.n = M.Q;
3     S.phi = M.delta(:, M.blank, 1)'; % phi(q) = delta(q, blank)
4     S.d = zeros(1, M.Q, 'like', 1i);
5     for q = 1:M.Q
6         S.d(q) = double(ismember(q, M.F)) + 1i*double(q == M.q0);
7     end
8 end
9
10 function fS = functor_TM_to_Struct_morphism(fQ)
11     fS = fQ;
12 end

```

3.38.3 Funtor Contravariante — Análise de Existência

Proposição 3.38.2 (Inexistência de functor contravariante não trivial). Não existe functor contravariante canônico $G : \mathbf{TuringMachines} \rightarrow \mathbf{Struct}$. Um morfismo $(f_Q, f_\Gamma) : M_1 \rightarrow M_2$ induziria $G(f_Q, f_\Gamma) : G(M_2) \rightarrow G(M_1)$, ou seja, um morfismo de $\mathbf{Struct}$ com $g \circ \varphi_2 = \varphi_1 \circ g$ e $d_2 = d_1 \circ g$. A condição sobre d requereria $g(q_0^{(2)}) = q_0^{(1)}$ e $g(F_2) \subseteq F_1$, que é exatamente uma condição de morfismo “inverso” em $\mathbf{TM}$. Tal mapa g só existe canonicamente quando f_Q é invertível.

Observação 3.38.1. Na subcategoria dos isomorfismos de $\mathbf{TuringMachines}$, $G(f_Q, f_\Gamma) = F(f_Q^{-1}, f_\Gamma^{-1})$ define um functor contravariante. No caso geral, só existe o functor contravariante constante $G(M) = S_0$, $G(f) = \text{id}_{S_0}$.

3.38.4 Transformações Naturais

Seja $G : \mathbf{Turing\ Machines} \rightarrow \mathbf{Struct}$ um segundo functor que usa um símbolo de referência diferente $a^{**} \neq a^*$ para definir o endomapa $\psi(q) = \delta(q, a^{**})_1$, mas o mesmo mapa d . Então existe uma transformação natural $\eta : F \Rightarrow G$ cujas componentes são:

$$\eta_M : F(M) \rightarrow G(M), \quad \eta_M = \text{id}_{\{1, \dots, m\}},$$

desde que φ e ψ coincidam (o que acontece se $\delta(q, a^*) = \delta(q, a^{**})$ para todo q). No caso geral, η_M é a identidade mas deixa de ser morfismo de **Struct**, pelo que a transformação natural genuína requer que os dois funtores partilhem o mesmo símbolo de referência.

Uma transformação natural sempre definida é $\varepsilon : F \Rightarrow C_{S_0}$, onde C_{S_0} é o functor constante no objeto trivial $S_0 = (0, \text{id}, 1)$, com ε_M o único morfismo de **Struct** para S_0 .

Listing 3.97: Transformação natural $\eta : F \Rightarrow G$ (mudança de símbolo de referência)

```

1 function ok = check_natural_transformation_TM_Struct(M, sym1,
2   sym2)
3   % Verifica se id_Q e morfismo de Struct entre F(M) usando
4   % e G(M) usando sym2, i.e., se phi1 == phi2
5   phi1 = zeros(1, M.Q);
6   phi2 = zeros(1, M.Q);
7   for q = 1:M.Q
8       phi1(q) = M.delta(q, sym1, 1);
9       phi2(q) = M.delta(q, sym2, 1);
10  end
11  ok = isequal(phi1, phi2);
12  % Se ok=true, eta_M = identidade e transformacao natural
   existe
end

```

3.39 Funtor $F : \mathbf{End} \rightarrow \mathbf{Mon}$

3.39.1 Fundamentação Teórica

O funtor $F : \mathbf{End} \rightarrow \mathbf{Mon}$ associa a cada endomapa φ o monoide das suas iterações distintas, capturando a estrutura algébrica do comportamento dinâmico de φ a longo prazo. Como X é finito, a sequência de iteradas estabiliza (torna-se eventualmente periódica), pelo que o monoide é sempre finito.

3.39.2 Funtor Covariante $F : \mathbf{End} \rightarrow \mathbf{Mon}$

Definição 3.39.1 (Funtor covariante $F : \mathbf{End} \rightarrow \mathbf{Mon}$). Dado $(X, \varphi) \in \mathbf{End}$, define-se o monoide das iterações distintas:

$$F(X, \varphi) = (\{\varphi^0, \varphi^1, \dots\}, \circ, \varphi^0).$$

Como X é finito, a sequência estabiliza (é eventualmente periódica). Dado $f : (X_1, \varphi_1) \rightarrow (X_2, \varphi_2)$, define-se $F(f)(\varphi_1^k) = \varphi_2^k$.

Proposição 3.39.1. F é covariante e bem definido: $f \circ \varphi_1^k = \varphi_2^k \circ f$ (por indução em k) garante que $F(f)$ é homomorfismo de monoides, preservando composição e neutro.

Listing 3.98: Funtor covariante $F : \mathbf{End} \rightarrow \mathbf{Mon}$

```

1 function Mon = functor_End_to_Mon(phi, n)
2     iterates = {};
3     cur = 1:n;          % phi^0 = identidade
4     while true
5         found = any(cellfun(@(x) isequal(x, cur), iterates));
6         if found, break; end
7         iterates{end+1} = cur;
8         cur = phi(cur);
9     end
10    m = numel(iterates);
11    Mon.elements = iterates;
12    Mon.identity = 1;
13    Mon.mult = zeros(m, m);
14    for i = 1:m
15        for j = 1:m
16            comp = iterates{j}(iterates{i});
17            for r = 1:m
18                if isequal(iterates{r}, comp)
19                    Mon.mult(i,j) = r; break;
20                end
21            end
22        end
23    end
24 end

```

3.39.3 Funtor Contravariante — Análise de Existência

Proposição 3.39.2 (Inexistência de functor contravariante canónico). Não existe functor contravariante canónico $G : \mathbf{End} \rightarrow \mathbf{Mon}$. Um morfismo $f : (X_1, \varphi_1) \rightarrow (X_2, \varphi_2)$ deveria induzir $G(f) : F(X_2, \varphi_2) \rightarrow F(X_1, \varphi_1)$, ou seja, um homomorfismo $\varphi_2^k \mapsto \varphi_1^k$. Mas $f \circ \varphi_1^k = \varphi_2^k \circ f$ não fornece um mapa inverso canónico entre os monoides (salvo se f for bijetivo).

Observação 3.39.1. Na subcategoria dos isomorfismos de $\mathbf{End}$, $G(f) = F(f^{-1})$ define um functor contravariante bem definido, pois $f^{-1} \circ \varphi_2^k = \varphi_1^k \circ f^{-1}$.

3.39.4 Transformações Naturais

Translação por k_0 iterações

Para $k_0 \geq 0$ fixo, a família de homomorfismos $\eta_{(X, \varphi)}^{(k_0)} : \varphi^k \mapsto \varphi^{k+k_0}$ define uma transformação natural $\eta^{(k_0)} : F \Rightarrow F$. A naturalidade segue de $F(f)(\varphi_1^{k+k_0}) = \varphi_2^{k+k_0}$.

Adjunção com $U : \mathbf{Mon} \rightarrow \mathbf{End}$

Existe uma bijeção natural $\mathbf{Mon}(F(X, \varphi), N) \cong \mathbf{End}((X, \varphi), U(N))$, onde U envia $(M, \cdot, e)$ no endomapa de multiplicação à esquerda por um gerador. Isto significa que F é adjunto à esquerda de U , e a unidade desta adjunção é uma transformação natural $\eta : \text{Id}_{\mathbf{End}} \Rightarrow U \circ F$ com $\eta_{(X, \varphi)}(x) = \varphi$ (o gerador).

Listing 3.99: Transformação natural $\eta^{(k_0)} : F \Rightarrow F$ (translação por k_0)

```

1 function eta_comp = apply_eta_Mon(Mon, k0)
2     % eta^(k0)(phi^k) = phi^(k+k0) via composicao no monoide
3     m = size(Mon.mult, 1);
4     gen = 2; % indice de phi^1 (phi^0=1, phi^1=2, ...)
5     pk0 = Mon.identity;
6     for i = 1:k0
7         pk0 = Mon.mult(pk0, gen);
8     end
9     eta_comp = zeros(1, m);
10    for k = 1:m
11        eta_comp(k) = Mon.mult(k, pk0);
12    end
13 end

```

Listing 3.100: Unidade da adjunção $\eta : \text{Id}_{\mathbf{End}} \Rightarrow U \circ F$ — componente em (X, φ)

```

1 function eta_x = unit_adjunction_End_Mon(phi, n)
2     % eta_(X,phi)(x) = indice de phi^1 no monoide F(X,phi)
3     % i.e., o gerador phi visto como elemento do monoide de
4     % iteracoes
5     Mon = functor_End_to_Mon(phi, n);
6     % 0 gerador e phi^1, que tem indice 2 por convencao (phi^0=1,
7     % phi^1=2)
8     gen_idx = 2;
9     % eta e o mapa constante que envia qualquer x em phi (o
10    % gerador)
11    eta_x = repmat(gen_idx, 1, n);
12    % Verificacao: U(F(X,phi)) e o endomapa "multiplicar por
13    % gen_idx"
14    % i.e., U(Mon)(k) = Mon.mult(k, gen_idx) para todo k no
15    % monoide
16    U_phi = zeros(1, numel(Mon.elements));
17    for k = 1:numel(Mon.elements)
18        U_phi(k) = Mon.mult(k, gen_idx);
19    end
20 end

```

3.40 Funtor $F : \mathbf{End} \rightarrow \mathbf{Trees}$

3.40.1 Fundamentação Teórica

O funtor $F : \mathbf{End} \rightarrow \mathbf{Trees}$ constrói, a partir de uma endomapa $\varphi : X \rightarrow X$, uma árvore binária de antecessores: cada elemento $x \in X$ tem como pai $\varphi(x)$ (se x não for ponto fixo),

ou a raiz virtual, caso contrário. Este funtor é definido na subcategoria de endomapas com grau de entrada no máximo 2 (condição necessária para a estrutura binária).

3.40.2 Funtor Covariante $F : \mathbf{End} \rightarrow \mathbf{Trees}$

Definição 3.40.1 (Funtor covariante $F : \mathbf{End} \rightarrow \mathbf{Trees}$). Dado $(X, \varphi) \in \mathbf{End}$ com $X = \{1, \dots, n\}$, constrói-se a *árvore de antecessores* de φ : o pai de x é $\varphi(x)$ se $\varphi(x) \neq x$, e a raiz virtual $r = n+1$ se x for ponto fixo. Os irmãos são ordenados por índice crescente (no máximo dois por nó). Dado $f : (X_1, \varphi_1) \rightarrow (X_2, \varphi_2)$, o morfismo induzido é $F(f) = [f, r_2]$.

Observação 3.40.1. Se $|\varphi^{-1}(x)| > 2$ para algum x , a estrutura não é binária. O functor é então definido na subcategoria de endomapas com grau de entrada ≤ 2 .

Listing 3.101: Funtor covariante $F : \mathbf{End} \rightarrow \mathbf{Trees}$

```

1 function T = functor_End_to_Trees(phi, n)
2     parent_arr      = phi;
3     fixed           = find(phi == 1:n);
4     parent_arr(fixed) = n+1;
5     parent_arr(n+1)  = 0;
6     side_arr = -ones(1, n+1);
7     for p = 1:(n+1)
8         ch = sort(find(parent_arr == p));
9         if numel(ch) >= 1, side_arr(ch(1)) = 0; end
10        if numel(ch) >= 2, side_arr(ch(2)) = 1; end
11    end
12    T.n = n+1; T.parent = parent_arr; T.side = side_arr;
13 end
14
15 function Tf = functor_End_to_Trees_morphism(f, n2)
16     Tf = [f, n2+1];
17 end

```

3.40.3 Funtor Contravariante — Análise de Existência

Proposição 3.40.1 (Inexistência de functor contravariante canónico). Não existe functor contravariante canónico $G : \mathbf{End} \rightarrow \mathbf{Trees}$. Um morfismo $f : (X_1, \varphi_1) \rightarrow (X_2, \varphi_2)$ deveria induzir $G(f) : F(X_2, \varphi_2) \rightarrow F(X_1, \varphi_1)$, ou seja, um morfismo de árvores que inverte a orientação dos morfismos de $\mathbf{End}$. Tal mapa exige uma secção $g : X_2 \rightarrow X_1$ com $g \circ \varphi_2 = \varphi_1 \circ g$, que só existe canonicamente quando f é bijetivo.

3.40.4 Transformações Naturais

Translação por k_0 iterações

Para $k_0 \geq 1$, seja $F^{(k_0)}$ o functor que usa φ^{k_0} como endomapa de referência. A transformação natural $\eta^{(k_0)} : F \Rightarrow F^{(k_0)}$ tem componentes $\eta_{(X, \varphi)}^{(k_0)} = \text{id}_{X \cup \{r\}}$. Esta é natural porque $f \circ \varphi_1^{k_0} = \varphi_2^{k_0} \circ f$ (por indução), logo $F^{(k_0)}(f) = F(f)$.

Adjunção com $U : \mathbf{Trees} \rightarrow \mathbf{End}$

O functor U extrai o endomapa pai de uma árvore. A unidade da adjunção $\text{Id}_{\mathbf{End}} \Rightarrow U \circ F$ tem componentes id_X , pois por construção $F(X, \varphi)$ tem exatamente φ como endomapa pai (nos não-raízes).

Listing 3.102: Verificação da naturalidade $\eta^{(k_0)} : F \Rightarrow F^{(k_0)}$

```

1 function ok = check_iter_nat_trees(phi, n, k0)
2     % Verifica que id_(n+1) e morfismo Trees de F(phi) para F(phi
   ^k0)
3     T1 = functor_End_to_Trees(phi, n);
4     phik = phi;
5     for i = 1:k0-1, phik = phi(phik); end
6     T2 = functor_End_to_Trees(phik, n);
7     id = 1:T1.n;
8     ok = is_tree_morphism(T1, T2, id);
9 end

```

3.41 Funtor $F : \mathbf{End} \rightarrow \mathbf{TuringMachines}$

3.41.1 Fundamentação Teórica

O funtor $F : \mathbf{End} \rightarrow \mathbf{TuringMachines}$ constrói, a partir de uma endomapa $\varphi : X \rightarrow X$, uma máquina de Turing simplificada cujos estados correspondem aos elementos de X e cuja função de transição é ditada por φ . A construção preserva a estrutura de morfismo: um mapa que comuta com as endomapas induz um par de mapas de estados e de fita compatível com as respetivas funções de transição.

3.41.2 Funtor Covariante $F : \mathbf{End} \rightarrow \mathbf{TuringMachines}$

Definição 3.41.1 (Funtor covariante $F : \mathbf{End} \rightarrow \mathbf{TuringMachines}$). Dado $(X, \varphi) \in \mathbf{End}$ com $X = \{1, \dots, n\}$, constrói-se a máquina de Turing $F(X, \varphi) = (Q, \Gamma, \delta, q_0, F_{\text{acc}}, B)$ da seguinte forma:

- $Q = X \cup \{n+1\}$ — os estados são os elementos de X mais um estado de paragem $q_{\text{halt}} = n+1$;
- $\Gamma = X \cup \{n+1\}$ — o alfabeto de fita são os elementos de X mais o símbolo em branco $B = n+1$;
- $q_0 = 1$ (estado inicial), $F_{\text{acc}} = \{n+1\}$ (único estado de aceitação);
- Função de transição: no estado $q \in X$, ao ler o símbolo $a \in X$, a máquina escreve $\varphi(a)$, move para a direita, e transita para $\varphi(q)$:

$$\delta(q, a) = (\varphi(q), \varphi(a), R), \quad q, a \in X;$$

ao ler B em qualquer estado, transita para q_{halt} : $\delta(q, B) = (n+1, B, N)$.

Dado um morfismo $f : (X_1, \varphi_1) \rightarrow (X_2, \varphi_2)$ em $\mathbf{End}$, o morfismo induzido é $F(f) = (f_Q, f_\Gamma)$ com $f_Q = [f, n_2+1]$ e $f_\Gamma = [f, n_2+1]$ (os estados e símbolos em X_1 mapeiam via f ; $q_{\text{halt}}^{(1)}$ mapeia para $q_{\text{halt}}^{(2)}$; B_1 mapeia para B_2).

Proposição 3.41.1. F é covariante: $f \circ \varphi_1 = \varphi_2 \circ f$ implica

$$\delta_2(f_Q(q), f_\Gamma(a)) = (f_Q(\varphi_1(q)), f_\Gamma(\varphi_1(a)), R),$$

que é exatamente a condição de morfismo de **Turing Machines**.

Listing 3.103: Funtor covariante $F : \mathbf{End} \rightarrow \mathbf{Turing\ Machines}$

```

1 function M = functor_End_to_TM(phi, n)
2     M.Q = n+1; M.Gamma = 1:(n+1); M.blank = n+1;
3     M.q0 = 1; M.F = n+1;
4     M.delta = zeros(n+1, n+1, 3);
5     for q = 1:n
6         for a = 1:n
7             M.delta(q,a,:) = [phi(q), phi(a), 1];
8         end
9         M.delta(q,n+1,:) = [n+1, n+1, 0];
10    end
11    for a = 1:(n+1)
12        M.delta(n+1,a,:) = [n+1, a, 0];
13    end
14 end
15
16 function [fQ, fG] = functor_End_to_TM_morphism(f, n2)
17     fQ = [f, n2+1]; fG = [f, n2+1];
18 end

```

3.41.3 Funtor Contravariante — Análise de Existência

Proposição 3.41.2 (Inexistência de functor contravariante canônico). Não existe functor contravariante canônico $G : \mathbf{End} \rightarrow \mathbf{Turing\ Machines}$. Para que $G(f) : F(X_2, \varphi_2) \rightarrow F(X_1, \varphi_1)$ fosse morfismo de **TM**, seria necessário um par (g_Q, g_Γ) com $g_Q \circ \varphi_2 = \varphi_1 \circ g_Q$, ou seja um morfismo de **End** “inverso” de f , inexistente em geral. Na subcategoria dos isomorfismos, $G(f) = F(f^{-1})$ é válido.

3.41.4 Transformações Naturais

Translação por k_0 iterações

Para $k_0 \geq 1$, seja $F^{(k_0)}$ o functor que usa φ^{k_0} . A transformação natural $\eta^{(k_0)} : F \Rightarrow F^{(k_0)}$ tem componentes $\eta_{(X, \varphi)}^{(k_0)} = (\text{id}_Q, \text{id}_\Gamma)$ e é natural porque $f \circ \varphi_1^{k_0} = \varphi_2^{k_0} \circ f$.

Composto com $F_{\mathbf{TM} \rightarrow \mathbf{Struct}}$

O composto $F_{\mathbf{TM} \rightarrow \mathbf{Struct}} \circ F : \mathbf{End} \rightarrow \mathbf{Struct}$ envia (X, φ) em $(\mathbf{d}_\varphi, \varphi, n+1)$. Existe uma transformação natural entre este composto e o functor de imersão $\iota : \mathbf{End} \rightarrow \mathbf{Struct}$ dado por $\iota(X, \varphi) = (0, \varphi, n)$, com componentes dadas pela inclusão $X \hookrightarrow X \cup \{q_{\text{halt}}\}$.

Listing 3.104: Verificação da naturalidade $\eta^{(k_0)} : F \Rightarrow F^{(k_0)}$ em $\mathbf{End} \rightarrow \mathbf{TM}$

```

1 function ok = check_TM_nat_trans(phi, n, k0)

```

```

2      % Verifica que id_(n+1) e morfismo TM de F(phi) para F(phi^k0
      )
3      M1 = functor_End_to_TM(phi, n);
4      phik = phi;
5      for i = 1:k0-1, phik = phi(phik); end
6      M2 = functor_End_to_TM(phik, n);
7      fQ = 1:(n+1); fGam = 1:(n+1);
8      ok = is_TM_morphism(M1, M2, fQ, fGam);
9  end

```

Listing 3.105: Componente da transformação natural $\iota \Rightarrow F_{\text{TM} \rightarrow \text{Struct}} \circ F$ em (X, φ)

```

1  function [S_iota, S_comp, inc] = nat_trans_iota_to_comp(phi, n)
2      % Calcula os dois objectos de Struct e a componente (inclusao
      )
3      % iota(X, phi) = (0, phi, n)
4      S_iota.n = n;
5      S_iota.phi = phi;
6      S_iota.d = zeros(1, n, 'like', 1i);
7
8      % (F_TM->Struct o F)(X, phi) = functor_TM_to_Struct(
      functor_End_to_TM(phi, n))
9      M = functor_End_to_TM(phi, n);
10     S_comp = functor_TM_to_Struct(M);
11
12     % Componente: inclusao X -> X U {q_halt} (i.e., 1:n -> 1:n)
13     inc = 1:n; % cada x in X mapeia para o mesmo indice em
      S_comp
14
15     % Verificacao: inc e morfismo de Struct de S_iota para S_comp
16     ok_phi = all(inc(phi) == S_comp.phi(inc)); % inc o phi =
      phi_comp o inc
17     ok_d = all(S_iota.d == S_comp.d(inc)); % d_iota =
      d_comp o inc
18     fprintf('inc e morfismo de Struct: phi ok=%d, d ok=%d\n',
      ok_phi, ok_d);
19 end

```